Éléments de programmation système (Télécom — ETE7-IFON1)

Part. Un — Cours et exercices

Année 2026 — 2027

Guillaume Mercier

Copyright © 2026 Guillaume Mercier
La diffusion —même partielle— de ce document, quel qu'en soit le format (physique ou non) est interdite sans l'accord de son auteur. Les illustrations sont © de leurs auteurs respectifs.



Table des matières

1 **Premier programme** 13

2 **Utilisation des fichiers** 15

2.1 **Notion de Système de fichiers** 15

2.2 **Structures du système pour la gestion des fichiers** 16

2.2.1 Structure d'I-Nœud 16

2.2.2 Structure de descripteur de fichier 17

2.2.3 Structure de fichier ouvert 18

2.3 **Nommage et partage de données** 18

2.4 **Interface pratique** 19

2.4.1 Appels-système 19

2.4.2 Appels de la librairie *libc* 29

2.4.3 Conversions entre flux et (index de) descripteurs 31

2.4.4 Pour aller plus loin : accès aux caractéristiques d'un fichier 32

2.5 **Exercices** 33

2.6 **Travaux Pratiques** 33

3 **Du programme au processus** 35

3.1 **Exécution d'un programme** 35

3.2 **Identification d'un processus** 36

3.2.1 Obtenir le PID d'un processus 37

3.2.2 Pour aller plus loin : identification des processus via un descripteur de fichier (Non POSIX, spécifique à Linux) 37

3.3 **Création de processus** 38

3.4 **Identifier le parent d'un processus** 41

3.5 Héritage inter-processus 41
3.6 Modèle *fork/join* 43
3.7 Terminaison d'un processus 45
3.8 États d'un processus 47
3.9 Recouvrement 48
3.10 Pour aller plus loin : exécution de commandes shell dans un programme 51

4 Communication inter-flots . . . 53
4.1 Tubes 53
4.1.1 Caractéristiques . . . 53
4.1.2 Création et accès à un tube . . . 54
4.1.3 Lecture dans un tube . . . 56
4.1.4 Écriture dans un tube . . . 57
4.1.5 Pour aller plus loin : exécution/interaction de/avec une commande shell . . . 58
4.1.6 Exercices sur les tubes . . . 58
4.2 Mémoire partagée 59
4.2.1 Interface IPC System V . . . 59
4.2.2 Projection de fichiers en mémoire . . . 59
4.2.3 Interface POSIX . . . 62
4.2.4 Partage de données (variables) . . . 64
4.2.5 Exercices . . . 65
4.3 Autre mécanismes de communication 65
4.3.1 Les sockets . . . 65
4.3.2 Les fichiers réguliers . . . 66
4.3.3 Les signaux . . . 66

5 Interlude non-déterministe . . . 67

6 Notification d'événements : les signaux . . . 69
6.1 Gestion des événements par interruption 69
6.2 Principes de fonctionnement des signaux 69
6.2.1 États d'un signal . . . 71
6.2.2 Délivrance d'un signal . . . 72
6.2.3 Structures de données mises en œuvre dans le noyau . . . 72
6.3 Émission et réception d'un signal 74
6.3.1 Fonctions d'émission . . . 74
6.3.2 Réception . . . 76
6.4 Traitants de signaux 76
6.4.1 Mise en place . . . 76
6.4.2 Règles d'écriture d'un traitant de signal . . . 79
6.4.3 Interactions entre appels-système et traitants de signaux . . . 81
6.4.4 Le traitant étendu . . . 82

6.5 Contrôle des signaux avec les masques 83
6.6 Points Divers 85
6.7 Pour aller plus loin : gestion des signaux *sans* les traitants. 86
6.7.1 Réception explicite synchrone sans traitant . . . 86
6.7.2 Gestion des signaux via des descripteurs de fichiers (Non-POSIX, spécifique à Linux). . . . 87
6.8 Travail dirigé : gestion des zombies avec les signaux 89
6.8.1 Récupération des zombies et traitant de signal . . . 89
6.8.2 Et maintenant, du côté du `main` . . . 89

7 Splendeur et tourments du multithreading . . . 91
7.1 Notions de multithreading 91
7.1.1 Vers toujours plus de concurrence : les threads . . . 91
7.2 L' interface Pthread 94
7.2.1 Identification . . . 94
7.2.2 Création de thread . . . 95
7.2.3 Terminaison d'un thread . . . 97
7.2.4 Synchronisation de threads . . . 98
7.2.5 Annulation de thread . . . 99
7.2.6 Accès concurrents à un fichier . . . 100
7.3 Concepts avancés 101
7.3.1 Gestion des *Thread Specific Data* . . . 101
7.3.2 MT-safe vs Async-signal-safe . . . 101
7.3.3 Threads et signaux . . . 102

8 Synchronisation inter-flots . . . 105
8.1 Les sémaphores 106
8.1.1 Fonctionnement théorique . . . 106
8.1.2 Interface pratique . . . 107
8.2 Les verrous 110
8.2.1 Fonctionnement théorique . . . 110
8.2.2 Interface pratique . . . 112
8.2.3 Cas particulier des fichiers . . . 114
8.2.4 Pour aller plus loin : retour sur la duplication de processus multithreadés . . . 115
8.3 Les variables de condition 116
8.3.1 Fonctionnement théorique . . . 116
8.3.2 Interface pratique . . . 117
8.3.3 Implémentation d'un moniteur par l'exemple . . . 119
8.4 Les barrières de synchronisation 121
8.4.1 Fonctionnement théorique . . . 121
8.4.2 Interface pratique . . . 122
8.5 Pour aller (vraiment) plus loin : synchronisation et variables atomiques 123
8.5.1 Rappels sur l'atomicité dans les systèmes d'exploitation . . . 123
8.5.2 Fonctionnement théorique (cas du langage C11 et +) . . . 124
8.5.3 Les variables atomiques en pratique . . . 129
8.6 Compléments et exercices 131

Références bibliographiques 133
Notes de lecture 135
Index 139



Table des figures

2.1 Structures d'implémentation du système de fichiers 16
2.2 Lien entre ouvertures de fichiers et structures 22
2.3 Lien entre duplication de descripteurs et structures 28

3.1 Ressources d'un processus 36
3.2 Duplication d'un processus 39
3.3 Zones mémoire d'un processus 42
3.4 Le modèle *fork/join*. 43
3.5 États d'un processus 47

4.1 Private v.s shared 63
4.2 Variables partagées 65

6.1 Fonctionnement schématique d'un signal 70
6.2 Vecteurs de gestion des signaux 73

7.1 Processus multithreadé 92
7.2 `fork` vs. `pthread_create` 96
7.3 Duplication d'un processus multithreadé 97



Avant-propos

Organisation de l'enseignement

Ce cours d'ETE7-IFON1 se situe dans la continuité des cours de Systèmes d'exploitation (ETE5-IFON1) et de Programmation impérative (ETE5-PROG1). Il est complémentaire du cours de Programmation réseau (ETE7-RESA1) et ces deux enseignements seront utilisés et mis en pratique conjointement lors du Projet Système et Réseau (ETE7-PROJ1). Enfin, ETE7-IFON1 sera complété par le cours d'ETE8-IFON1 (Systèmes d'exploitation) du S8.

Ce support est destiné au cours de *Programmation Système* (ETE7-IFON1). Il a été rédigé dans le but de gagner du temps pendant les séances de cours, afin de vous offrir plus d'occasions pour programmer et manipuler les concepts que vous allez découvrir cette année. Bien entendu, il n'a en aucun cas été rédigé pour vous permettre de ne *plus* assister aux cours. Il ne s'agit que d'un complément et n'est pas suffisant par lui-même.

Objectifs du cours d'ETE7-IFON1

Le principal objectif de ce cours est de vous appendre les rudiments de la programmation système, c'est-à-dire d'écrire des programmes (en langage C) qui vont utiliser de façon explicite et intentionnelle les fonctionnalités offertes par le système d'exploitation. Par exemple, comment lire et écrire à partir ou dans un fichier sur le disque dur, comment faire communiquer des programmes entre-eux, etc.

Dans ce cours, nous n'utiliserons que des systèmes de type UNIX, mais la majorité des concepts que vous allez découvrir et manipuler sont applicables dans d'autres systèmes, car les fonctionnalités sont la plupart du temps existantes. En pratique, il faudra faire attention à la mise en place, à la syntaxe et à la sémantique qui peuvent varier d'un système à l'autre.

De plus, nous essayerons autant que faire se peut de n'utiliser que des fonctions conformes au standard POSIX (**P**ortable **O**perating **S**ystem **I**nterfaces for Uni**X**).

Pré-requis

Les pré-requis scolaires pour ce cours sont les suivants :

1. un niveau correct de programmation en langage C ;
2. des notions de base de systèmes d'exploitation.

Les autres pré-requis sont les suivants :

1. un cerveau en état de marche ;
2. un minimum d'envie d'apprendre des choses ;
3. un minimum d'implication dans ce cours.

Outils et logiciels

En ce qui concerne les outils et logiciels dont vous aurez besoin pour ce cours, en voici une liste raisonnable :

— un éditeur de texte
— un compilateur (gcc *a priori*)
— des terminaux pour lancer vos programmes
— et surtout : le manuel (`man`). Le manuel sera votre meilleur ami ce semestre. Et je vous rappelle que cela n'a **aucun** sens de chercher sur internet, parce que l'information que vous allez récupérer ne correspond pas forcément au système pour lequel vous allez développer vos programmes. De plus, le jour où vous n'aurez plus/pas de réseau, le manuel sera toujours dipsonible.

En particulier, vous devrez avoir le réflexe de regarder le manuel dès que sera mentionné un prototype ou un nom de fonction durant ce cours.

Lectures complémentaires

Vous êtes invité(e)s à compléter ce cours (avec son support) par d'autres lectures. Il est possible que les informations que vous lirez soient contradictoires avec certaines de celles disponibles dans ce support. Dans un tel cas, il ne faudra pas hésiter à solliciter l'enseignant pour plus d'éclairsissements.

Le livre d'Achille Braquelaire [1] reste une valeur sûre (rien que pour les aspects programmation en C) et contient une partie "programmation système" dont la lecture peut constituer un bon point d'entrée.

Le livre de Rifflet et Yunes [5] couvre non seulement la programmation système UNIX mais également la programmation réseau, ce qui en fait un complément idéal pour ce cours ETE7-IFON1 mais aussi celui de Programmation Réseau (ETE7-RESA1).

Conventions



Un point notable à retenir ou auquel il faut prêter attention.

Exercice (⋆)
Un petit exercice/manipulation à faire après avoir lu le paragraphe correspondant, de niveau de difficulté basique.

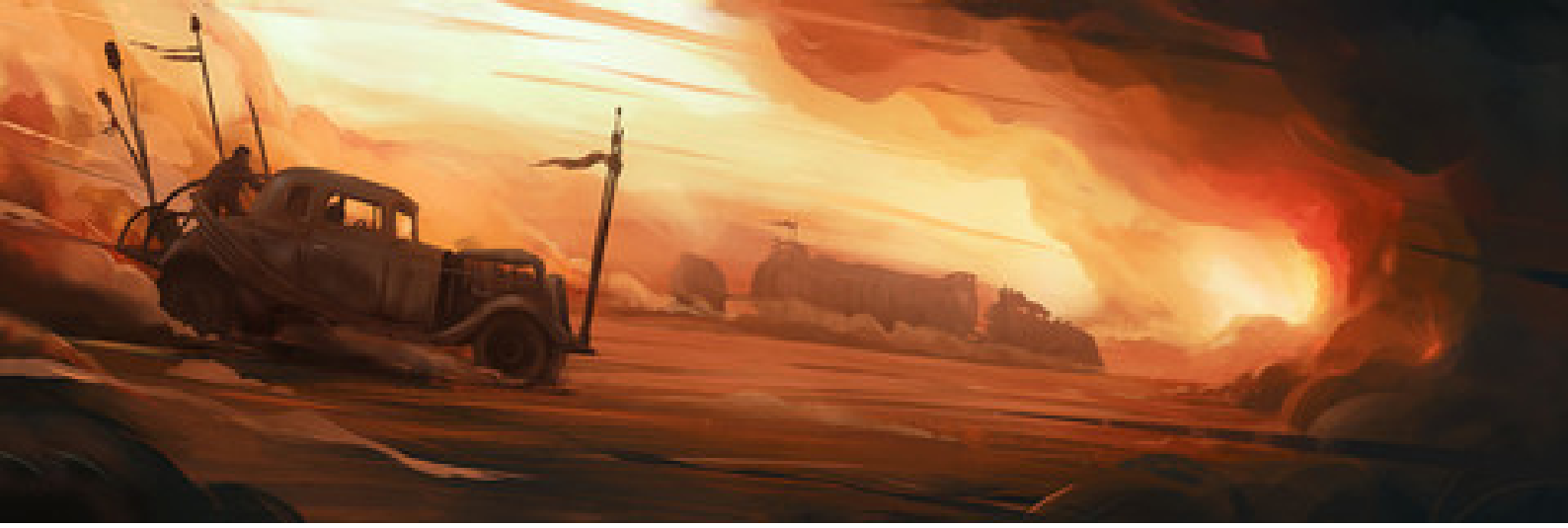
Exercice (⋆⋆)
Un exercice de niveau de difficulté intermédiaire.

Exercice (⋆⋆⋆)
Un exercice de niveau de difficulté avancé.



La promo T2, prête à déguster les délices de la programmation système.

Ce cours est approuvé par l'Institut Banzaï.



1. Premier programme

En fait, vous avez déjà écrit des programmes faisant appel aux fonctionnalités du système d'exploitation sous-jacent. Rappelez-vous votre tout premier programme écrit en C :

```c
#include <stdio.h>

int main(int argc, char *argv[])
{
  printf("Hello World!\n");

  exit(EXIT_SUCCESS);
}
```

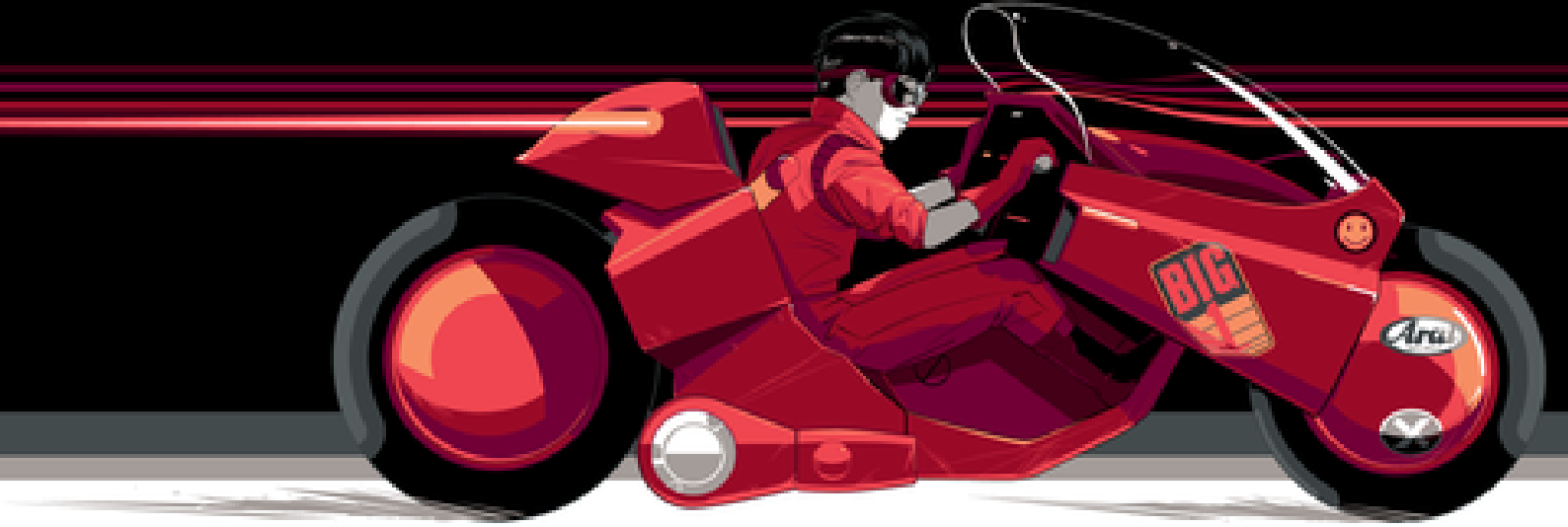
Que fait ce programme ? Il affiche à l'écran la chaîne de caractères `Hello World!` [1]. Quelque part, on peut considérer qu'effectuer cet affichage, c'est déjà utiliser les fonctionnalités du système d'exploitation sous-jacent. En fait, les programmes n'utilisant *pas* ces fonctionnalités sont rares ou bien ont un intérêt limité. Un tel programme sera par exemple une application qui effectue un certain calcul en mémoire, mais dont le résultat de ce calcul sera résident dans cette mémoire, en effet, aucune entrée/sortie ne sera possible car une telle entrée/sortie sera forcément une utilisation du système. En résumé, vous faites de la programmation système depuis que vous avez commencé à faire du C.

À partir de maintenant, nous allons écrire (enfin, surtout vous), des programmes un peu plus ambitieux, qui vont mobiliser des mécanismes plus complexes au sein du système qu'un "simple" affichage de chaîne de caractères. En particulier, vous allez apprendre à créer des applications réparties, communiquantes, éventuellement scindées en multiples *flots d'exécution*.

Mais tout d'abord, quelques rappels sur la façon dont un *code* (un fichier texte, donc) est amené à être transformé en un *programme exécutable*. Ceci est réalisé par le biais d'un processus appelé *compilation*. En effet, pour rappel, C est un langage typé et compilé dans l'écrasante majorité des

1. En fait, utiliser `printf` n'est pas si trivial : `https://www.irill.org/events/ghm-gnu-hackers-meeting/videos/jim-meyering-goodbye-world-the-perils-of-relying-on-output-streams-in-c`

cas. Il est donc quasi-nécessaire de disposer d'un programme appelé compilateur afin de pouvoir créer des programmes.



2. Utilisation des fichiers

2.1 Notion de Système de fichiers

Nous allons travailler sur des systèmes de fichiers présents dans les systèmes de type UNIX. Cependant, les concepts évoqués sont en général applicables à d'autres systèmes (aussi bien d'exploitation que de fichiers).

Dans UNIX, un **système de gestion de fichiers** (ou SGF) possède certaines caractéristiques :

— l'ensemble des noms des fichiers forme une **arborescence**, donc ne présente pas de cycles [1].
— la racine de cette arborescence est « / ».
— dans tout répertoire, on trouve toujours deux autres répertoires : « . » et « .. » (sauf pour la racine, bien évidemment).

Types de fichiers

Un principe important des systèmes de fichiers sous UNIX est que *« tout est fichier »*. Qu'est-ce que cela veut dire ? Tout simplement, c'est que **tout est accessible *via* le système de fichier**. En fait il faut faire la distinction entre les **fichiers réguliers**, qui sont les fichiers au sens où vous les entendez sûrement, c'est à dire des données et les **fichiers**, qui sont des éléments du système de gestion de fichier. Selon cette définition, sont donc considérés comme des fichiers :

— les fichiers réguliers (évidemment)
— les répertoires
— les sockets de communication
— les tubes de communication (cf. Section 4.1)
— les liens symboliques (cf. ci-dessous)
— les périphériques
— et d'autres choses encore

Il est donc important de bien faire la différence entre un fichier régulier et un fichier, c'est-à-dire un élément du système de fichiers.

1. Pour rappel, en théorie des graphes, un arbre est un graphe acyclique.



Vous pouvez d'ors et déjà remarquer une certaine **unification** des mécanismes [a] de communication puisque les fichiers réguliers, les tubes et les sockets sont mis sur le même plan. Ce point est important, car cela veut dire qu'en terme de programmation, vous allez pouvoir effectuer les **mêmes** opérations sur ces objets, même s'ils possèdent des natures différentes. En effet, ils seront tous accessibles et manipulables via des **descripteurs de fichiers** (cf. Section 2.2). C'est en particulier vrai de la fonction `poll` (la fonction qui remplace `select`, car plus récente) : vous avez l'habitude de monitorer un ensemble de descripteurs correspondants à des sockets, mais il est possible de monitorer **en même temps** des tubes et des fichiers réguliers ! Pensez-y !

a. Et non pas mécanimses (mais qui rimerait fort bien avec cyclimse). Merci de votre compréhension.

2.2 Structures du système pour la gestion des fichiers

Le système d'exploitation utilise un ensemble de structures pour la gestion des fichiers quand ils sont utilisés par des programmes ou commandes shell. Il est important de comprendre les liens existants entre ces structures pour bien appréhender le comportement de vos programmes quand ils accèdent à des ressources de type fichiers.

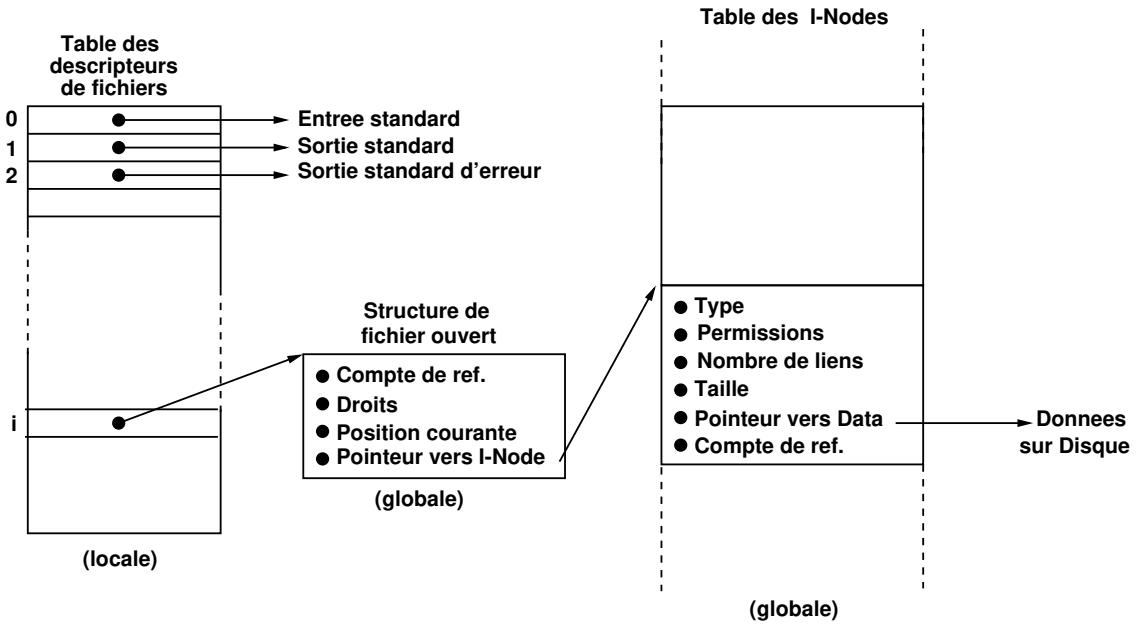


FIGURE 2.1 – Liens entre les différentes structures de données.

2.2.1 Structure d'I-Nœud

Vocabulaire 2.1 — I-Nœud. L'I-Nœud (*I-Node*) est la structure de données du système qui sert à représenter et à manipuler les fichiers au sein de ce dernier. Chaque I-Nœud possède un numéro unique servant à l'identifier de façon univoque.

Cette structure contient notamment les informations suivantes :

— le type de fichier ;
— les permissions ;

— le nombre de liens sur le fichier (i.e le nombre de noms du fichier, cf. Section 2.3) ;
— la taille du fichier ;
— un tableau avec les adresses des blocs de données sur le disque.

Ces I-nœuds sont stockés dans une table qui se trouve sur un disque dur contenant un système de fichiers. Chaque disque possède donc une telle table. Ces I-Nœuds sont appelés I-Nœuds statiques, à la différence des I-Nœuds dynamiques, qui sont stockés, eux, en mémoire physique (RAM), dans une unique table, que nous appellerons pour simplifier la Table des I-Nœuds. Cette table est donc **globale** au sein du système (d'exploitation). Les I-Nœuds dynamiques contiennent des informations supplémentaires par rapport à leur version statique, en particulier un **compte de référence** qui sert à comptabiliser le nombre d'utilisations d'un fichier, ce qui correspond en fait au nombre de structures de fichiers ouverts (cf. ci-dessous) qui pointent sur l'I-Nœud en question. Quand un fichier est utilisé, on charge son I-Nœud statique en mémoire et il devient dynamique. C'est cette version de la structure de donnée qui sera utilisée en pratique quand le système manipule le fichier correspondant.



Attention à ne pas confondre le compte de référence d'une entrée de la Table des I-Nœuds avec le nombre de liens de l'I-Nœud statique correspondant.

Exercice 1 (⋆⋆)
Si la structure d'I-Nœud contient un tableau avec les adresses des blocs de données sur le disque et que la taille d'un fichier est susceptible de changer au cours du temps, comment peut-on faire pour stocker ce tableau sans que la taille de la structure d'I-Nœud elle-même ne change ?

2.2.2 Structure de descripteur de fichier

Une autre structure utilisée est le **descripteur de fichier**. Un descripteur de fichier est en fait un « moyen » (pointeur par exemple, mais peu importe) pour accéder à une autre structure, la structure de fichier ouvert (cf. paragraphe ci-dessous). Les descripteurs de fichiers sont stockés dans un tableau linéaire, appelé **Table des descripteurs de fichiers**, d'une taille de 1048576 [2] (soit 1024*1024). Cette table est **locale**, c'est-à-dire que **chaque** programme possède une telle table qui lui est propre et qu'il ne partage avec aucun autre [3]. Cette table étant linéaire, les numéros sont donc consécutifs de 0 à 1048575. Cependant, les trois premiers numéros sont traditionnellement réservés de la façon suivante :

0 (`STDIN_FILENO`) est dédié à l'entrée standard du programme stdin
1 (`STDOUT_FILENO`) est dédié à la sortie standard du programme stdout
2 (`STDERR_FILENO`) est dédié à la sortie d'erreur du programme stderr

Pour des questions de lisibilité et de portabilité, il est fortement recommandé d'utiliser les synonymes plutôt que les index 0, 1, et 2 directement dans vos programmes.



Il est courant de faire un abus de langage et de confondre l'index du descripteur avec le descripteur proprement dit. Par exemple, on dira souvent que la fonction `open` (cf. Section 2.4.1) renvoie un descripteur alors qu'elle renvoie en réalité un

2. Cette limite est la limite *hard* qui est supérieure à la limite *soft* qui est mise en place par défaut dans le système pour chaque processus. Les valeurs de ces limites sont consultables avec la commande `ulimit` (`ulimit -n` pour connaître le nombre de descripteurs de fichiers ouvrables simultanément, 1024 *a priori*). Changer ces valeurs est possible mais demande de posséder des droits d'administrateur. Il est possible de faire des modification pour tous les processus ou bien pour des processus particuliers.
3. Un programme ne partagera **jamais** sa Table de descripteurs, quelles que soient les circonstances.

index de descripteur. Autant vous y habituer le plus tôt possible.



Ainsi que nous l'avons vu, « dans UNIX, tout est fichier ». C'est pour cela que des objets de natures très différentes seront potentiellement accessibles via des (index de) descripteurs de fichiers. Ce sera le cas notamment des fichiers réguliers, des sockets de communication (réseau) et des tubes de communication (cf. Section 2.1).

2.2.3 Structure de fichier ouvert

La dernière structure dont nous allons parler est la **structure de fichier ouvert**. Ainsi que vu au paragraphe précédent, un descripteur de fichier utilisé par un programme pointe **toujours** sur une telle structure. *A contrario*, un descripteur inutilisé (ou libre) ne pointe sur rien. Ces structures de fichier ouvert sont **globales**, c'est-à-dire qu'elles sont partagées (ou partageables) par **tous** les programmes du système. Cette structure contient un ensemble de champs utiles pour la gestion d'un fichier ouvert, en particulier :

- un compte de référence, qui sert à comptabiliser le nombre de descripteurs qui pointent sur la structure de fichier ouvert
- les droits avec lesquels le fichier a été ouvert (lecture/écriture/exécution,etc.)
- un pointeur vers la structure d'I-Nœud correspondant au fichier (et qui permet *in fine* d'accéder aux données de ce dernier)
- la **position courante** dans le fichier, c'est-à-dire l'endroit dans le fichier où pourra avoir lieu la prochaine lecture ou écriture. Cette position courante est exprimé en octets depuis le début du fichier [4].

Vous pouvez légitimement vous demander pourquoi cette structure existe : en effet, ne serait-il pas plus simple que le descripteur pointe directement sur l'I-Nœud du fichier ? Bien sûr, mais il y a d'autres considérations à prendre en compte. Nous y reviendrons plus tard.



Attention à ne pas confondre le compte de référence d'une structure de fichier ouvert avec le compte de référence d'une entrée de la Table des I-Nœuds.

2.3 Nommage et partage de données

Un fichier est accessible via un *chemin* dans l'arborescence du système de gestion de fichier. Ce chemin peut être **absolu** ou **relatif**. Le chemin absolu est le nom complet, avec toutes les composantes, en partant de la première, c'est-à-dire la racine « / ». Le *nom du fichier* n'est que la dernière composante du chemin et sous-entend toutes les précédentes (i.e tous les noms de répertoires qu'il faut parcourir pour arriver jusqu'au fichier proprement dit).

Par exemple, si un programme accède à un fichier nommé `toto.txt` qui se trouve dans le répertoire `titi` qui se trouve lui-même dans le répertoire `tutu`, alors alors le chemin absolu vers ce fichier est `/tutu/titi/toto.txt` et son chemin relatif est `toto.txt`, si le programme est lancé depuis le répertoire `titi`. Si le programme est lancé depuis le répertoire `tutu`, alors le chemin relatif du fichier sera `titi/toto.txt`.

Dans cette arborescence, il est possible d'accéder à un même fichier via de multiples chemins. Ceci peut se réaliser par le truchement des opérations de lien (*link*) :

1. **les liens physiques** (`ln target link_name`) : créer un lien physique sur un fichier revient à créer un nouveau chemin dans l'arborescence pour accéder à ce fichier. Accéder via un

4. C'est pourquoi on parle souvent de décalage (*offset*).

chemin ou l'autre fait qu'au final on accède à la **même structure d'I-nœud** [5]. En pratique les liens physiques sont peu utilisés, une utilisation possible étant le fait de pouvoir disposer de commandes ayant des comportements différents selon le nom utilisé (e.g. `bzip2` et `bunzip2`). Il n'est pas possible de créer un lien physique vers un fichier inexistant.

2. **les liens symboliques** (`ln -s target link_name`) : un lien symbolique peut être vu comme une sorte de « raccourci » [6]. Un lien symbolique est un fichier à part entière, possédant sa **propre structure d'I-Nœud**, mais ce fichier et son interprétation sont particulières : un lien symbolique ne contient que le chemin vers le fichier sur lequel il pointe et tout accès au lien est résolu comme un accès au chemin contenu dans le lien. Il est donc possible de créer un lien symbolique vers un fichier inexistant. Les liens symboliques sont très utiles pour la gestion des systèmes, notamment pour les noms/versions de bibliothèques.

Exercice 2 (⋆⋆)
Quelles sont les différences, d'un point de vue des structures utilisées, entre copies de fichiers, liens physiques et liens symboliques ?

2.4 Interface pratique

Nous allons maintenant voir un ensemble de fonctions servant à la manipulation des fichiers (réguliers). Il existe bien plus de fonctions que celles qui sont listées ici, mais celles présentées devraient couvrir la majeure partie de vos besoins. Il existe en fait deux types d'interfaces de manipulation : d'une part les appels-système et d'autre part des fonctions appartenant à la *libc*, de niveau utilisateur.

2.4.1 Appels-système

Les fonctions qui sont décrites dans cette section sont des **appels-système**, c'est-à-dire des fonctions qui sont implémentées dans le noyau du système d'exploitation (cf. Cours d'ETE8-IFON1 au S8).

Ouverture de fichier

Le première opération que l'on peut faire sur fichier, c'est l'ouvrir. Cette opération est réalisable à l'aide de la fonction `open` dont voici le prototype :

```
int open(const char *pathname, int flags);
```

Paramètre 1 : Nom (relatif ou absolu) du fichier à ouvrir.
Paramètre 2 : Droits d'ouverture :
- `O_RDONLY` : ouverture en lecture seule.
- `O_WRONLY` : ouverture en écriture seule.
- `O_RDWR` : ouverture en lecture et écriture.
- `O_EXEC` : ouverture en exécution

Une seule de ces valeurs doit être utilisée en paramètre. Ces droits sont néanmoins composables avec d'autres options, dont voici un florilège :
- `O_APPEND` : ouverture du fichier en mode *append*, c'est-à-dire pour rajouter des données en fin de fichier. **La position courante est mise en fin de fichier avant chaque écriture.**

5. Donc deux chemins différents peuvent pointer sur la même structure d'I-Nœud : c'est un cycle. Mais dans la section précédente, seul l'espace des noms de fichiers a été évoqué en tant qu'arborescence.
6. Pour reprendre une terminologie Windows.

— `O_TRUNC` : si le fichier existe et qu'un accès en écriture est accordé (i.e. `O_RDWR` ou `O_WRONLY`) alors la **taille du fichier est remise à zéro** (écrasement des données).
— `O_CREAT` : création du fichier s'il n'existe pas. Le prototype de la fonction prend alors un argument supplémentaire (cf. ci-dessous).
— `O_EXCL` : mode exclusif, une erreur est renvoyée si le fichier existe déjà et qu'on demande sa création. `O_EXCL` est souvent utilisé conjointement avec `O_CREAT`.
— `O_CLOEXEC` : fermeture automatique du fichier lors d'un recouvrement avec `exec` (cf. Section 3.9).
— `O_NONBLOCK` : ouverture en mode non-bloquant.
— `O_TMPFILE` : cette option permet la création d'un fichier temporaire. Dans ce cas, le premier paramètre (`pathname`) sera le nom du *répertoire* au sein duquel le fichier sera créé. Ce dernier est toutefois particulier car il ne possède pas de nom dans le système de gestion de fichier (i.e son nombre de liens est égale à 0), ce qui signifie qu'en cas de fermeture ou de perte du dernier descripteur correspondant à ce fichier (car il est possible de copier un descripteur, cf. Section 2.4.1), alors le contenu du fichier sera perdu, à moins qu'un nom (i.e un lien physique) ne soit donné à ce fichier (cf. la fonction `link` dans le manuel). À noter que dans le cas de `O_TMPFILE`, le sens du flag `O_EXCL` diffère : il empêche la création de lien physique sur le fichier temporaire. La création d'un tel fichier temporaire peut être utile dans le cas de partage de mémoire entre processus via une projection de fichier (cf. Section 4.2.2).

Code de Retour : -1 si problème (cf. valeur d'`errno`) / (index du) nouveau descripteur de fichier créé.



Les options se composent avec l'opérateur **ou bitwise** (`|`). Par ex. `O_RDWR | O_CREAT | O_EXCL`.



La position courante d'un fichier nouvellement ouvert est 0 par défaut (i.e. au début du fichier).

Si `O_CREAT` est spécifié dans les `flags`, alors `open` admet un paramètre supplémentaire et son prototype devient :

```
int open(const char *pathname, int flags, mode_t mode);
```

Paramètre 3 : Droits avec lequels le fichier est créé s'il n'existe pas. (en particulier `S_IRWXU`, `S_IRUSR`,`S_IWUSR` et `S_IXUSR`).

Un appel à `open` crée **systématiquement** un nouveau descripteur dans la table des descripteurs de fichiers du programme, ainsi qu'une nouvelle structure de fichier ouvert.



(L'index de) Ce nouveau descripteur est **toujours le plus petit disponible** dans la table des descripteurs de fichiers.



Un programme peut accéder à un même fichier via des (index de) descripteurs distincts et/ou des structures de fichier ouvert distinctes.



Une ouverture de fichier en mode lecture + écriture ne crée qu'une seule structure de fichier ouvert, avec les droits correspondants.

Par exemple, la figure 2.2 montre ce qui se passe quand le code suivant est exécuté (remarquez les valeurs des différents comptes de référence) :

```c
int main(int argc, char *argv[])
{
  int fd1;
  int fd2;

  /* ... du code ... */

  fd1 = open("truc.txt", O_RDONLY);
  fd2 = open("truc.txt", O_RDWR);

  /* Ici, fd2 vaut *obligatoirement* */
  /* (fd1 + 1) a condition qu'aucun  */
  /* appel a close n'a ete effectue  */
  /* avant le premier appel a open.  */

  /* ... du code ... */

  exit(EXIT_SUCCESS);
}
```

Exercice 3 ($\star\star\star$)

Quelle est l'utilité du compte de référence de la structure de fichier ouvert puisqu'il semblerait qu'il ne puisse valoir qu'un ?

Exercice 4 ($\star$)

Créez un fichier dans votre répertoire de travail (avec une commande shell). Écrivez un programme qui ouvre ce fichier et affiche la valeur (de l'index) de descripteur de fichier. Cette valeur est-elle conforme à vos attentes ?

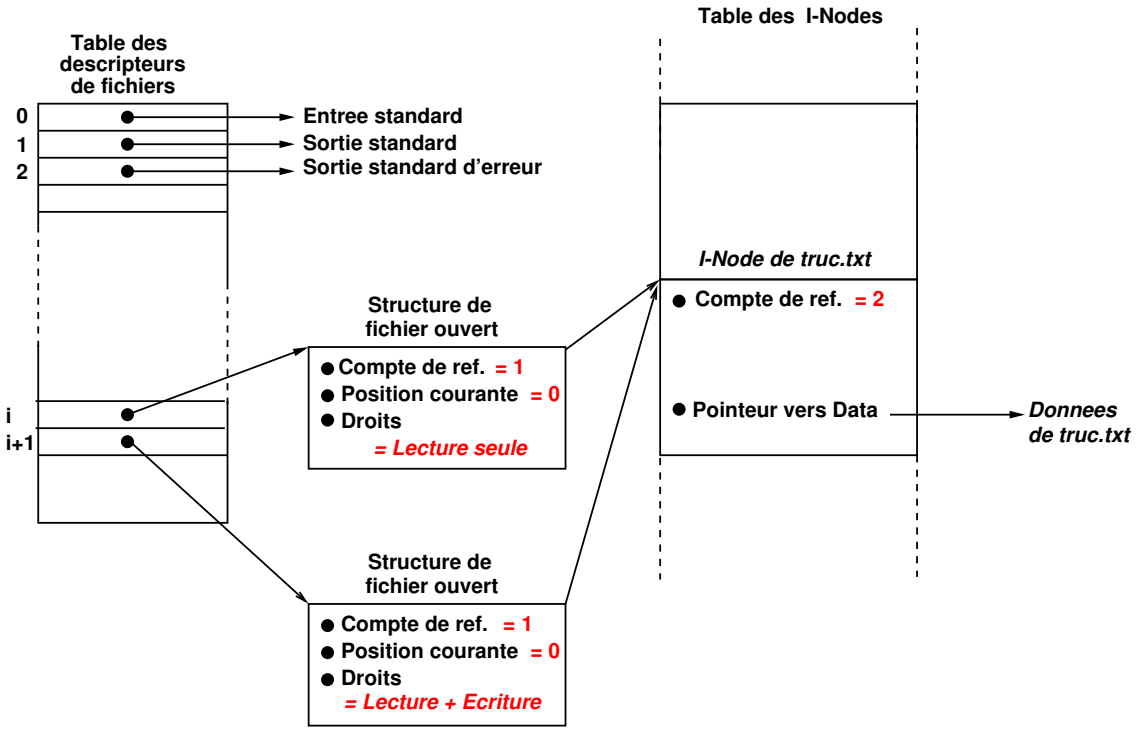


FIGURE 2.2 – Lien entre ouvertures de fichiers et structures

Exercice 5 ($\star$)

Écrivez un programme qui ouvre un fichier en lecture seule et qui crée le fichier en mode lecture/écriture/exécution (uniquement pour vous). On supposera que ce fichier n'existe pas déjà.

Fermeture de (descripteur de) fichier

L'opération duale est la fermeture, non pas d'un fichier, mais d'un **descripteur de fichier**. Fichier est ici à comprendre non pas comme fichier régulier, mais bien comme élément du système de gestion de fichiers. Cela pourra donc être un fichier régulier, un tube de communication, une socket réseau ou une autre chose encore à laquelle est associé un descripteur. L'appel-système pour réaliser cette fermeture est `close` dont voici le prototype :

```
int close(int fd);
```

Paramètre : (index du) descripteur de fichier à fermer.
Code de Retour : -1 si problème (cf. valeur d'`errno`) / 0 si succès.

`close` libère le descripteur (d'index) `fd` de la table des descripteurs de fichiers du programme. L'(index de) descripteur peut donc être réutilisé en cas de nouvelle ouverture (par exemple). Toute fermeture de descripteur est **définitive** !



`close` n'élimine pas un fichier du système (effacement) !

Effacement de fichier

`close` ne doit pas être confondu avec `unlink` qui efface un fichier du disque.

```
int unlink(const char *pathname);
```

Paramètre : nom du fichier à éliminer du système de fichiers (effacement).
Code de Retour : -1 si problème (cf. valeur d'`errno`) / 0 si succès.

En fait, `unlink` élimine un nom du système de fichier (un lien, d'où le nom de la fonction). Rappelez-vous qu'un fichier peut avoir plusieurs noms dans le SGF (notion de lien physique). Si le lien éliminé est le dernier et que le fichier n'est pas utilisé par un programme en cours, alors il sera effacé.

Pour aller plus loin : modifier le comportement d'un descripteur

Le comportement d'un descripteur est déterminé au moment de sa création (i.e opération `open`, `socket` ou encore `pipe` (cf. Section 4.1)). Cependant, il peut arriver que l'on souhaite modifier ce comportement en cours d'exécution du programme, par exemple passer une socket ou un tube de communication (initialement en mode bloquant) en mode non-bloquant ou l'inverse. Ceci est possible avec la fonction `fcntl` dont voici le prototype :

```
int fcntl(int fd, int cmd, ... /* arg */ );
```

Paramètre 1 : (index du) descripteur de fichier à modifier.
Paramètre 2 : commande de modification.
Paramètres 3 et + : arguments éventuels de la commande.
Code de Retour : -1 si problème (cf. valeur d'`errno`) / valeur de retour dépendante de la commande si succès.

`fcntl` est une fonction à nombre variable d'arguments, dont la lecture de la page de manuel est indispensable en cas d'utilisation.

Accès aux fichiers

Maintenant que nous disposons d'un descripteur, il est temps d'effectuer des opérations un peu plus intéressantes avec, c'est-à-dire lire ou écrire des données. Il existe pour ce faire deux opérations fondamentales, dont les interfaces sont (presque) identiques :

```
ssize_t read (int fd,       void *buf, size_t count);
ssize_t write(int fd, const void *buf, size_t count);
```

Paramètre 1 : (Index du) descripteur de fichier depuis lequel des données sont à lire (`read`) ou écrire (`write`).
Paramètre 2 : Adresse à laquelle les données seront écrites (`read`) ou lues (`write`).
Paramètre 3 : Taille des données à lire (`read`) ou écrire (`write`). Le pseudo-type `size_t` indique un nombre d'octets.
Code de Retour : -1 si problème (cf. valeur d'`errno`) / taille effectivement lue (`read`) ou écrite (`write`) si succès.

Lire ou écrire des données dans un fichier implique **une modification du champ décalage, ou position courante** (*offset*) de la structure de fichier ouvert sur laquelle pointe le descripteur de fichier (dont l'index est) pris en premier argument des appels `read` et `write`, à condition que cette position courante existe pour le fichier considéré.



Dans le cas de `read`, une valeur de retour de 0 indique que la fin de fichier est atteinte. Si le fichier considéré est une socket, cela signifie qu'une opération de type `shutdown` sur la partie en écriture a été effectuée par un autre processus utilisant cette socket.



Le nombre d'octets effectivement lus peut-être inférieur au nombre demandé (`count`), ce qui n'est pas une erreur. En effet, cela peut arriver si moins de données sont disponibles que demandé. Par exemple, cette situation peut se produire quand des données sont lues dans un fichier et que la fin — du fichier — est proche. Cela peut également se produire en cas de lecture dans un tube (cf. Section 4.1) ou bien quand la lecture est interrompue par un signal (cf. Chapitre 6).



`size_t` et `ssize_t` sont appelés des **pseudo-types**. Les pseudo-types sont utilisés pour rendre plus lisibles et compréhensibles les prototypes de fonctions en programmation système.



Les appels-système `read` et `write` prennent en premier paramètre un **(index de) descripteur de fichier**. Ils peuvent donc être utilisés avec tout autre objet référencé par un (index de) descripteur de fichier. C'est le cas notamment des fichiers réguliers (bien entendu) mais également des **tubes de communication** (cf. Section 4.1) et aussi des **sockets réseau** (cf. le cours de Programmation Réseau) !

Le comportement de `read` et `write` peut varier selon l'objet considéré (fichier régulier, tube ou socket). Dans le cas des sockets réseau, il faut aussi faire attention à la façon de lire/écrire les données. En particulier, un appel tel que :

```
1 write (fd, data_ptr, to_send);
```

DOIT être remplacé par :

```
1  size_t  sent = 0; /* taille courante emise   */
2  ssize_t ret  = 0;
3  do{
4    ret = write (fd, data_ptr + sent, to_send - sent);
5    if(-1 != ret){ /* gestion des erreurs et/ou */
6      sent += ret; /* du mode non-bloquant      */
7    } else {
8      /* Gestion des problemes  */
9    }
10  } while (sent != to_send);
```

Et la même chose pour un appel à `read`, cela va sans dire. Sinon, votre programme **ne fonctionnera pas**, tout simplement [7].

7. D'accord, vous avez déjà écrit des programmes comme ça et ça fonctionnait jusqu'à présent. La chance du (de la) débutant(e) ...La programmation exige de la rigueur, faites-en preuve, surtout dans le cas d'applications réseau/distribuées.



Josette, sympathique élève de T2 (quoiqu'au funeste destin), ayant décidé de faire fi des judicieux conseils prodigués quant à l'utilisation de `read` et `write`.



Jean-Eudes, son binôme, manifestement inattentif aux errements de sa condisciple.

Exercice 6 (⋆)
Écrivez un programme qui écrit **un entier** dans un fichier (i.e. pas sous forme de chaîne de caractères). Dans un terminal, affichez le contenu de ce fichier (à l'aide d'une commande shell comme `cat`, `less` ou `more` et non pas d'un programme C).
— Que constatez-vous ?

Exercice 7 (⋆)
Modifiez le programme précédent et rajoutez-y la lecture (et l'affichage) de l'entier que vous venez d'écrire dans le fichier.

Il existe également d'autres fonctions, `pread` et `pwrite`, dont les prototypes sont les suivants :

```
ssize_t pread (int fd,       void *buf, size_t count, off_t offset);
ssize_t pwrite(int fd, const void *buf, size_t count, off_t offset);
```

Paramètre 1 : (Index du) descripteur de fichier à lire/écrire.
Paramètre 2 : Adresse à laquelle les données seront écrites (`read`) ou lues (`write`).
Paramètre 3 : Taille des données à lire/écrire.
Paramètre 4 : Position dans le fichier à partir de laquelle s'effectue la lecture/écriture. Cette position est exprimée comme un décalage depuis le début du fichier (pseudo-type `off_t`).
Code de Retour : -1 si problème (cf. valeur d'`errno`) / taille effectivement lue/écrite (0 = fin

de fichier) si succès.

Ces fonctions peuvent être utilisées **uniquement** sur des fichiers pour lesquels une position courante existe et est modifiable (cf. `lseek` ci-dessous), ce qui n'est pas le cas des tubes de communication et des sockets réseau.



Contrairement à `read` et `write`, les appels à `pread` et `pwrite` **ne modifient pas le champ décalage** de la structure de fichier ouvert pointée par le descripteur (d'index) `fd` ! Ces fonctions règlent donc les éventuels problèmes d'accès concurrents à un même fichier par plusieurs programmes qui partagent ce champ décalage (cf. Sections 3.5 et 7.2.6).

Les appels `write` et `pwrite` prennent en argument un (index de) descripteur de fichier mais n'offrent pas les optimisations offertes par les appels de la famille `printf` qui travaillent sur les flux (objets de type `FILE`, cf. Section 2.4.2). Il existe néanmoins la fonction `dprintf`, qui permet de faire un appel à `printf` mais avec un (index de) descripteur :

```
int dprintf(int fd, const char *restrict format, ...);
```

Paramètre 1 : (Index du) descripteur de fichier sur lequel faire l'affichage
Paramètre 2 : chaîne de caractères à afficher (même format que `printf`)
Paramètres 3+ : élément à afficher
Code de Retour : valeur < 0 si problème / nombre d'octets affichés si réussite (sans `\n`)



`dprintf` est une fonction de la libc et non pas un appel-système.

Déplacement dans un fichier

Il est quelquefois utile de pouvoir se déplacer dans un fichier pour lire/écrire des données à une position particulière (i.e. pas forcément au début). `read` et `write` réalisent cette opération, mais s'accompagent d'une lecture ou écriture obligatoire de données. Afin de ne modifier **que** la position courante, il est possible d'utiliser la fonction `lseek` :

```
off_t lseek(int fd, off_t offset, int whence);
```

Paramètre 1 : (Index du) descripteur de fichier dans lequel on se déplace.
Paramètre 2 : Valeur de la position courante à appliquer (peut être positive ou négative).
Paramètre 3 : Directive de déplacement, c'est-à-dire une des trois valeurs suivantes :
- `SEEK_SET` : nouvelle position courante = `offset` (i.e début du fichier + `offset`)
- `SEEK_CUR` : nouvelle position courante = position courante + `offset`
- `SEEK_END` : nouvelle position courante = fin du fichier + `offset`

Code de Retour : -1 si problème (cf. valeur d'`errno`) / valeur de la position courante si succès.

Linux supporte (de façon non portable) deux autres valeurs pour `whence` : `SEEK_DATA` et `SEEK_HOLE` qui permettent de répérer respectivement des données ou des trous dans un fichier pour y positionner son `offset` (cf. le manuel pour plus de précisions).

Exercice 8 (⋆)

Écrivez un programme qui écrit n caractères dans un fichier. Les arguments du programme seront le nom du fichier et la valeur de n. Le i-ème caractère devra être écrit à la position 2^i dans

le fichier, en supposant que la numérotation des caractères commence à 1.

Duplication de descripteurs

Il existe tout un panel d'opérations sur les fichiers, mais la dernière que nous allons voir est appelée **duplication** de descripteur. Cette opération consiste à créer un nouveau descripteur, qui sera le synonyme d'un autre descripteur déjà existant. Concrètement, le nouveau descripteur pointera sur la même structure de fichier ouvert que celle pointée par le descripteur dupliqué. La conséquence est donc que ces deux descripteurs vont **partager la position courante dans le fichier**. Cette opération de duplication est notamment très utile pour effectuer ce qu'on appelle des **redirections d'entrées/sorties**. Par exemple, les redirections peuvent être utilisées pour modifier le comportement d'un programme sans en changer le code directement [8]. La duplication est possible avec les fonctions de la famille `dup` :

```
int dup(int oldfd);
```

Paramètre : (Index du) descripteur de fichier à dupliquer.
Code de Retour : -1 si problème (cf. valeur d'`errno`) / (index du) descripteur de fichier synonyme si succès.

`dup` renvoie **toujours le descripteur de plus petit index disponible** dans la Table des descripteurs de fichiers du programme.

Par exemple, la figure 2.3 montre ce qui se passe quand le code suivant est exécuté. Remarquez les valeurs des différents comptes de référence, ainsi que le partage de la position courante entre les deux descripteurs de fichiers.

```
1  int main(int argc, char *argv[])
2  {
3    int fd1;
4    int fd2;
5
6    /* ... du code */
7
8    fd1 = open("truc.txt", O_RDONLY);
9    fd2 = dup(fd1);
10
11   /* fd2 vaut encore (fd1 + 1) dans */
12   /* ce cas si aucun close n'a ete  */
13   /* effectue avant l'appel a open. */
14
15   /* ... du code */
16
17   exit(EXIT_SUCCESS);
18 }
```

Exercice 9 (⋆)

8. C'est ce qui se passe quand vous utilisez > ou < dans une commande shell.

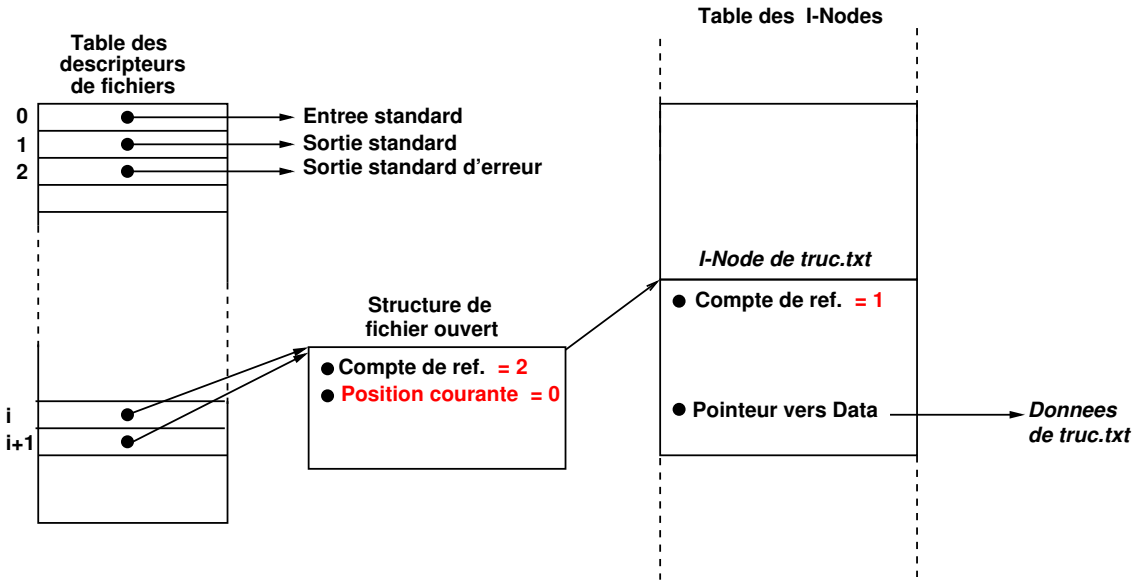


FIGURE 2.3 – Lien entre duplication de descripteurs et structures

Écrivez un programme qui redirige tous ses appels à `printf` dans un fichier `output.txt` plutôt que sur sa sortie standard, sans modifier les appels à `printf` dans le programme d'origine.

Exercice 10 (⋆)

Que se passera-t-il si un programme effectue les instructions suivantes ?

```
1 close(STDOUT_FILENO);
2 open("toto.txt",O_RDWR);
```

NB : On supposera que le fichier `toto.txt` *existe.*

Ainsi que vous pouvez le voir, il n'est pas forcément pratique d'utiliser la fonction dup car le choix du descripteur est imposé et dépend de l'état de la Table des descripteurs de fichiers du programme au moment de l'appel. Afin d'offrir un peu plus de flexibilité, il existe la fonction dup2 qui lève cette limite.

```
int dup2(int oldfd, int newfd);
```

Paramètre 1 : (Index du) descripteur de fichier à dupliquer.
Paramètre 2 : (Index du) nouveau descripteur synonyme.
Code de Retour : -1 si problème (cf. valeur d'`errno`) / (index du) descripteur de fichier synonyme si succès.

`dup2` fait la même chose que `dup`, mais au lieu d'utiliser le descripteur disponible de plus petit index, cette fonction utilise le numéro pris en second paramètre (`newfd`). Si `newfd` est déjà utilisé au moment de l'appel, le descripteur correspondant est fermé avant la duplication. Cette fermeture puis réutilisation du descripteur est faite de façon **atomique**[9].



Attention, c'est `newfd` qui devient synonyme de `oldfd`. En général, vous faites l'inverse dans 80% des cas.

9. Chose impossible à réaliser avec de simples appels à `close` puis `dup`.

Enfin, il existe `dup3` (spécifique à Linux, non-POSIX), qui est similaire à `dup2`, mais accepte en plus un troisième paramètre d'options pour le descripteur nouvellement créé (typiquement `O_CLOEXEC`) :

```
int dup3(int oldfd, int newfd, int flags);
```

Paramètre 1 : (Index du) descripteur de fichier à dupliquer.
Paramètre 2 : (Index du) nouveau descripteur synonyme.
Paramètre 3 : options éventuelles (cf. le manuel)
Code de Retour : -1 si problème (cf. valeur d'`errno`) / (index du) descripteur de fichier synonyme si succès.

2.4.2 Appels de la librairie *libc*

Toutes les fonctions vues jusqu'à présent étaient des appels-système, qui manipulent ou renvoient des (index de) descripteurs de fichiers. Ce n'est pas le seul moyen de travailler sur les fichiers puisqu'il existe une autre interface, basée sur des fonctions implémentées dans la *libc*, et qui sont de **niveau utilisateur·trice** (par opposition aux appels-système, qui sont de **niveau noyau** (*kernel*). Ces fonctions ne travaillent pas sur/avec des descripteurs, mais avec des pointeurs sur objets opaques de type `FILE` [10]. Les fonctionnalités disponibles sont peu ou prou les mêmes que celles vues précédemment, à la **notable exception de la duplication** (quoique...).

Libc vs appels-système

L'utilisation des fonctions de la libc plutôt que les appels-système est motivée par des questions de performances. En effet, même si les fonctions de la *libc* sont implémentées avec (et au-dessus de) ces mêmes appels-système, elles ne les utilisent pas systématiquement et mettent en place des mécanismes d'optimisation qui permettent d'en atténuer les coûts.

Sans surprise, on commence donc avec l'opération d'ouverture :

```
FILE *fopen(const char *path, const char *mode);
```

Paramètre 1 : Nom (chemin) du fichier à ouvrir.
Paramètre 2 : Mode d'ouverture du fichier (cf. le manuel et la table ci-dessous).
Code de Retour : `NULL` si problème (cf. valeur d'`errno`) / pointeur sur l'objet de type `FILE` correspondant si succès.

Correspondance entre les modes d'ouverture de `fopen` et d'`open` :

`fopen()` mode	`open()` flags
`"r"`	`O_RDONLY`
`"r+"`	`O_RDWR`
`"w"`	`O_WRONLY | O_CREAT | O_TRUNC`
`"w+"`	`O_RDWR | O_CREAT | O_TRUNC`
`"a"`	`O_WRONLY | O_CREAT | O_APPEND`
`"a+"`	`O_RDWR | O_CREAT | O_APPEND`

10. Opaque signifie que le type est une boîte noire, peu importe ce qu'elle contient, nous n'en n'avons pas besoin pour l'utiliser.



En cas de création de fichier avec `fopen`, les permissions de création ne sont pas passées en arguments à la fonction. Ces sont les permissions positionnées avec `umask` qui sont utilisées.

Pour la fermeture, c'est `fclose` qui est à utiliser :

```
int fclose(FILE *fp);
```

Paramètre : Pointeur sur la structure `FILE` associée au fichier à fermer.
Code de Retour : EOF si problème (cf. valeur d'`errno`) / 0 si succès.

Exercice 11 ($\star$)
Écrivez un programme qui ouvre un fichier avec la *fonction* `fopen` puis qui ouvre un fichier (le même ou un autre, peu importe) avec l'*appel-système* `open`. Affichez la valeur de l'index de descripteur associé.
— Que constatez-vous ?
— Que pouvez-vous en déduire ?

Les accès au fichier sont possibles avec le couple `fread`/`fwrite`, dont l'interface est un peu différente de celle de `read` et `write` car deux paramètres sont utilisés pour les données : d'une part le nombre d'éléments à lire ou écrire et la taille de chaque élément (en octets).

```
size_t fread (      void *ptr, size_t size, size_t nmemb,FILE *stream);
size_t fwrite(const void *ptr, size_t size, size_t nmemb,FILE *stream);
```

Paramètre 1 : Adresse à laquelle les données seront écrites (`fread`) ou lues (`fwrite`).
Paramètre 2 : Taille d'un élément à lire ou écrire.
Paramètre 3 : Nombre d'éléments à lire ou écrire.
Paramètre 4 : Pointeur sur la structure `FILE` associée au fichier depuis lequel des données sont lues ou dans lequel des données sont écrites.
Code de Retour : le nombre *d'éléments* lus ou écrits si succès / 0 ou un nombre inférieur à celui demandé en cas d'erreur ou de fin de fichier.

La fonction `fread` ne fait pas la différence entre la fin d'un fichier et une erreur. Pour faire la distinction, il faut appeler la fonction `feof` ou `ferror` (à regarder dans le manuel) :

```
int feof(FILE *stream);
```

Paramètre : Pointeur sur la structure `FILE` associée au fichier.
Code de Retour : Pas d'échec / $\neq 0$ si fin de fichier / 0 sinon

La modification de la position courante dans le fichier est possible avec `fseek` :

```
int fseek(FILE *stream, long offset, int whence);
```

Paramètre 1 : Pointeur sur la structure `FILE` associée au fichier.
Paramètre 2 : Valeur de la position courante à appliquer (peut être positive ou négative).
Paramètre 3 : Directive de déplacement, c'est-à-dire l'une des trois valeurs suivantes :
— `SEEK_SET` : nouvelle position courante = `offset` (i.e début du fichier + `offset`)
— `SEEK_CUR` : nouvelle position courante = position courante + `offset`
— `SEEK_END` : nouvelle position courante = fin du fichier + `offset`

Code de Retour : -1 si problème (cf. valeur d'`errno`) / 0 si succès.

La duplication n'existe pas avec les fonctions de la libc : il est donc *a priori* impossible d'effectuer des redirections d'E/S. Cependant, la fonction `freopen` permet cependant d'y arriver :

```
FILE * freopen(const char *restrict pathname, const char *restrict mode,
.              FILE *restrict stream);
```

Paramètre 1 : Nom du fichier à ouvrir.
Paramètre 2 : Mode d'ouverture du fichier (même utilisation que dans le cas de `fopen`).
Paramètre 3 : Pointeur sur la structure `FILE`
Code de Retour : Un pointeur sur le nouvel objet `FILE`

La fonction `freopen` ouvre le fichier dont le nom est donné en premier paramètre, selon le mode d'ouverture indiqué par le deuxième paramètre. Ce fichier est désormais associé à l'objet `FILE` pointé par le dernier paramètre. L'objet `FILE` de départ (s'il existe) est fermé avant cette association. `freopen` possède deux utilisations principales :

— Modifier le mode d'ouverture original d'un objet `FILE`. Dans ce cas, le premier paramètre (`pathname`) doit être `NULL` ;
— Changer le fichier associé des flux standards (`stdin`, `stdout` et `stderr`)



Avec les descripteurs de fichiers, il est possible de sauvegarder (via la duplication) `STDIN_FILENO`, `STDOUT_FILENO` ou `STDERR_FILENO` avant de procéder à un redirection. De cette façon, un retour à la situation de départ est toujours envisageable. Dans le cas de `freopen`, une fois l'association effectuée, le retour en arrière n'est plus possible.

Exercice 12 (⋆)
Écrivez un programme qui redirige tous ses appels à `fprintf(stdout, ...)` dans un fichier `output.txt` plutôt que sur sa sortie standard, sans modifier les appels à `fprintf` dans le programme d'origine et sans utiliser des (index de) descripteurs de fichiers ni la duplication.

2.4.3 Conversions entre flux et (index de) descripteurs

Il est possible de récupérer un pointeur sur un objet de type `FILE` à partir d'un (index de) descripteur de fichier et réciproquement. Les fonctions `fileno` et `fdopen` offrent de telles fonctionnalités :

```
int fileno(FILE *stream);
```

Paramètre : Pointeur sur l'objet de type `FILE` associé au fichier.
Code de Retour : -1 si problème (cf. valeur d'`errno`) / (index de) descripteur associé au fichier si succès.

```
FILE *fdopen(int fd, const char *mode);
```

Paramètre 1 : (Index de) descripteur associé au fichier.
Paramètre 2 : Mode d'ouverture (idem que pour `fopen`), doit être compatible avec celui du descripteur pris en argument (il est donc possible de modifier les droits).
Code de Retour : `NULL` si problème (cf. valeur d'`errno`) / pointeur sur l'objet de type `FILE`

correspondant si succès.



Passer dans un autre univers peut avoir des conséquences insoupçonnées.

2.4.4 Pour aller plus loin : accès aux caractéristiques d'un fichier

Il est possible d'accéder aux caractéristiques d'un fichier (par exemple, sa taille) à l'aide de la fonction `stat` (et ses dérivées). Cette fonction retourne un pointeur sur une structure dont les champs sont consultables.

```
int stat(const char *pathname, struct stat *buf);
int lstat(const char *pathname, struct stat *buf);
```

Paramètre 1 : Nom du fichier (chaîne de caractères). Dans le cas de `lstat`, si le nom référence un *lien symbolique*, les informations renvoyées concernent le lien et non pas le fichier sur lequel il pointe.
Paramètre 2 : Pointeur sur une structure contenant les informations.
Code de Retour : -1 si problème (cf. valeur d'`errno`) / 0 si succès.

La structure `stat`, quant à elle, contient les informations suivantes (liste non-exhaustive) :

```
struct stat {
dev_t     st_dev;     /* ID of device containing file */
ino_t     st_ino;     /* inode number */
mode_t    st_mode;    /* file type and mode */
nlink_t   st_nlink;   /* number of hard links */
uid_t     st_uid;     /* user ID of owner */
gid_t     st_gid;     /* group ID of owner */
dev_t     st_rdev;    /* device ID (if special file) */
off_t     st_size;    /* total size, in bytes */
blksize_t st_blksize; /* blocksize for filesystem I/O */
blkcnt_t  st_blocks;  /* number of 512B blocks allocated */
}
```



La taille d'un fichier et son occupation sur le disque dur sont deux choses différentes ! La taille d'un fichier est toujours un décalage par rapport au début du fichier (pseudotype `off_t`). Il existe également des fichiers à trous.

Il est possible d'obtenir ces mêmes informations en utilisant un (index de) descripteur de fichier plutôt que son nom, avec la fonction `fstat`.

```
int fstat(int fd, struct stat *buf);
```

Paramètre 1 : (index de) descripteur du fichier pour lequel on cherche à obtenir des informations.
Paramètre 2 : Pointeur sur une structure contenant les informations.
Code de Retour : -1 si problème (cf. valeur d'`errno`) / 0 si succès.

2.5 Exercices

Les exercices de cette section sont à faire *obligatoirement*.

Exercice 13 (⋆⋆)

Écrivez un programme qui lit l'*intégralité* d'un fichier, caractère par caractère (i.e. octet par octet) et qui affiche le tout sur sa sortie standard d'*erreur*.
N.B L'utilisation de `stat` ou `fstat` n'est pas autorisée pour cet exercice.

Exercice 14 (⋆⋆)

On suppose l'existence d'un fichier texte contenant *un* mot par ligne avec plusieurs lignes, par exemple :

```
toto
titi
tata
```

Écrivez un programme qui :

1. ouvre ce fichier en lecture seule ;
2. compte le nombre de lignes dans le fichier ;
3. alloue un tableau dynamiquement pour contenir les différents mots (on suppose que leur taille fait au maximum `MAX_SIZE`). La taille de ce tableau doit correspondre *exactement* au nombre de mots (i.e. de lignes) dans le fichier ;
4. stocke les mots dans le tableau ;
5. affiche le contenu de ce tableau.

2.6 Travaux Pratiques

Exercice 15 (⋆)

Écrivez une version en **C** de la commande `cat` en utilisant uniquement les fonctions standards d'entrées/sorties de la bibliothèque du **C** `fread` et `fwrite`, et en lisant et écrivant des blocs d'**un** caractère (oui, oui, un seul caractère . . .).

Exercice 16 (⋆)

Écrivez une seconde version de `cat` en utilisant uniquement les appels-système `read` et `write`.

Exercice 17 (⋆)

En utilisant la commande interne `time` du shell `bash`, comparez les performances des deux versions pour une lecture/écriture d'un fichier raisonnablement volumineux (i.e. de plus de 1 Mo).
Exemple :

```
$ time ./my_cat /lib/libc.a > tst
  real    0m8.658s
  user    0m2.920s
  sys     0m5.740s
$
```

Interprétez les résultats obtenus. Quelles sont les commandes de plus bas niveau : `read/write` ou `fread/fwrite` ?

Exercice 18 ($\star$)

Essayez le programme suivant :

```
#include <stdio.h>
#include <unistd.h>

int main(int argc, char *argv[])
{
  fwrite("Hello", sizeof(char), 6, stdout);
  write(STDOUT_FILENO, "World !", 7);
  exit(EXIT_SUCCESS);
}
```

L'observation est-elle en accord avec votre interprétation des résultats précédents ?



3. Du programme au processus

3.1 Exécution d'un programme

Pour exécuter un programme, il suffit de le lancer depuis un terminal ou un interpréteur de commandes. À ce moment, le programme devient ce que nous appellerons désormais un *processus*.

Vocabulaire 3.1 — Processus. Un **processus** est un programme **en cours d'exécution**. Un processus mobilise de multiples ressources au sein du système dans lequel il s'exécute. On peut notamment citer (de façon non exhaustive) :

- une entrée dans la **table des processus** du système (cf. paragraphe ci-dessous) ;
- un **espace d'adressage virtuel** (cf. cours d'ETE8-IFON1 (O.S) au S8) ;
- une table de **descripteurs de fichiers**.

Donc quand vous lancez un programme, vous créez *de facto* un nouveau **processus** dans le système. Notez que ce processus est représenté et manipulé par le noyau du système à l'aide d'une structure de données appelée *Process Control Block* (ou PCB). Cette structure est stockée dans le système dans une table de taille finie, appelée la **table des processus** du système. **Quand tous les PCB ont été alloués, alors il n'est plus possible de créer de nouveaux processus** et seuls les processus déjà présents sont susceptibles de continuer leur exécution. Dès qu'un processus est terminé **et que son code de retour a été récupéré**, le PCB associé est libéré est peut être utilisé pour un nouveau processus. Nous reviendrons sur ce point important ultérieurement.

Programmes et processus

Dans la partie sur le système de fichiers, j'ai préféré utiliser le terme de programme plutôt que celui de processus. Il ne faut pas confondre les deux termes, qui ne sont pas interchangeables. En effet, il conviendrait d'utiliser le terme application plutôt que celui de programme et de se souvenir qu'une application peut éventuellement s'appuyer sur plusieurs processus. C'est le cas par exemple de la partie serveur d'une application de type client-serveur. Vous avez vu (ou allez voir) en cours de programmation réseau qu'il existe plusieurs architectures possibles pour mettre en place un tel serveur. Une des architectures est nommée multi-processus, où un processus est créé pour traiter

chaque nouvelle requête reçue par le serveur.

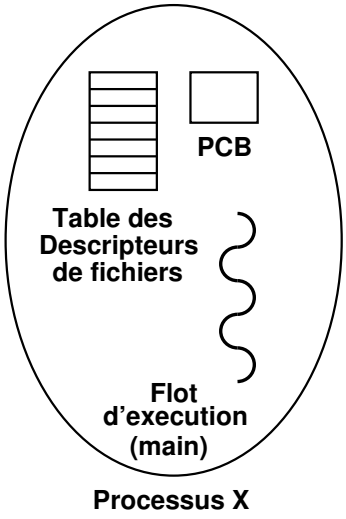


FIGURE 3.1 – Certaines des ressources occupées par un processus.



Le nombre de processus que l'on peut créer dans un système est donc **limité** par le nombre de PCB libres dans la table des processus. Cependant, lorsqu'il n'est plus possible de créer de nouveaux processus à un instant donné, cela ne veut pas dire que le système est bloqué puisque les processus déjà existants peuvent continuer leur exécution.



Il ne faut pas confondre un processus avec un programme exécutable. Un processus est une entité dynamique, tandis que le programme exécutable est simplement un fichier statique, stocké sur le disque dur.

Rappel

Quand un processus débute son exécution, il commence toujours par exécuter la fonction `main`. Par conséquent, un programme exécutable doit toujours contenir une telle fonction (et une seule, bien entendu). Le `main` est donc **le point d'entrée d'exécution d'un processus**.



Si un programme exécutable contient *toujours* un `main`, il est en revanche possible d'écrire du code ne possédant pas une telle fonction : par exemple, lorsque vous écrivez le code d'une bibliothèque (*library*), il n'y aura pas de `main`.

3.2 Identification d'un processus

Un processus est identifié de façon unique au sein du système par un numéro appelé *Process Identifier* (*Process ID*, ou plus simplement *PID*). C'est ce PID que vous allez utiliser pour désigner un processus particulier. Les processus peuvent interagir les uns avec les autres par le biais de ce numéro. Dès qu'un processus se termine, son PID est libéré.



Les PID sont attribués de façon **cyclique** et sont (probablement) codés à l'aide d'un entier non signé. Quand la dernière valeur possible de PID a été attribuée, on redémarre la numérotation du début et le système alloue les numéros disponibles. Donc, il est possible qu'un processus se voie attribuer le PID 3456 (par exemple) à un instant donné mais que ce numéro soit réalloué plus tard pour un autre processus, complètement différent du premier.

3.2.1 Obtenir le PID d'un processus

Un processus ne connait pas son propre PID quand il commence son exécution. Il peut cependant le connaître en appelant la fonction :

```
pid_t getpid(void);
```

Paramètres : Aucun.
Code de Retour : L'identifiant du processus appelant, de type `pid_t` (un autre *pseudo-type*).

Exercice 19 ($\star$)
Écrivez un programme qui affiche le PID de son processus.

3.2.2 Pour aller plus loin : identification des processus via un descripteur de fichier (Non POSIX, spécifique à Linux)

L'identification d'un processus par le truchement d'un PID peut poser des problèmes car ce numéro est recyclé par le système d'exploitation. Par exemple, quand un processus envoie un signal (cf. Chapitre 6) à un autre processus (cet envoi implique d'utiliser un PID), rien ne garantit que le processus auquel on souhaite envoyer le signal n'a pas disparu entre-temps et qu'il a été remplacé par un *autre* processus identifié par le même PID.

Afin de pallier ce genre de problème, une nouvelle interface est apparue dans les versions récentes de Linux (depuis la version 5 et + du noyau) [1] où un descripteur de fichier est associé à un processus. Ce descripteur restera valide dans le processus qui l'utilise et ce quel que soit l'état du processus-cible qu'il identifie. Cependant, vous noterez qu'un (index de) descripteur de fichier est une information locale à un processus tandis que le PID est une information globale (et donc utilisable par tous les processus du système).

La fonction permettant de récupérer un index de descripteur est la suivante :

```
int pidfd_open(pid_t pid, unsigned int flags);
```

Paramètres 1 : Le PID du processus auquel on veut associer un descripteur de fichier.
Paramètres 2 : De possibles options (paramètre non utilisé pour le moment).
Code de Retour : -1 si problème (cf. valeur `errno`) / (index de) descripteur de fichier associé au processus sinon.

Cependant, cette fonction n'est pas utilisable directement, il faut utiliser la fonction `syscall` car `pidfd_open` ne possède pas de *wrapper* dans la libc (avec un appel du style :

```
syscall(SYS_pidfd_open, pid, flags)
```

au lieu du simple appel :

```
pidfd_open(pid, flags).
```

1. Elle n'est donc pas POSIX.



L'utilisation de `pidfd_open` n'est pas l'unique façon d'obtenir un (index de) descripteur associé à un PID (processus) : comme les processus sont tous accessibles via un système de fichier particulier (dans *procfs*, un processus est considéré comme un répertoire dont le nom est `/proc/pid`), il est donc possible d'ouvrir le répertoire associé au processus-cible (avec la fonction `opendir`) puis de récupérer l'index de descripteur de fichier associé (avec la fonction `dirfd`).

Avec cette interface, il est désormais possible pour un processus de dupliquer un descripteur de fichier d'un *autre* processus avec la fonction `pidfd_getfd` :

```
int pidfd_getfd(int pidfd, int targetfd, unsigned int flags);
```

Paramètres 1 : le descripteur de fichier associé au processus-cible (obtenu avec `pidfd_open` par exemple).
Paramètres 2 : l('index d)e descripteur de fichier du processus-cible dont on veut créer un synonyme dans le processus appelant.
Paramètres 3 : De possibles options (paramètre non utilisé pour le moment).
Code de Retour : -1 si problème (cf. valeur `errno`) / (index de) descripteur de fichier sinon.

Tout comme `pidfd_open`, cette fonction n'est pas utilisable directement et il faut passer par l'intermédiaire de `syscall`.



L'identification d'un processus via un descripteur de fichier reste complètement cohérente avec le principe du « tout est fichier » (cf. Section 2.1).

3.3 Création de processus

Nous venons de voir comment créer un processus simplement : en lançant un nouveau programme depuis un terminal. Mais il est possible pour un processus donné de créer un nouveau processus. Cette création est effectuée en appelant la fonction `fork` dans le code de l'ancien processus :

```
pid_t fork(void);
```

Paramètres : Aucun
Code de Retour : -1 si problème (cf. valeur `errno`) / 0 ou PID du nouveau processus créé si succès (cf. Section 3.3)

Le processus qui engendre un nouveau processus est appelé **processus-parent** tandis que le processus nouvellement créé est appelé **processus-enfant**.

Fonctionnement de `fork`

En fait le fonctionnement de fork n'est pas intuitif. En effet, en appelant `fork`, on crée un nouveau processus-enfant, mais quel sera le code éxécuté par ce nouveau processus-enfant ? Quelle sera la fonction `main` qu'il va appeler ? Quand un processus appelle la fonction `fork`, il ne créé pas *ex-nihilo* un nouveau processus-enfant, il va en fait **se dupliquer** ! Par conséquent le processus-enfant nouvellement créé va exécuter le **même code que celui de son géniteur**.

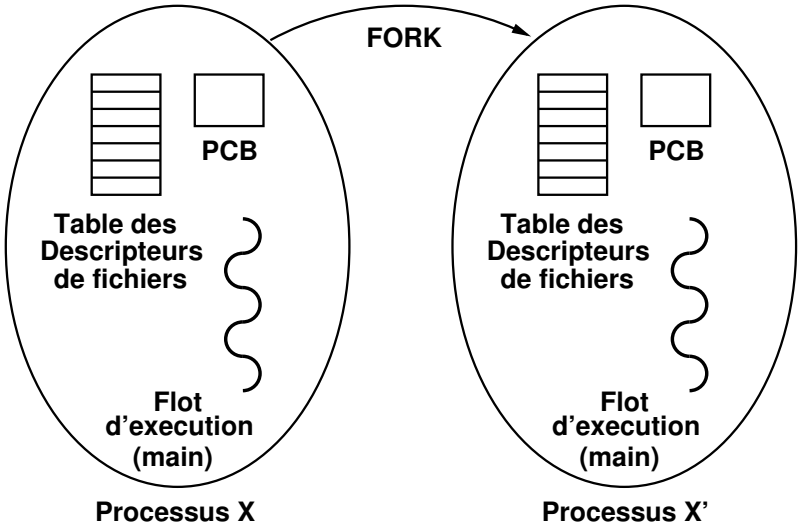


FIGURE 3.2 – Duplication d'un processus avec `fork` et copie des structures de données.



En fait, tout se passe comme si le nouveau processus-enfant commençait son exécution comme s'il **venait juste** de sortir de la fonction `fork` et non pas depuis le début du `main`.

Donc, si les deux processus (le parent **et** l'enfant) sortent tout les deux de la fonction `fork`, comment faire la distinction si on se trouve dans l'un ou dans l'autre processus ? La réponse est simple : il faut se servir du **code de retour de** `fork`, qui sera différent selon le cas. Nous aurons donc :

1. **un code de retour valant** -1 en cas d'impossibilité de création d'un nouveau processus.
2. **un code de retour valant** 0 si c'est le processus-**enfant** qui ressort du `fork`.
3. **un code de retour égal au PID du processus-enfant créé** si c'est le processus-**parent** qui ressort du `fork`.

Généalogie des processus

L'ensemble des processus du système forme une espèce de *généalogie* possédant certaines propriétés :

1. Un processus peut créer plusieurs enfants.
2. Un processus ne possède qu'un unique parent.
3. Un processus-enfant peut lui aussi créer d'autres processus à son tour.
4. Les processus s'exécutent **concurremment** dans le système (et donc en parallèle dans le cas de machines multicœur).
5. Aucune hypothèse n'est formulable sur la façon dont sont ordonnancés les processus du système. En particulier, ne supposez pas que le processus-enfant commence son exécution dès l'appel à `fork`, car cela dépend de l'implémentation, c'est donc non portable (et potentiellement faux).

Partage de données et indépendance des processus

Dans UNIX, tous les processus sont indépendants les uns des autres et ne partagent (a priori) rien entre eux. Donc les processus-enfant créés par un processus-parent ne partagent rien avec leur parent, sauf le fait qu'ils exécutent le même code. En particulier, les processus **affiliés** ne partagent aucune variable suite au `fork`. Ils ne partagent aucune donnée en fait (cf. Section 3.5).

Vous pensez posséder désormais des pouvoirs magiques ? Vous n'en n'êtes pas très loin [2].



Un autre exemple de `fork` incompréhensible.

Exercice 20 ($\star$)

Écrivez un programme qui crée un processus-enfant et affiche ensuite une chaîne de caractères.

— Que constatez-vous ?

— Est-ce cohérent par rapport à ce que vous connaissez du fonctionnement de `fork` ?

Exercice 21 ($\star\star$)

Reprenez le programme de l'exercice précédent et rajoutez un affichage de chaîne de caractères *avant* l'appel à `fork`.

— L'affichage est-il en accord avec vos attentes ? Pourquoi ?

— Que se passe-t-il lorsque il n'y a pas de caractère `\n` à la fin de la chaîne affichée par le `printf` placé avant le `fork` ?

Exercice 22 ($\star\star$)

Écrivez un programme qui crée **exactement** 10 processus-enfants.

Pour aller plus loin : création des dæmons.

Un *dæmon* est un processus serveur (le «`d`» à la fin de `ftp`, `sshd` ou encore `httpd` signifie « *dæmon* »). Pour créer un nouveau processus-dæmon, il est nécessaire de détacher le nouveau processus de son terminal de lancement.

```
int daemon(int nochdir, int noclose);
```

Paramètre 1 : Si égal à 0, pas de changement de répertoire de travail, sinon répertoire racine « / ».

2. Cf. le premier chapitre du livre d'Abelson et Sussman : « Structure et interprétation des programmes informatiques » et la notion de pensée magique.

Paramètre 2 : Si égal à 0, redirection des I/O vers `/dev/null`, pas de changement sinon.
Code de Retour : -1 en cas de problème/ 0

Cette fonction permet de ne pas utiliser la technique du `fork-setsid-fork` mais n'est pas POSIX. Pour respecter la norme, il faut donc créer une nouvelle session avec un appel à :

```
pid_t setsid(void);
```

Le processus qui va être transformé en démon ne doit pas être leader de session, d'où le second appel à `fork`.

3.4 Identifier le parent d'un processus

Il est possible pour un processus de connaître le PID de son parent en appelant la fonction `getppid` (pour Get Parent PID) :

```
pid_t getppid(void);
```

Paramètres : Aucun.
Code de Retour : Le PID du processus-parent du processus appelant.



Il existe donc une fonction pour connaître son propre PID et une autre pour connaître celui de son parent. En revanche, il n'existe pas de fonction pour connaître les PID des processus-enfants qu'un processus crée. En effet, comme un processus ne possède qu'un parent, il est facile de renvoyer un tel résultat, tandis que le nombre d'enfants n'étant pas fixé, il serait peu aisé de renvoyer une liste de PID. Cependant, vous pouvez connaître le PID d'un enfant au moment de sa création (c'est le code de retour de `fork` dans le cas du processus-parent). Il vous incombe donc de conserver cette information au fur et à mesure des créations successives.

Exercice 23 (⋆)
Écrivez un programme qui crée un processus-enfant puis affiche le PID de ce dernier tandis que le processus-enfant affichera le PID de son parent. La chaîne à afficher sera donc différente selon que l'on se trouve dans le parent ou dans l'enfant.

Exercice 24 (⋆)
Écrivez un programme qui affiche son PID puis crée un processus-enfant. Le processus-parent affiche ensuite le PID de son enfant tandis que son enfant affichera son propre PID puis celui de son parent. Vérifiez la cohérence de l'affichage.

3.5 Héritage inter-processus

Nous avons vu précédemment que les processus sont tous indépendants les uns des autres et qu'il ne partagent pas de données (pas de variables par exemple) en cas de duplication via `fork`. Cependant, il existe une notion d'**héritage** entre processus affiliés. Voici quelques éléments qui sont hérités du parent par l'enfant au moment de sa création :

— **la structure de l'espace d'adressage** de l'enfant est identique à celle de son parent. En particulier, si le parent a fait un `malloc` **avant** le `fork`, alors tout se passe comme si l'enfant avait également fait cet appel à `malloc` (alors que ce n'est pas le cas réellement). De la

même façon, si le parent initialise des variables avant le `fork`, elles seront initialisées avec les mêmes valeurs dans l'enfant. Enfin toutes les projections en mémoire [3] (appels à `mmap`, cf. Section 4.2.2) faites par le parent sont répercutées sur l'enfant.

> ! En résumé : toute manipulation de la mémoire avant l'appel à `fork` est répercutée dans le processus-enfant, même si ce dernier n'a pas effectué ces opérations *stricto sensu*.

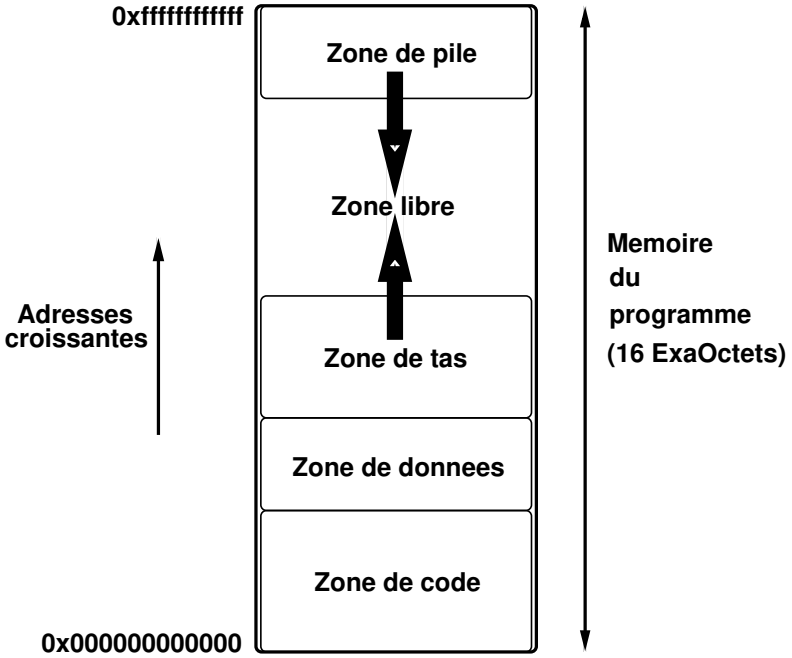


FIGURE 3.3 – Organisation des zones mémoire d'un processus.

— **la table des descripteurs de fichiers** de l'enfant est l'exacte **copie** de celle du parent au moment du `fork`.

> ! Si un processus ouvre un fichier avant l'appel à `fork`, alors l'enfant pourra accéder également à ce même fichier. Dans ce cas précis, la **position courante dans le fichier (*offset*) est partagée** par les processus parents et enfants. **Donc si le parent (resp. l'enfant) lit ou écrit dans un fichier, alors cette position courante sera modifiée pour l'enfant (resp. le parent) !** Dans ce cas, les descripteurs du parent et de l'enfant vont pointer sur la même structure de fichier ouvert [a]. Il n'y aura pas de structure dédiée à chaque descripteur. `fork` est donc le second cas (après `dup`) où le compteur de référence de la structure de fichier ouvert est incrémentée.
>
> *a.* Cela répond aussi à la question que nous nous posions en 2.2.3.

— **les traitants de signaux (cf. Section 6.4) et les masques de signaux (cf. Section 6.5)** sont hérités par l'enfant.
— les signaux pendants ne sont pas hérités (cf. Section 6.2.1).
— le répertoire de travail est hérité.

3. *Memory Mappings*

— le temps d'exécution de l'enfant est remis à zéro.
— les *callbacks* éventuels de fin de processus (cf. `atexit`, `on_exit`) sont hérités.



Le buffer utilisé de façon interne par les fonctions `printf/fprintf` est hérité par le processus-enfant s'il n'est pas vidé par le processus-parent avant de faire le `fork`.

Exercice 25 (⋆⋆)
Soit un processus qui ouvre un fichier régulier quelconque. Ce processus crée ensuite un processus-enfant à l'aide de `fork()`. Faites le schéma résumant l'état des structures de données : tables des descripteurs de fichiers, structures de fichiers ouverts et Table des I-Nœuds suite à ces opérations.

3.6 Modèle *fork/join*

Les processus que vous pouvez créer avec `fork` sont appelés de façon courante des processus UNIX ou encore des processus **lourds** (*heavyweight processes*). Ce terme désigne en fait les coûts associés à la gestion de ces entités :
— la création (cf. Section 3.3) ;
— la terminaison (cf. Section 3.7) ;
— les opérations de synchronisation (cf. Chapitre 8).
Nous verrons plus tard (cf. Chapitre 7) qu'il existe des processus pour lesquels ces coûts sont moindres. On parlera alors de **processus légers (ou *threads*)**.

Le modèle de programmation des processus UNIX est de type *fork/join*. C'est à dire qu'il existe une opération de création de processus (`fork` dans le cas présent) et une autre opération (le *join*) qui permet aux processus parents et enfants de synchroniser leurs exécutions respectives. Concrètement cette opération va permettre au processus-parent de prendre connaissance du code de retour de ses enfants et de **rester en sommeil** tant que ce code n'est pas disponible (tant qu'il n'y a pas un de ses enfants qui a terminé son exécution par exemple).

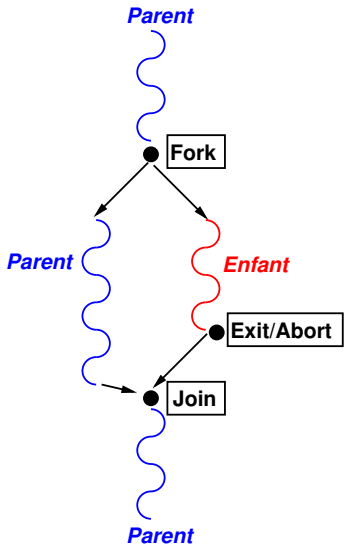


FIGURE 3.4 – Le modèle *fork/join*.



Ce code de retour est la valeur retournée par un processus quand il termine son exécution. En particulier, c'est la valeur passée en argument à la fonction `exit` (située n'importe où dans le programme) ou bien au `return` (uniquement dans le `main` du programme). Cette valeur peut donc être communiquée au processus-parent afin de le renseigner sur l'état d'exécution de son enfant (terminaison normale ou avec incident par exemple).

En pratique, la fonction permettant d'effectuer cette opération de synchronisation s'appelle `wait` :

```
pid_t wait(int *status);
```

Paramètre : Un pointeur avec des informations sur l'enfant terminé ou arrêté (`NULL` si paramètre inutilisé).

Code de Retour : -1 si problème (cf. valeur d'`errno`) / PID d'un processus-enfant terminé si succès.

Un processus appelant cette fonction va rester **bloqué** jusqu'à ce qu'un de ses processus-enfants soit stoppé ou ait terminé son exécution. Le processus-parent peut prendre connaissance du code de retour de l'enfant terminé à l'aide du paramètre `status`. Ce paramètre `status` ne **doit pas être consulté directement**, il faut utiliser un ensemble de macros. Voici quelques exemples (regarder le manuel pour avoir une liste plus complète) :

— `WIFEXITED(status)` : retourne `TRUE` si l'enfant s'est terminé correctement (i.e. appel à `exit`, `_exit` ou `return` dans le `main`)
— `WEXITSTATUS(status)` : retourne les 8 bits de poids faible de l'entier renvoyé par le processus-enfant lors de son appel à `exit`, `_exit` ou `return` dans le `main`
— `WIFSIGNALED(status)` : retourne `TRUE` si l'enfant s'est terminé suite à la réception d'un signal (cf. Chapitre 6)
— `WTERMSIG(status)` : retourne le numéro du signal qui a causé la terminaison de l'enfant



Il n'est **pas possible de choisir l'enfant** que l'on souhaite attendre avec `wait`.

Exercice 26 (⋆)

Écrivez un programme qui :

1. crée un processus-enfant
2. effectue une petite boucle d'attente active, par exemple :
```
do{int toto = 0;
while(++toto);
}while(0);
```
3. se synchronise avec cet enfant pour sa terminaison

Exercice 27 (⋆)

Reprenez le programme précédent et affichez dans le parent le PID de l'enfant terminé, ainsi que son code de retour. Le processus-enfant affichera son PID, celui de son parent puis attendra un petit peu avant d'appeler `exit`. Vous donnerez une valeur de retour bien identifiable pour l'enfant (i.e ni 0, 1 ou -1).

La fonction `wait` possède deux défauts qui restreignent son utilisation :
— seul un fonctionnement bloquant est disponible ;
— le processus-parent ne peut pas choisir le processus-enfant qu'il souhaite attendre.
Pour un contrôle plus fin, il est recommandé d'utiliser à sa place la fonction `waitpid` qui permet de s'affranchir de ces limitations :

```
pid_t waitpid(pid_t pid, int *status, int options);
```

Paramètre 1 : Le PID du processus-enfant avec lequel le parent veut se synchroniser.
Paramètre 2 : Un pointeur avec des informations sur l'enfant terminé/stoppé (`NULL` si paramètre inutilisé).
Paramètre 3 : Options éventuelles pour l'appel à la fonction.
Code de Retour : -1 si problème (cf. la valeur d'`errno`) / 0 (si option `WNOHANG`) / PID du processus-enfant terminé.

Au niveau des options, il en existe une très importante, qui permet de faire basculer cette fonction en mode **non-bloquant** : c'est l'option `WNOHANG`. Quand elle est utilisée, `waitpid` renvoie la valeur 0 et retourne immédiatement sans bloquer le processus appelant dans le cas où il n'y a pas d'enfant terminé ou stoppé. Pour avoir l'équivalent d'un `wait`, mais non-bloquant, il suffit d'appeler[4] (cf. le manuel) :

```
waitpid(-1, &status, WNOHANG);
```



La norme POSIX recommande de ne plus utiliser `wait` ni de pas passer la valeur `-1` en premier argument à `waitpid` car cela ne permet pas une synchronisation fine avec les processus-enfants. En effet, dans les cas où un processus-parent ferait appel à des fonction comme `system` (cf. Section 3.10) ou `popen/pclose` (cf. Section 4.1.5), l'utilisation de `wait` ou `waitpid(-1,...)` pourrait avoir des interactions indésirables. Dans de telles éventualités, il est donc recommandé d'utiliser `waitpid` avec un PID explicite en premier argument. Dans le cas d'un appel à `waitpid` dans un traitant de signal pour `SIGCHLD`, il est toujours possible d'avoir le PID du processus émetteur dudit signal en utilisant le traitant étendu (cf. Section 6.4.4) .

Exercice 28 (⋆⋆)
Écrivez un programme qui crée quelques processus-enfants, puis fait une boucle d'attente active et enfin les attend dans l'ordre décroissant de leur valeur de PID. Les processus-enfants feront quant à eux une boucle d'attente (active ou passive) de durée variable selon leur PID.

3.7 Terminaison d'un processus

Un processus se termine normalement (i.e. sans problème d'exécution) quand il effectue un appel à la fonction `exit` (ou `_exit`) ou bien lorsqu'il atteint l'instruction `return` dans la fonction `main` (qui en fait appelle `exit` dans ce cas précis). Dès qu'un processus a fini son exécution, il notifie à son parent sa terminaison en lui envoyant un signal de type `SIGCHLD` (cf. Chapitre 6). Ensuite il passe dans un état spécifique : l'état *zombie*[5].

4. On suppose ici que la variable `status` est déclarée sous la forme `int status`
5. Vous verrez plus tard qu'il ne sert à rien de tuer un zombie : il est **déjà** mort ...

Vocabulaire 3.2 — Processus zombie. Un **processus zombie** est un processus dont l'exécution est terminée, mais qui est encore présent dans le système. Un tel processus n'occupe plus le processeur, ne possède plus d'espace d'adressage virtuel, mais mobilise encore des ressources du système. En particulier, **un processus zombie occupe un PCB** (cf. Section 3.1) dans la table des processus.



Le PCB du zombie reste occupé tant que le parent de ce processus n'aura pas pris en compte sa terminaison. C'est pourquoi il est **indispensable** de faire un appel à `wait` ou `waitpid` pour chaque processus-enfant créé, sinon, il peut arriver que la création de nouveaux processus soit impossible, car tous les PCB sont occupés par des zombies.



Un processus-parent n'est pas obligé de prendre en compte le code de retour de ses enfants, mais il est est obligé de prendre en compte le fait que ces derniers ont terminé leur exécution (i.e. un appel à `wait(NULL)` est suffisant) pour qu'ils libèrent leur PCB.

Exercice 29 (⋆⋆⋆)
Pouvez-vous donner un type de programme qui ne fonctionnera pas correctement (voire pas du tout) à cause d'un nombre de zombies trop important ?

Comme c'est le parent d'un processus qui est responsable de son élimination totale du système, on peut se poser la question suivante : que se passe-t-il si jamais un parent se termine avant ses enfants sans avoir fait appel à la fonction `wait` ? À ce moment, il n'y a plus de processus capable d'éliminer les zombies du système et donc ce dernier peut potentiellement se retrouver dans l'incapacité de créer de nouveaux processus. Le problème vient du fait de la présence de processus à la fois zombies **et** orphelins [6]. Dans ce cas, c'est un autre processus qui va **adopter** tous ces zombies orphelins afin de pouvoir les éliminer (un beau *happy end*, en somme). Le processus en question est *systemd* [7], le tout premier processus du système, celui à partir duquel tous les autres sont dupliqués [8].

Exercice 30 (⋆⋆)
Écrivez un programme qui crée un processus-enfant et qui se termine immédiatement.
Le processus-enfant devra s'endormir pendant quelques secondes puis afficher le PID de son parent.
— Exécutez ce programme plusieurs fois. Qu'observez-vous ?
— Est-ce cohérent ?

Pour aller plus loin : Retour sur la synchronisation parent-enfant

`SIGCHLD` n'est pas envoyé seulement au moment de la terminaison, mais plus généralement quand le processus-enfant connait des événements d'exécution comme stop, cont et terminaison (évidemment). `wait` (ou `waitpid`) permet de se synchroniser avec un process qui envoie `SIGCHLD`. Il faut tester la variable `status` pour savoir si l'on a bien affaire à une véritable terminaison (ou pas) avec les macros `WIFSTOPPED(status)` ou `WIFCONTINUED(status)`

6. C'est triste, vraiment.
7. Le remplaçant d'init
8. Je vous laisse réfléchir à la portée philosophique de ce fait.

3.8 États d'un processus

Nous venons de voir qu'un processus peut être dans un état zombie, par opposition à son état « normal », c'est-à-dire en exécution. Les choses sont toutefois un peu plus complexes et en réalité un processus peut se trouver à un instant donné dans un des états suivants :

prêt : le processus est prêt à s'exécuter. C'est l'état dans lequel il se trouve quand il est créé au moment de l'appel à `fork`. Un tel processus peut se voir attribuer un processeur pour pouvoir s'exécuter.

actif : le processus est en cours d'exécution, il occupe un (ou des) processeur(s) et donc les instructions du programme sont exécutées. Notez qu'en fait cet état se décompose en deux sous-états : **actif user** et **actif kernel** [9].

bloqué/endormi : le processus n'occupe plus la ressource processeur, mais pourra la récupérer plus tard, quand certaines conditions seront remplies. Cet état existe pour optimiser l'utilisation des ressources du système : en effet il ne sert à rien de donner le processeur à un processus qui ne peut rien en faire. Une fois débloqué/réveillé, un processus bascule dans l'état « prêt ». Par exemple, un processus qui fait un appel à `wait` va rester dans l'état bloqué jusqu'à ce qu'un de ses enfants se termine. Il sera alors débloqué et pourra reprendre son exécution.

zombie : le processus ne s'exécute plus (il n'occupe plus le processeur), mais il mobilise encore des ressources dans le système (et notamment un PCB dans la table des processus).

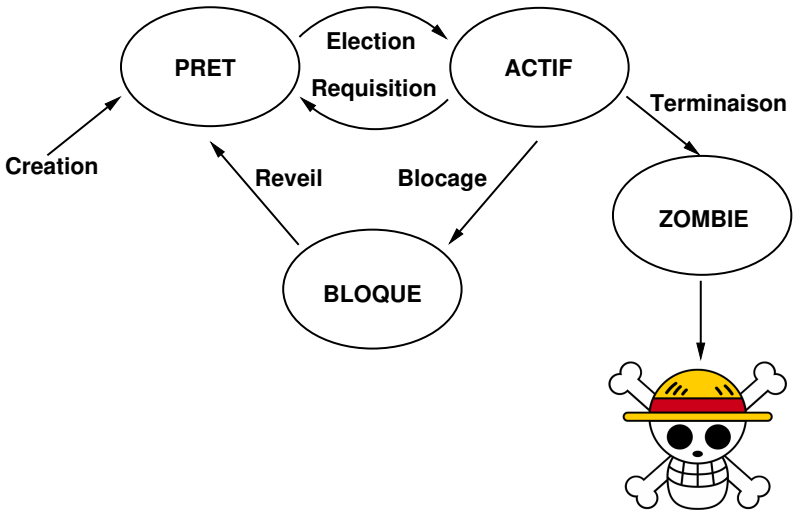


FIGURE 3.5 – Diagrammes des états d'un processus (avec les transitions).

Il est possible de mettre en sommeil volontairement un processus à l'aide de la fonction `sleep` :

```
unsigned int sleep(unsigned int seconds);
```

Dans ce cas, le processus va faire ce que l'on appelle de **l'attente passive**, par opposition à de **l'attente active**, où il occupe effectivement le processeur pendant qu'il attend [10].

9. Une transition de l'état actif user vers actif kernel (ou l'inverse) est un changement de contexte intra-processus.
10. C'est le cas de : `do{int toto = 0 ; while(++toto);}while(0);` par exemple.



L'attente passive permet d'economiser les ressources du système, elle est en règle générale préférée à l'attente active.

Exercice 31 ($\star$)
Reprenez un des exercices précédents avec de l'attente active et remplacez-la par un appel à `sleep`. Qu'observez-vous ?

Un processus peut également décider de rendre la ressource processeur qu'il occupe à un autre processus (qu'il ne choisit pas) en appelant la fonction `sched_yield` :

```
int sched_yield(void);
```

3.9 Recouvrement

Jusqu'à présent, nous avons vu comment créer de nouveaux processus par duplication (`fork`), et comment synchroniser la terminaison des enfants avec leur parent (`wait`/`waitpid`). Pour rappel, un processus dupliqué exécute le même code que celui de son parent, ce qui peut être perçu (sans doute à juste titre) comme une restriction. Comment faire pour que ce processus nouvellement créé puisse exécuter n'importe quel code ? Cette opération existe dans UNIX et s'appelle une opération de **recouvrement**, car tout se passe comme si l'espace d'adressage du processus (qui comprend notamment le code a exécuter) était **recouvert** par un nouvel espace d'adressage, avec un nouveau code et de nouvelles données.

Retour sur le `main`

Ainsi qu'évoqué précédemment (cf. Section 3.1), le point d'entrée de l'exécution d'un processus est la fonction `main`. Vous connaissez le prototype de cette fonction :

```
int main(int argc, char *argv[]);
```

Pour rappel, `argc` est le nombre d'arguments du programme (et le nom du programme fait partie de ces arguments, donc `argc` est toujours au moins égal à 1), tandis que `argv` est un tableau de chaînes de caractères contenant les arguments proprement dits. Mais `main` peut éventuellement accepter un argument supplémentaire, `envp`. Cet argument supplémentaire va également être un tableau de chaînes de caractères, servant à stocker les **variables d'environnement** du programme. Il est ainsi possible de consulter l'environnement du programme autrement que par des appels aux fonctions habituelles telles que `getenv` et `setenv` [11]. Le prototype de `main` dans ce cas sera alors :

```
int main(int argc, char *argv[], char *envp[]);
```

Fonctions de recouvrement

La fonction de base permettant de faire cette opération de recouvrement est `execv` :

```
int execv(const char *name, char *const argv[]);
```

Paramètre 1 : Le nom du fichier exécutable (ou du script) à exécuter (une chaîne de caractères).
Paramètre 2 : Les arguments du programme (un tableau de chaînes de caractères).
Code de Retour : -1 si problème (cf. la valeur `errno`) / pas de retour si succès !

11. Voir le manuel à ce sujet.

Le fonctionnement d'`execv` est simple sur le principe : quand un processus fait un appel à cette fonction, alors tout se passe comme s'il exécutait le `main` du programme exécutable (ou le script) dont le nom est pris en (premier) argument. Le tableau `argv` de ce `main` sera en fait celui passé en paramètre à la fonction `execv`. Ce processus « saute » sur un nouveau code et l'ancien n'existe plus ! Le programme (ou script) a exécuter est recherché dans le **répertoire de travail** du processus appelant (en cas de nom relatif, bien entendu).



Attention à la construction de ce tableau d'arguments ! Par convention, la première chaîne de ce tableau **doit être le nom du programme** et la dernière **doit être** `NULL` : on se sert de ce pointeur pour connaître le nombre d'arguments du programme à exécuter, c'est-à-dire le `argc` en premier argument du `main` (cf. l'exemple d'appel à `execl` ci-dessous).



`execv` ne retourne une valeur qu'en cas d'échec, car si l'appel réussit, alors on exécute un **autre code** dans le processus, qui n'a plus rien à voir avec le code qui a fait l'appel à `execv` ! Donc, du code situé **après** un appel à `execv`, ne sera **jamais** exécuté, sauf en cas d'échec de l'appel à cette fonction. Il est donc totalement inutile de placer un `printf` après cet appel pour vérifier qu'il a correctement fonctionné ...

Exercice 32 ($\star$)
Créez un premier programme basique (qui effectue juste un appel à `printf` par exemple) puis créez un second programme qui effectue un recouvrement avec le code de ce premier programme.

Exercice 33 ($\star\star$)
Reprenez le programme de l'exercice précédent, mais le recouvrement devra être effectué par un processus-enfant créé dans le second programme.

Exercice 34 ($\star\star$)
Reprenez l'exercice précédent, mais le programme utilisé pour le recouvrement effectuera la somme de ses paramètres (dont on on supposera que ses arguments sont uniquement des entiers).

Il existe d'autres fonctions au fonctionnement similaire, mais offrant des possibilités supplémentaires :

```
int execvp (const char *name, char *const argv[]);
int execve (const char *path, char *const argv[], char *const envp[]);
int execvpe(const char *name, char *const argv[], char *const envp[]);
```

`execve` et `execvpe` prennent un paramètre supplémentaire :
Paramètre 3 : les variables d'environnement pour l'exécution du programme (un tableau de chaînes de caractères). Ces variables sont stockées sous la forme `key=value`.

Cet argument additionnel permet de passer un nouvel environnement au processus qui va effectuer le recouvrement. Tout se passe comme si un appel à `execve` ou `execvpe` faisait exécuter le `main` du programme exécutable avec les paramètres `argv` et `envp` passés en arguments à ces fonctions.

Par défaut, la recherche du programme exécutable se fait dans le répertoire de travail du processus appelant. S'il n'y est pas trouvé, alors la fonction renvoie une erreur. Dans le cas des fonctions `execvp` et `execvpe`, cette recherche se fait dans le `PATH` du processus. Dans ce cas, il suffit de préciser le nom du programme (ou du script), il n'est pas nécessaire de donner le chemin absolu de ce dernier.

Il existe également trois autres fonctions de recouvrement, et la différence est juste d'ordre pratique, et concerne la façon dont sont précisés les arguments du programme à exécuter. En effet, avec les fonctions vues précécemment, il faut construire un **vecteur** d'arguments [12], ce qui n'est pas toujours pratique. Vous pouvez donc passer les arguments du programme sous la forme d'une liste (i.e. les uns à la suite des autres), chaque argument du programme formant un paramètre de la fonction utilisée pour le recouvrement. Ces fonctions possèdent un `l` (comme *list*) dans leur nom, pour le reste, leur utilisation est similaire à celles vues juste au-dessus.

```
int execl (const char *path, const char *arg, ... /*, (char *)NULL */);
int execlp(const char *name, const char *arg, ... /*, (char *)NULL */);
int execle(const char *path, const char *arg, ... /*, (char *)NULL */,
.                                                  char *const envp[]);
```



N'oubliez pas que le nom d'un programme est son premier argument ! Il doit donc être mis dans la liste ou le tableau passé en paramètre à la fonction `execXX`. N'oubliez pas non plus que le dernier argument doit être le pointeur `NULL`. Par exemple, si je veux exécuter le programme `toto` (en supposant qu'il se trouve dans le répertoire de travail du processus) :

```
execl("./toto","toto",NULL);
```

Le premier argument est le nom de l'exécutable et les second et troisième arguments sont les paramètres de `toto` ; comme il n'en n'a pas, c'est juste son nom et la liste se termine par `NULL`.

Après l'`exec` ...

Il ne faut pas croire qu'effectuer un recouvrement revient à effectuer une sorte de « remise à zéro » du processus. En effet, même si l'espace d'adressage est complètement recouvert suite à l'appel à une fonction de la famille `exec`, le comportement du processus est tout de même dépendant ce qui a été effectué **avant** ce recouvrement. Voici une liste non-exhaustive de ce qui est conservé ou non après un recouvrement :

Au niveau des signaux :

— Les traitants de signaux (cf. Section 6.4) sont réinitialisés après `exec` ;
— Les masques de signaux (cf. Section 6.5) sont conservés après l'appel à `exec` ;
— Les piles alternatives de traitants de signaux ne sont pas conservées.

Au niveau de l'espace d'adressage :

— Les mappings mémoire (`mmap`, cf. Section 4.2.2) ne sont pas conservés (en particulier les zones de mémoire partagée de type POSIX (cf. Section 4.2.3)) ;
— Les segments de mémoire partagée de type system V sont détachés (cf. Section 4.2.1) ;
— Les sémaphores POSIX sont fermés (cf. Section 8.1.2) ;
— Les callbacks éventuels de fin de processus (cf. `atexit`, `on_exit`) ne sont pas conservés.

12. D'où le `v` dans le nom des fonctions.

Sinon :

— les descripteurs de fichiers sont fermés suite à un appel à `exec` **uniquement** si une option particulière a été positionnée soit à la création du descripteur (option `O_CLOEXEC`), soit plus tard avec la fonction `fcntl` (option `FD_CLOEXEC`) ; **Dans le cas contraire, les descripteurs restent utilisables dans le nouveau code exécuté avec** `exec`.
— les verrous pris sur des fichiers sont conservés (cf. section 8.2.3) ;
— le PPID est conservé après un exec ;
— la valeur de `nice` est conservée.



Le point concernant les descripteurs de fichiers est fondamental !

Exercice 35 (⋆⋆)

Écrivez un programme qui ouvre un fichier quelconque et affiche la valeur (de l'index) du descripteur associé. Exécutez ce programme :

1. en le lançant directement depuis un terminal
2. en créant un processus-enfant qui ouvre d'abord un fichier (le même ou un autre, peu importe) puis qui effectue un recouvrement (via `exec`) avec le premier programme.

— Que constatez-vous ?
— L'affichage est-il cohérent ?

Exercice 36 (⋆)

Reprenez l'exercice précédent en positionnant l'option `O_CLOEXEC` au moment de l'ouverture du fichier avant d'effectuer le recouvrement (ie. l'appel à `exec`). Que constatez-vous ?

3.10 Pour aller plus loin : exécution de commandes shell dans un programme

Un·e programmeur·se peut être parfois amené·e à vouloir exécuter une (série de) commande(s) shell au sein d'un programme C. Si cette série de commandes utilise des redirections ou des tubes de communication (cf. Section 4.1), il peut-être utile de laisser le shell procéder à l'interprétation de cette (série de) commande(s). C'est typiquement le service qui est rendu par un appel à la fonction `system`.

L'appel à cette fonction déclenche essentiellement trois choses :

1. la création d'un processus-enfant (comme si `fork` avait été appelé) par le processus appelant qui devient donc un processus-parent ;
2. un recouvrement par le processus-enfant afin d'exécuter la (série de) commande(s). Tout se passe comme si un appel à `execl("/bin/sh", "sh", "-c", command, NULL);` était effectué par le processus-enfant ;
3. le processus-parent attend la terminaison de son processus-enfant, comme si un appel à `waitpid` avait été effectué.

Par ailleurs, les signaux `SIGINT` et `SIGQUIT` sont ignorés, tandis que le signal `SIGCHLD` est bloqué.

```
int system(const char *command);
```

Paramètre 1 : Une chaîne de caractère correspondant à la série de commandes shell à éxécuter. Dans le cas ou le pointeur `NULL` est donné en argument, l'appel permet de connaître l'état de disponibilité d'un shell ou non sur le système.

Code de Retour :

- Si le processus-enfant ne peut pas être créé, -1 est retourné et le type d'erreur est positionné dans `errno` ;
- Si un shell ne peut pa être exécuté dans le processus-enfant, la valeur de retour est comme si ce processus avait appelé `_exit(127)` ;
- Si tous les appels-systèmes dans la (série de) commande(s) se déroulent correctement, la valeur de retour est celle du processus-enfant (i.e le code de retour de la dernière commande exécutée par ce dernier).

Dans les deux derniers cas, le code de retour est un « status » consultable via les macros dédiées (cf. Section 3.6).



4. Communication inter-flots

4.1 Tubes

Les tubes (*pipes*) de communication sont un moyen de communication inter-flots,pour des entités appartenant à la **même machine**. Les tubes sont des fichiers (car ils appartiennent au système de gestion de fichier [1]), mais d'une utilisation particulière. Les conséquences sont les suivantes :

— la manipulation d'un tube se fait via des descripteurs de fichier ;
— la lecture/écriture dans un tube s'effectue avec les appels-système `read`/`write` ;
— les tubes sont utilisables pour mettre en place des **redirections d'entrées/sorties**.

4.1.1 Caractéristiques

Les tubes possèdent les caractéristiques suivantes :

1. ce sont des mécanismes de communication **unidirectionnels**.
2. un tube possède deux extrémités **jouant chacune un rôle différent** :
   — une extrémité en lecture ;
   — une extrémité en écriture.
   À chacune de ces extrémités correspond un descripteur de fichier. Pour un tube, il existera au plus deux structures de fichier ouvert.
3. un tube est géré en mode FIFO
4. les données qui transitent dans le tube constituent un **flux** : toute lecture dans le tube **détruit les données lues**, laissant ainsi de la place pour en écrire de nouvelles.
5. les lectures et les écritures sont **décorellées** [2]
6. un tube possède une **capacité (taille) finie** .

Vocabulaire 4.1 — Nombre de lecteurs. Le nombre de lecteurs d'un tube est le nombre de descripteurs de fichiers associés à l'extrémité en lecture de ce tube.

1. Et donc, pour un tube donné, il existe un unique I-Nœud associé.
2. C'est un flux qui transite dans le tube et pas des messages.

Vocabulaire 4.2 — Nombre d'écrivains. Le nombre d'écrivains d'un tube est le nombre de descripteurs de fichiers associés à l'extrémité en écriture de ce tube.



Le nombre de lecteurs (écrivains) d'un tube n'est donc **pas le nombre de processus** (*a priori*) qui lisent à partir dudit (écrivent dans ledit) tube.



Il faut pas confondre le fait qu'un tube est unidirectionnel avec le fait qu'un processus soit cantonné au rôle exclusif de lecteur ou celui d'écrivain (ce qui est souvent le cas). Il est tout à fait possible pour un processus de jouer ces deux rôles dans un même programme.

4.1.2 Création et accès à un tube

Il existe plusieurs façons d'accéder à un tube, qui dépendent des processus l'utilisant. En effet, les systèmes supportent à la fois des tubes **anonymes** et des tubes **nommés** :

— Un processus peut accéder à un tube anonyme soit en le créant avec l'appel-système `pipe` (cf. ci-dessous), soit **en héritant** les descripteurs associés avec un `fork`. Par conséquent, **seuls les processus affiliés peuvent utiliser des tubes anonymes** [3].

— Un processus peut toujours accéder à un tube nommé à partir du moment où il connait le nom (de fichier) dudit tube. Des processus quelconques peuvent donc utiliser des tubes nommés, qu'ils soient affiliés ou non.

Cas des tubes anonymes

Un flot peut créer et ouvrir un tube anonyme avec l'appel-système `pipe`, dont le prototype est le suivant :

```
int pipe(int pipefd[2]);
```

Paramètre : Un tableau de deux (index de) descripteurs de fichiers.
Code de Retour : -1 si problème (cf. valeur d'`errno`) / 0 si succès.

Par convention, `pipefd[0]` est l'index de descripteur correspondant à l'extrémité en lecture du tube, tandis que `pipefd[1]` est l'index de descripteur correspondant à l'extrémité en écriture du tube.



Il est totalement **inutile** d'initialiser le tableau pris en argument ! Les valeurs seront écrasées au retour de l'appel-système `pipe`.

Par défaut, les lectures/écritures dans un tube anonyme sont **bloquantes**, il est toujours possible de modifier ce comportement avec un appel à la fonction `fcntl` (cf. Section 2.4.1).

Il est également possible de créer un tube en positionnant les options idoines, grâce à la fonction `pipe2` (fonction spécifique à Linux, et donc non-portable) :

3. De façon plus générale, une ressource anonyme n'est utilisable qu'entre processus affiliés.

```
int pipe2(int pipefd[2], int flags);
```

Paramètre 1 : Un tableau de deux (index de) descripteurs de fichiers.
Paramètre 2 : Des options de création du tube. Par exemple :
- `O_CLOEXEC` permet de fermer automatiquement les extrémités du tube en cas de recouvrement (appel à `exec`) ;
- `O_NONBLOCK` permet de passer le tube en mode non-bloquant pour les écritures et les lectures ;
- `O_DIRECT` : permet de faire transiter des « paquets » dans le tube. Chaque écriture dans le tube est traitée comme un paquet distinct et chaque lecture permet de récupérer *un* paquet à la fois. Une écriture supérieure à `PIPE_BUF` (cf. 4.1.4) sera scindée en plusieurs paquets. Si un buffer en lecture a une taille inférieure au paquet à lire, alors les données exédentaires sont perdues (un buffer de taille `PIPE_BUF` permet donc de récupérer le plus grand paquet possible). Un paquet ne peut pas avoir une taille nulle. Cette option peut également être positionnée ultérieurement avec un appel à `fcntl`.

Code de Retour : -1 si problème (cf. valeur d'`errno`) / 0 si succès.

Cas des tubes nommés

Un tube nommé peut être créé avec un appel à la fonction `mkfifo` :

```
int mkfifo(const char *pathname, mode_t mode);
```

Paramètre 1 : Le chemin du tube à créer.
Paramètre 2 : Les droits d'accès sur le tube (cf. version d'`open` avec trois arguments en section 2.4.1).
Code de Retour : -1 si problème (cf. valeur d'`errno`) / 0 si succès.

Par défaut, un tube nommé est bloquant. De plus, cette fonction crée le tube mais ne l'**ouvre pas**, à la différence de `pipe`. Il faut donc utiliser l'appel-système `open` pour obtenir des (index de) descripteurs correspondants aux extrémités en lecture (mode : `O_RDONLY`) ou écriture du tube (mode :`O_WRONLY`) [4].

- En cas d'ouverture bloquante : les processus restent bloqués dans l'appel à `open` tant qu'il n'y a pas de lecteurs et pas d'écrivains. Ceci constitue donc un **mécanisme de synchronisation** inter-processus. La conséquence est qu'il est impératif d'ouvrir plusieurs tubes dans le **même ordre** pour chaque processus.
- En cas d'ouverture non-bloquante :
  - si ouverture de l'extrémité en lecture, il y a acquisition d'un descripteur, de façon classique
  - si ouverture de l'extrémité en écriture, il y aura un échec si le nombre de lecteurs est nul

Choix du rôle pour un processus

Techniquement, rien n'empêche un processus d'être à la fois lecteur et écrivain pour un tube donné auquel il a accès. Donc, il serait théoriquement possible d'utiliser un unique tube pour mettre en place des échanges bidirectionnels entre deux processus : il faudrait alors utiliser le tube en mode *half-duplex*, c'est-à-dire qu'un processus qui voudrait écrire dans le tube devrait s'assurer au préalable que les données éventuellement contenues ont bien été récupérées par l'autre processus (et donc que le tube est vide). Ce mode de fonctionnement est plutôt contraignant et assez peu pratique à mettre en place. Donc, s'il est techniquement possible d'utiliser un tube en mode *half-duplex*,

4. Un tube étant unidirectionnel, l'ouverture en mode `O_RDWR` n'a aucun sens.

on optera en pratique pour l'utilisation d'une **paire de tubes** afin d'obtenir un fonctionnement **bidirectionnel** (i.e. *full-duplex*) pour les communications inter-processus s'appuyant sur les tubes [5].

Cependant, les algorithmes de lecture et d'écriture dans un tube étant basés sur les nombres de lecteurs et d'écrivains, **il est impératif de systématiquement fermer toutes les extrémités inutiles des tubes de communication** afin de ne pas fausser ces nombres et d'éviter des problèmes (interblocages, erreurs non reportées, etc.)

Au final, pour un tube donné :

— dans le cas d'un tube anonyme : un processus sera lecteur de ce tube s'il en ferme l'extrémité en écriture. Réciproquement, un processus sera écrivain dans ce tube s'il en ferme l'extrémité en lecture. Donc la fermeture des extrémités inutiles d'une tube revient à imposer un rôle aux processus l'utilisant.

— dans le cas d'un tube nommé : un processus sera lecteur de ce tube s'il ouvre le fichier correspondant en lecture seule. Ce même processus sera écrivain dans ce tube s'il ouvre le fichier correspondant en écriture seule.



« Fermer les extrémités inutiles d'un tube anonyme, tu devras. »

4.1.3 Lecture dans un tube

Pour lire dans un tube, il faut utiliser l'appel-système `read`. Cependant, dans le cas d'un tube, le comportement de `read` est spécifique. Si l'on suppose que l'on effectue l'appel :

```
nb_lu = read(pipefd[0], buffer, taille_demandee);
```

alors :

1. **si le tube n'est pas vide** et contient `n` octets, alors `read` extrait du tube `nb_lu = min(n,taille_demandee)` octets qui sont copiés à l'adresse `buffer`.
2. **si le tube est vide**
   - 2.1 **si le nombre d'écrivains est nul** : la fin de fichier est atteinte, aucun octet n'est lu : `nb_lu` est égal à 0. En effet on cherche à lire dans un tube où personne n'écrit ...
   - 2.2 **si le nombre d'écrivains n'est pas nul**
     - 2.2.1 **si la lecture est bloquante** (cas par défaut) : le processus appelant est mis en sommeil jusqu'à ce que des données soient disponibles dans le tube.

5. Le problème sera encore plus importants dans le cas où il faudrait gérer de multiples lecteurs et/ou écrivains utilisant le même tube.

2.2.2 **si la lecture est non-bloquante** : le retour de la fonction est immédiat et la valeur de retour est égale à -1 et `errno` vaut `EAGAIN`

4.1.4 Écriture dans un tube

De la même façon, l'écriture dans un tube se fait avec l'appel-système `write`, qui là encore adopte un comportement particulier dans le cas d'un tube. Supposons que l'on effectue l'appel suivant :

```
nb_ecrit = write(pipefd[1], buffer, n);
```

alors :

1. **si le nombre de lecteurs du tube est nul** : alors on écrit dans un tube mais aucun flot ne lira ces informations. Dans ce cas, le processus appelant reçoit un signal `SIGPIPE`, Le comportement par défaut d'un processus reçevant un tel signal est de se terminer immédiatement [6].
2. **si le nombre de lecteurs du tube n'est pas nul**
   - 2.1 **si l'écriture est bloquante** : la fonction retourne quand `n` octets ont été écrits dans le tube.
     - 2.1.1 **Si** `n <= PIPE_BUF` (`PIPE_BUF` est une constante du système, cf. point ci-dessous), cette écriture est **atomique**. Il est donc possible que le processus soit mis en sommeil en attendant que le tube se vide.
     - 2.1.2 **Si** `n > PIPE_BUF`, il n'y a plus de garantie d'atomicité et un entrelacement des écritures est possible dans le cas d'écrivains multiples.
   - 2.2 **si l'écriture est non-bloquante**
     - 2.2.1 **si** `n > PIPE_BUF` : la fonction retourne un nombre compris entre 1 et`n` s'il reste de la place dans le tube et une erreur sinon. On aura donc une écriture **partielle** et un possible entrelacement dans le cas d'écrivains multiples sur le tube.
     - 2.2.2 **si** `n <= PIPE_BUF` **et s'il y au moins** `n` **octets libres dans le tube**, alors une écriture **atomique** est réalisée
     - 2.2.3 **si** `n <= PIPE_BUF` **et s'il y a moins de** `n` **octets libres dans le tube** : la fonction retourne immédiatement et il n'y a pas d'écriture (car l'atomicité ne peut pas être garantie). La valeur de retour est `-1` et `errno` vaut `EAGAIN`.



Un tube de communication offre donc une garantie d'**atomicité**, mais **uniquement au niveau des écritures** et tant que la taille à écrire ne dépasse pas `PIPE_BUF`.



`PIPE_BUF` est une constante du système qui est potentiellement différente de la capacité d'un tube. POSIX spécifie que `PIPE_BUF` est d'au moins 512 octets (4096 octets dans Linux). La taille d'un tube peut-être consultée ou modifiée par un appel à la fonction `fcntl` avec les opérations `F_GETPIPE_SZ` et `F_SETPIPE_SZ` respectivement. Dans les antédiluviennes versions de Linux, la capacité d'un tube était justement de 4096 octets (4ko, taille d'une page-mémoire ou d'un bloc-disque), elle est passée de 16ko puis à 1 Mo. Pour utiliser `PIPE_BUF` dans vos programmes, il sera peut-être nécessaire d'inclure le fichier `<limits.h>`.

6. C'est ce qui se passe quand vous voyez le message `broken pipe` dans un shell.

4.1.5 Pour aller plus loin : exécution/interaction de/avec une commande shell

La fonction `system` décrite en section 3.10 permet l'exécution d'une (suite de) commande(s) shell dans un programme mais ne permet pas vraiment de « dialoguer » avec le processus-enfant qui exécute ladite (suite de) commande(s). Or, il est parfois utile de récupérer des informations provenant de cette (série de) commmande(s) (par exemple, une partie de son affichage sur sa sortie standard). La fonction `popen` permet de récupérer ce type d'information. Par rapport à `system`, `popen` crée non seulement un processus-enfant pour exécuter la (série de) commande(s) mais également un *tube de communication* entre le parent et son enfant. Le processus-parent peut dès lors interagir avec la (série de) commande(s) shell via ce tube.

```
FILE *popen(const char *command, const char *type);
```

Paramètre 1 : Une chaîne de caractère contenant une ligne de commande shell.
Paramètre 2 : Une chaîne de caractères qui est soit `"r"`, soit `"w"` car le processus appelant est soit lecteur, soit écrivain, tandis que le processus-enfant jouera le rôle opposé. La chaîne spécifie le sens de circulation des informations dans le tube créé par `popen`.
Code de Retour : Un pointeur sur un objet `FILE` (un flux d'entrée/sortie). Une écriture sur ce flux permet d'écrire sur l'entrée standard de la commande `command`. Une lecture depuis ce flux permet de lire la sortie standard de la commande `command`. Ce flux devra être fermé avec un appel à `pclose` et non pas avec un appel à `fclose`.



Le tube créé est comme à l'accoutumée *unidirectionnel* mais le processus-parent ne peut être **que** lecteur **ou** écrivain.

```
int pclose(FILE *stream);
```

Paramètre 1 : Un flux d'entrée/sortie obtenu avec un appel à `popen`
Code de Retour : -1 si problème (cf. valeur d'`errno`) / statut de terminaison de la commande passée en argument à `popen` sinon.

La fonction `pclose` attend que le processus associé aux flux `stream` (et obtenu suite à un appel à `popen` se termine. `pclose` effectue une synchronisation entre le processus appelant `popen` (le processus-parent) et le processus exécutant la commande passée en argument à `popen` (le processus-enfant), de la même façon qu'un appel à `wait` ou `waitpid` le ferait.

4.1.6 Exercices sur les tubes

Exercice 37 ($\star$)
Écrivez un programme avec deux processus qui s'échangent une chaîne de caractères via un tube de communication *anonyme*.

Exercice 38 ($\star$)
Reprenez l'exercice précédent, mais remplacez le tube *anonyme* par un tube *nommé*.

Exercice 39 ($\star\star$)
Écrivez un programme qui effectue l'équivalent de la commande shell suivante :

```
cat toto.txt | grep "truc" | wc -l
```

Indications :
- il n'est **pas** demandé de réécrire les commandes `cat`, `grep` et `wc`.
- l'utilisation des fonctions `system` ou `popen` est interdit.

4.2 Mémoire partagée

Le deuxième mécanisme de communication inter-flot que nous allons détailler s'appuie sur la mémoire partagée. Le principe en est simple : une partie de la mémoire va être déclarée commune entre des processus différents. Ainsi, toute modification des données placées dans une telle zone sera automatiquement répercutée dans tous les processus qui peuvent accéder à cette zone dans leur espace d'adressage. Il existe plusieurs interfaces permettant de mettre en place un tel partage de mémoire [7].

4.2.1 Interface IPC System V

La première interface existante est celle des segments de mémoire partagée IPC System V. Cette interface est utilisable entre des processus quelconques, c'est-à-dire qui ne possèdent pas de relation d'affiliation particulière. Les segments de mémoire partagée sont des objets qui ont une existence et une durée de vie supérieures à celles du processus qui les a créés. Donc, si le processus créateur de cette ressource disparait, ce(s) segment(s) existe(nt) encore dans le système. D'autres processus sont donc susceptibles d'attacher ce(s) segment(s) dans leur propre espace d'adressage par la suite.

Je me contenterai juste de mentionner cette interface, qui n'est pas des plus pratique à utiliser : vous devez néanmoins être au courant de son existence [8] :
- `ftok` : création d'une clef d'identification d'une ressource System V (cette fonction sert également pour les sémaphores IPC Sys. V ainsi que pour les files de messages).
- `shmget` : création d'un segment de mémoire partagée
- `shmat` : attachement du segment dans l'espace d'adressage du processus
- `shmctl` : contrôle du segment (permet son effacement)

4.2.2 Projection de fichiers en mémoire

La seconde interface repose sur l'utilisation d'une fonction permettant de manipuler l'espace d'adressage d'un processus : `mmap`. Le but premier de cette fonction est de permettre de projeter dans l'espace d'adressage du processus appelant un fichier ou un périphérique [9]. Accéder au fichier directement depuis l'espace d'adressage du processus permet de se passer des appels-système (ou des fonctions de la libc) pour effectuer des lectures et des écritures.

Notion de page-mémoire

Avant d'examiner plus en détail le fonctionnement de `mmap`, il est nécessaire d'introduire le concept de **page-mémoire**. Les systèmes d'exploitation/processeurs actuels ne gèrent pas la mémoire octe t par octet (pour des questions de performances) mais par blocs appelés des pages-mémoire. La taille d'une telle page-mémoire peut être obtenue avec un appel à `sysconf(_SC_PAGE_SIZE)` (*a priori* 4ko). La conséquence est que la gestion de l'espace d'adressage d'un processus repose sur l'allocation/désallocation de ces blocs. Par exemple, quand vous effectuez un appel à `malloc` et

7. Ce partage de mémoire n'a aucun rapport avec des optimisations de gestion de la mémoire physique mises en place au sein du système d'exploitation, comme le *Copy-on-Write* par exemple.
8. Tout comme l'interface IPC System V des sémaphores.
9. Je vous rappelle que les périphériques sont accessibles via le SGF dans UNIX, ce sont donc des fichiers ...

que la taille demandée n'est pas un multiple de la taille d'une page-mémoire, le système allouera dans tous les cas une quantité de mémoire correspondant à un nombre entier de pages-mémoire [10].

Prototype de `mmap`

```
void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);
```

Paramètre 1 : L'adresse à laquelle sera effectuée la projection : mettre `NULL` pour laisser le système choisir (quasiment tout le temps pour vous). Cette adresse devrait préférablement être alignée sur une frontière de page-mémoire

Paramètre 2 : La taille de la projection à effectuer (idéalement cette taille devrait être un multiple de la taille d'une page-mémoire)

Paramètre 3 : Les permissions sur la projection. Il s'agit de `PROT_NONE` (aucun accès autorisé) ou d'un *bitwise OR* des valeurs suivantes :

- `PROT_EXEC` : les pages projetées peuvent être exécutées
- `PROT_READ` : les pages projetées peuvent être lues
- `PROT_WRITE` : les pages projetées peuvent être écrites

Paramètre 4 : Les propriétés de la projection. La première de ces propriété est **obligatoire** : il s'agit de la **visibilité de la projection**, qui peut être de type :

- **Privé** : ce type est positionné avec `MAP_PRIVATE`. Dans ce cas, les données lues/écrites dans la projection ne sont accessibles qu'au processus appelant. Ces données ne sont donc **ni partagées avec un autre processus, ni reportées dans le fichier qui a servi de support à cette projection**.
- **Partagé** : ce type est positionné avec `MAP_SHARED` et est à interpréter comme « le contraire de `MAP_PRIVATE` ». Cela signifie donc que la projection obtenue est potentiellement partageable avec d'autres processus et/ou que les données de la projection sont répercutables dans le fichier de support (cf. la fonction `msync` ci-dessous).

D'autres propriétés optionnelles sont disponibles (je vous invite à consulter le manuel pour en obtenir la liste) mais celle que vous allez le plus utiliser est `MAP_ANONYMOUS` [a], qui permet de faire une **projection anonyme** : dans ce cas les paramètres `fd` et `offset` sont ignorés (même s'il est recommandé de mettre une valeur de -1 pour `fd`).

Paramètre 5 : L'(index du) descripteur du fichier à projeter en mémoire.

Paramètre 6 : Le décalage depuis le début du fichier indiquant la partie du fichier à projeter en mémoire. Cette valeur doit être un multiple d'une taille de page-mémoire.

Code de Retour : `MAP_FAILED` (i.e -1) si erreur (cf. valeur d'`errno`) / adresse de la projection si succès. Cette adresse sera un multiple de la taille d'une page-mémoire et supérieure ou égale à l'adresse demandée (le paramètre `addr`).

a. Il vous faudra peut-être utiliser l'option de compilation `-D_GNU_SOURCE` pour compiler votre programme.

La fonction `mmap` permet donc de projeter à l'adresse `addr` une partie du fichier désigné par `fd`, commençant au décalage `offset` et sur une longueur `length`. La projection possède les droits d'accès spécifiés par `prot` et les propriétés désignées par `flags`.



En cas d'erreur, la fonction `mmap` ne retourne pas un pointeur `NULL` !

10. C'est ce qui explique que quand vous dépassez d'une zone mémoire allouée avec `malloc`, vous n'obtiendrez pas *systématiquement* une erreur d'accès mémoire (*segmentation fault*). Cf. le cours d'ETE8-IFON1 du S8.



Le fichier de support peut être fermé immédiatement après l'appel à `mmap`, sans aucune conséquente sur l'accessibilié de la projection.



Si la taille d'un fichier n'est pas un multiple de page-mémoire, alors le reste de la projection en mémoire est complété avec des 0. De même, si la taille de projection demandée est supérieure à la taille effective du fichier, la projection est complétée avec des 0 (mais dans la limite du multiple de taille de page-mémoire le plus proche). Toute modification de la zone contenant des 0 ne sera pas reportée dans le fichier (avec une synchronisation automatique ou via un appel à `msync`).

4.2.2.1 Retirer une projection

Pour retirer une projection de l'espace d'adressage du processus, il faut appeler la fonction suivante :

```
int munmap(void *addr, size_t length);
```

Paramètre 1 : L'adresse de la zone mémoire à retirer de l'espace d'adressage du processus appelant.
Paramètre 2 : La taille de la projection à retirer de l'espace d'adressage du processus appelant.
Code de Retour : -1 si problème (cf. valeur d'`errno`) / 0 si succès.

Un accès à une projection retirée suite à un appel à `munmap` entraînera un événement de type *segmentation fault* dans le processus.

4.2.2.2 Synchronisation entre le fichier de support et sa projection en mémoire

Dans le cas où la projection est de type `MAP_SHARED`, des synchronisations entre la projection et le fichier de support sont réalisées à intervalles réguliers. Il est cependant possible de realiser explicitement cette opération de synchronisation via un appel à `msync` :

```
int msync(void *addr, size_t len, int flags);
```

Paramètre 1 : Adresse du début de la zone à synchroniser ;
Paramètre 2 : Longeur de la zone à mettre à jour ;
Paramètre 3 : Type de synchronisation
- `MS_SYNC` : Écritures synchrones ;
- `MS_ASYNC` : Écritures asynchrones (retour immédiat de la fonction). No-op sous Linux.
- `MS_INVALIDATE` : Invalidation de copies dans les caches ;

On doit spécifier au moins `MS_SYNC` ou `MS_ASYNC` (mais pas les deux simultanément).
Code de Retour : -1 si problème (cf. valeur d'`errno`) / 0 sinon.

Lors du retrait d'une projection (appel à `munmap`), une synchronisation est effectuée. `msync` permet de garantir qu'une synchronisation sera bien effectuée avant un appel à `munmap`.

4.2.2.3 Projection de fichier et mémoire partagée

Jusqu'à présent, nous avons vu comment projeter un fichier en mémoire, mais quel est le rapport avec le partage de mémoire entre processus ? En fait pour obtenir un tel partage de mémoire, il suffit que les processus projettent **le même fichier** et que cette projection soit déclarée comme étant **partagée** avec l'option `MAP_SHARED` !



La projection en mémoire d'un fichier-support pour obtenir une zone de mémoire partagée est possible entre processus quelconques, sans relation ni affiliation particulière. Il est même ainsi possible de partager de la mémoire entre processus appartenant à des utilisateurs·trices différent·e·s.

En général, il est courant de créer un fichier dédié à cette projection dont le véritable rôle est de servir de support pour une zone de mémoire partagée entre processus. Pour rappel, il y a des restrictions concernant la taille de la zone et la taille du fichier qui sert de support : afin d'obtenir un fichier de la bonne taille, il est possible d'adapter sa taille avec la fonction `ftruncate` une fois le fichier ouvert avec `open` :

```
int ftruncate(int fd, off_t length);
```

Paramètre 1 : L'(index du) descripteur de fichier à retailler.
Paramètre 2 : La taille demandée pour le fichier.
Code de Retour : -1 si problème (cf. valeur d'`errno`) / 0 si succès.

Cette fonction ne modifie pas la position courante du fichier.

En ce qui concerne le fichier, il peut être utile de créer un **fichier temporaire** plutôt qu'un fichier classique (cf. Section 2.4.1). En effet, ce fichier n'a pour finalité que de jouer le rôle de support pour la projection mémoire.

Exercice 40 (⋆⋆)
Créez un programme avec deux processus qui se partagent de la mémoire pour s'échanger des données (une chaîne de caractères par exemple) en utilisant un fichier en support.

4.2.2.4 Mappings mémoire et héritage

Ainsi que vu en Section 3.5, toutes les manipulations de l'espace d'adressage avant un appel à `fork` sont héritées par les processus-enfants. La visibilité de la projection (i.e. `MAP_PRIVATE` ou `MAP_SHARED`) produit deux résultats très différents, comme le montre la Figure 4.1 :

Cas des mappings anonymes

Les processus **affiliés** (ex. processus-parent et processus-enfants) peuvent **se passer d'un fichier de support** pour créer une zone de mémoire partagée avec `mmap`.
En effet, il suffit au parent :

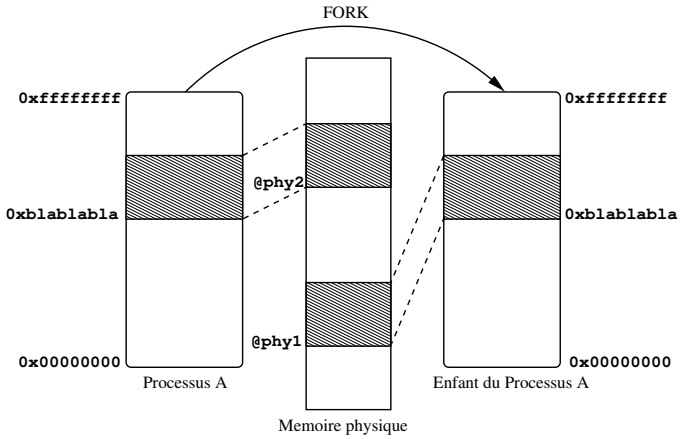
1. de créer une projection **anonyme** (avec l'option `MAP_ANONYMOUS`) et **partagée** (avec `MAP_SHARED`)
2. de **se dupliquer** ensuite avec un appel à `fork`

pour obtenir un processus-enfant avec la projection partagée (qui est héritée par l'enfant au moment de l'appel à `fork`). Il s'agit là d'un moyen plus simple de partager de la mémoire entre processus affiliés car le fichier de support n'étant plus nécessaire, il n'y a plus à se poser de questions quant à sa création ou sa taille.

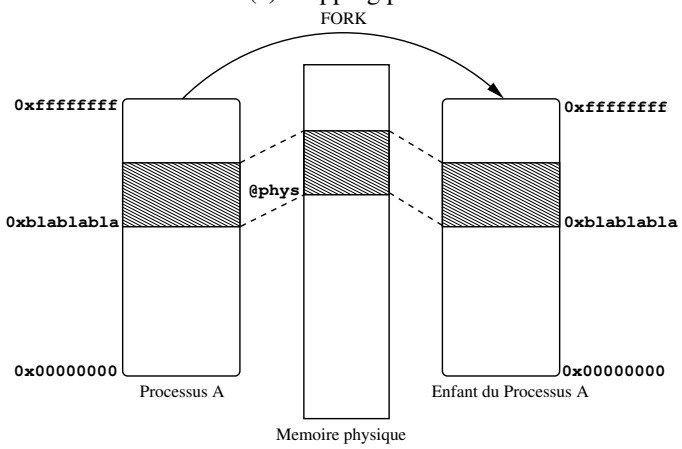
Exercice 41 (⋆)
Reprenez l'exercice précédent, mais créez des processus **affiliés** et utilisez une projection **anonyme** pour mettre en place la zone de mémoire partagée.

4.2.3 Interface POSIX

POSIX permet d'utiliser en support des fichiers spécifiques plutôt que des fichiers réguliers pour effectuer une projection en vue de créer des zones de mémoire partagée inter-processus. La



(a) Mapping privé



(b) Mapping partagé

FIGURE 4.1 – Différence entre mapping privé et partagé après un `fork`

fonction `shm_open` permet de créer de façon portable un fichier de support spécifique « mémoire partagée » qui sera utilisé par `mmap` pour créer la zone partagée. Ces fichiers remplaçant les fichiers réguliers de support, des processus quelconques peuvent les utiliser. La différence avec les fichiers réguliers réside dans leur coût d'utilisation, qui est plus important que dans le cas des fichiers spécifiques « mémoire partagée » (`shm`).



Pour utiliser les fonctions présentées dans cette section, vous devez compiler vos programmes avec l'option `-lrt`.

4.2.3.1 Création d'un fichier support spécifique `shm`

La fonction suivante permet de créer un tel fichier :

```
int shm_open(const char *name, int oflag, mode_t mode);
```

Paramètre 1 : Le nom du fichier : doit être de la forme `/nom` et d'une taille inférieure à `NAME_MAX`.

Paramètre 2 : Les droits d'accès du fichier (et donc de la zone de mémoire) : comprend au moins `O_RDONLY` ou `O_RDWR` et un *bitwise OR* de : `O_CREAT`, `O_EXCL`, `O_TRUNC`.

Paramètre 3 : Si `O_CREAT` est spécifié, ce paramètre permet de définir les droits de création du fichier.
Code de Retour : -1 si problème (cf. valeur d'`errno`) / (index de) descripteur de fichier si succès.

L'index de descripteur renvoyé est le plus petit disponible dans la table des descripteurs du processus. L'option `FD_CLOEXEC` est positionnée par défaut sur ce descripteur. Les restrictions sur la taille sont encore applicables pour ces objets « mémoire partagée » : il faut utiliser `ftruncate` pour en ajuster la taille. Une fois l'appel à `mmap` effectué, il est possible de fermer le descripteur, sans incidence sur la projection mémoire associée.



Tout comme pour les fichiers, l'option `O_EXCL` peut être utilisée en conjonction avec `O_CREAT` pour savoir si l'objet existe déjà (une erreur est renvoyée si c'est le cas). **Le test d'existence de la ressource et sa création sont effectués atomiquement**.

4.2.3.2 Élimination de l'objet support

Pour éliminer la ressource, la fonction suivante existe :

```
int shm_unlink(const char *name);
```

Paramètre : Le nom de l'objet à effacer.
Code de Retour : -1 si problème (cf. valeur d'`errno`) / 0 si succès

En fait cette fonction efface le nom de l'arborescence dans le SGF : la ressource ne sera effectivement eliminée du système que lorsque le nombre de lien sera nul et tous les processus qui utilisent la zone de mémoire partagée auront appelé la fonction `munmap` sur cette dernière. Les objets de type « mémoire partagée » ont donc, tout comme les segments System V, une pérénité supérieure à celle des processus qui les créent. Toutefois, la conservation en l'état d'un objet « mémoire partagée » après un redémarrage du système n'est pas spécifiée. Enfin, l'opération `msync` sur un tel objet n'a aucun effet.

4.2.4 Partage de données (variables)

Variables partagées

Dans le cas de mémoire partagée entre des processus, les variables déclarées dans une telle zone doivent être le plus souvent qualifiées par le mot-clef `volatile`. En fait, le mot-clef `volatile` est interprétable comme « modifié par un évènement extérieur au processus » [11].



Le mot-clef `extern` ne permet **jamais** de partager des variables entre processus différents. Il ne permet qu'une meilleure organisation du code-source dans le cas où ce dernier serait réparti dans plusieurs fichiers [a].

a. Le nombre de fichiers n'est jamais un indicateur de la nature d'un programme (i.e. mono- ou multi-processus)

Cas particulier des threads (cf. chapitre 7)

Les threads constituent un cas particulier car ils se partagent l'espace d'adressage du processus auquel ils apartiennent. Donc, dans le cas des threads, le partage de mémoire fait intrinsèquement partie du modèle, le plus souvent par le biais de variables **globales** (cf. Figure 4.2).

11. Cette façon de voir les choses peut même s'appliquer aux variables partagées entre traitants de signaux (cf. chapitre 6) et le `main` : le traitant (qui modifie la variable) est appelé lors de la prise en compte du signal, qui est bien un évènement extérieur au processus.

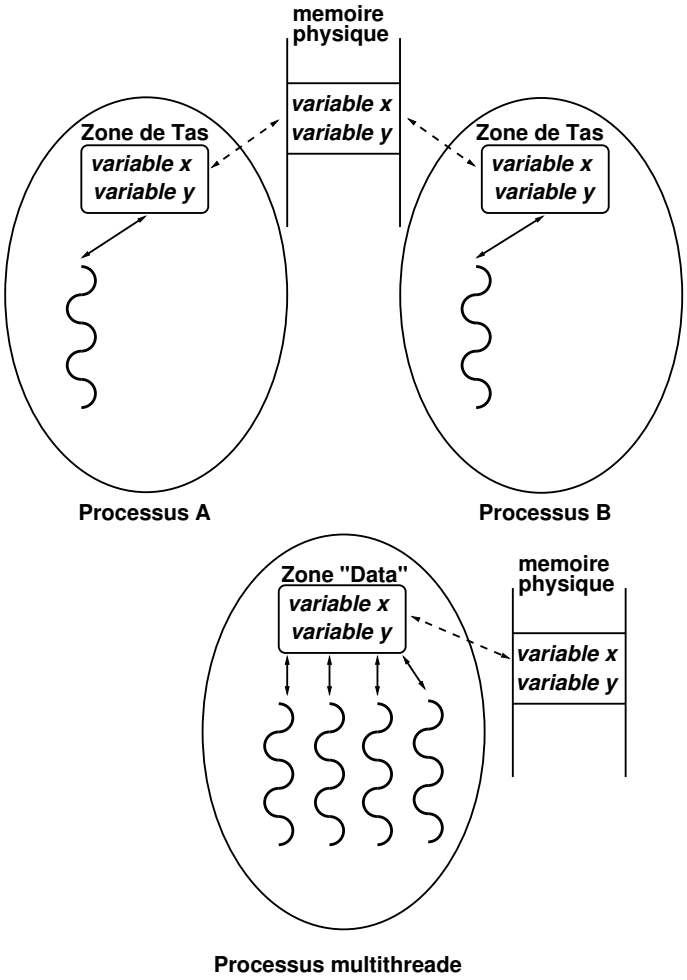


FIGURE 4.2 – Variables partagées : comparaison entre processus et threads

4.2.5 Exercices

Exercice 42 (⋆⋆⋆)
Écrivez un programme qui calcule le minimum d'un tableau d'entiers.
Ce tableau sera partagé par plusieurs processus et sera placé dans une zone de mémoire partagée.

4.3 Autre mécanismes de communication

4.3.1 Les sockets

Les sockets sont un mécanisme de communication assez large puisqu'il est possible par ce biais de faire communiquer des processus appartenant à la même machine où à des machines différentes. Les sockets ne sont pas couvertes dans ce document, même si quelques éléments les concernant se trouvent dans ce document. Quelques rappels sur les sockets :

— une socket est utilisable via un descripteur de fichier ;
— une socket est un mécanisme de communication *full-duplex* ;
— pour lire ou écrire dans une socket, vous pouvez utiliser les appels-système `read` ou `write` ;
— vous pouvez utiliser `poll` plutôt que `select` pour gérer les communications ;
— il est possible d'envoyer par le réseau autre chose que des chaînes de caractères, surtout en

environnement homogène [12] ...

Il existe plusieurs types (famille) de sockets, notamment :

— la famille `AF_INET`, qui permet une communication aussi bien locale qu'entre machines physiquement différentes. C'est ce type de sockets que vous utilise(re)z en Programmation Réseau ;

— la famille `AF_UNIX`, qui permet une communication locale (i.e. au sein de la même machine) uniquement.

Dans le cas de sockets de la famille `AF_UNIX`, il faut utiliser une structure de type `struct sockaddr_un` (un = UNIX) pour la création de la socket au lieu d'une structure de type `struct sockaddr_in` (in = Internet). Pratiquement, il convient d'initialiser à zéro l'ensemble de la structure avant de positionner *a minima* les champs suivants :

— `sun_family` : mettre la valeur `AF_UNIX` ;

— `sun_path` : copier dans ce champ le **nom du fichier** qui va servir de support pour la socket locale.

Enfin, il faut utiliser la macro `SUN_LEN` pour déterminer la taille de la structure dans un appel à `bind` (par exemple). Voici un exemple de code de création d'une socket locale :

```
1  int create_socket(void)
2  {
3    int desc;
4    struct sockaddr_un soc_in = {0};
5
6    soc_in.sun_family = AF_UNIX;
7    strcpy(soc_in.sun_path,"/tmp/ma_socket");
8
9    desc = socket (AF_UNIX, SOCK_STREAM, 0);
10
11   bind (desc, (struct sockaddr *)&soc_in, SUN_LEN(&soc_in));
12
13   return desc;
14 }
```

4.3.2 Les fichiers réguliers

Le premier moyen de communication utilisable entres processus s'exécutant au sein de la même machine repose sur les fichiers réguliers. En effet, il est simple d'imaginer un système de communication où un (des) processus écri(ven)t dans un fichier qui sera subséquemment lu par d'autres processus. La mise au point d'un tel système doit néanmoins faire l'objet d'une attention particulière au niveau des accès au(x) fichier(s), qui devront éventuellement être synchronisés.

4.3.3 Les signaux

Il est possible d'utiliser les signaux (cf. Chapitre 6) en tant que mécanisme de communication inter-processus, processus s'exécutant toujours sur la même machine.

12. D'un point de vue des CPUs.



5. Interlude non-déterministe

Il est temps de faire un petit bilan des notions que vous avez apprises jusqu'à présent. D'une part, vous savez désormais faire interagir un programme avec le système de fichiers de la machine sur laquelle il s'exécute. Vous savez également ce qu'est un processus et comment en créer de nouveaux et synchroniser leurs exécutions respectives.

Nous allons maintenant aborder des choses un peu plus compliquées. Tout d'abord, il est important de comprendre qu'en réalité, un programme n'est pas qu'une suite d'instructions qui sont exécutées séquentiellement par le processeur de la machine. La manière dont ces instructions sont organisées et exécutées est extrêmement importante et doit être prise en compte également.

En effet, il y a tout un ensemble de facteurs que vous ne contrôlez pas lors de l'exécution d'un processus et si le résultat du programme est déterministe, son exécution, elle, ne l'est pas. Par exemple, le nombre de processus présents dans le système à un instant donné peut modifier l'exécution. Dans un système préemptif, un processus peut être interrompu à n'importe quel moment et perdre le processeur au profit d'un autre processus. Bien entendu, il récupérera la main ultérieurement et finira son exécution comme prévu. Sauf que ces interruptions ne sont pas contrôlables et se produisent de façon totalement transparentes pour vous : vous n'avez pas conscience qu'elles se produisent et pourtant, c'est bien le cas. Donc, au lancement d'un programme, il est impossible de prévoir la séquence exacte de son exécution [1], d'où la notion de non-déterminisme.

Et avant même cette exécution, le compilateur lui-même peut prendre certaines libertés avec le code que vous avez écrit et est susceptible de changer l'ordre des instructions si cela lui permet d'optimiser le code produit (pour qu'il soit plus rapide par exemple). Prenez par exemple cette série d'instructions :

```
1 int a;
```

1. Et je mets de côté le fait que le processeur peut de lui-même réorganiser l'exécution des instructions (*out of order execution*), sans rien vous dire, bien évidemment ...

```
2 int b;
3
4 a = 2;
5 b = 4;
```

En supposant que ces deux variables sont indépendantes l'une de l'autre, alors rien n'interdit de modifier le code de la façon suivante :

```
1 int a;
2 int b;
3
4 b = 4;
5 a = 2;
```

Ce genre d'acrobaties n'est pas inhabituel, mais peut poser parfois des problèmes inattendus. Nous verrons quand il le faudra comment s'en prémunir. Donc, non seulement, vous ne pouvez pas contrôler l'exécution des programmes que vous allez écrire (i.e la préemption de l'ordonnanceur du système d'exploitation), mais en plus, le code qui s'exécute peut ne pas complètement correspondre à ce que vous avez initialement écrit. Et si vous pensez qu'en positionnant de bonnes options à la compilation, le problème sera résolu : mauvaise pioche. En effet, quelle que soit la compilation du programme, le processeur pourra lui aussi procéder à une exécution des instructions selon ce qu'il considère comme le plus efficace (processeurs *out-of-order*)...
Le but n'est pas ici de vous faire peur, mais de vous faire comprendre cette notion de non-déterminisme dans l'exécution, avec laquelle il va vous falloir composer désormais. Surtout que nous allons maintenant commencer la deuxième partie de notre voyage, où le non-détermninisme sera encore plus grand et surtout, provoqué par vos soins ! Cependant, retenez bien la chose suivante : s'il existe un ordonnancement pour lequel votre programme ne fonctionne pas, alors c'est que votre programme est faux !



Un programme juste doit avoir un fonctionnement correct et produire le résultat attendu quels que soient son ordonnancement et son exécution.



6. Notification d'événements : les signaux

6.1 Gestion des événements par interruption

Étant de jeunes personnes curieuses et taraudées par une inextinguible soif de connaissance [1], vous vous êtes déjà naturellement posé la question (fort pertinente au demeurant) du fonctionnement de votre souris ou de votre clavier : comment le système peut-il prendre en compte les mouvements ou les touches frappées alors qu'un processus est en train de s'exécuter et occupe le(s) processeur(s) de la machine ?

Afin de traiter ces événements (i.e. mouvements et frappes de touches), c'est-à-dire les prendre en compte, le processeur doit manifestement arrêter son activité courante pour s'occuper de gérer l'événement en question. Il n'est pas possible d'attendre tranquillement la fin de l'exécution de la tâche en cours, parce que sa durée est inconnue. Le système va alors lever ce qu'on appelle une **interruption** : le processeur va donc être **interrompu** dans sa tâche courante par un signal électrique émanant du périphérique qui a généré l'événement à traiter. Le processeur va alors exécuter le code associé à la gestion de cet événement, qui s'appelle un **traitant d'interruption**. Le nombre de ces traitants est fixe et à chaque événement matériel est associé un tel traitant.

Le point important est donc que le processeur peut être interrompu à n'importe quel moment et **changer de contexte** [2] pour « sauter » sur le code du traitant et l'exécuter. Une fois le traitant terminé, le processus reprend son exécution au point interruption. Tout se passe comme si le processus exécutait une fonction en réponse à l'arrivée d'un événement, tout en étant incapable de contrôler que cet événement va survenir.

Les interruptions sont fondamentales dans les systèmes d'exploitation, notamment pour mettre en place des systèmes préemptifs et multitâches.

6.2 Principes de fonctionnement des signaux

Quel est le rapport entre les interruptions et les signaux ? Sur le fond, le fonctionnement est similaire, sauf qu'**un signal va servir à notifier un événement logiciel** (et non pas matériel) à un

1. Si, si, je le sais, ne faites pas les modestes.
2. C'est un changement de contexte intra-processus.

processus (et pas au processeur). Cet événement est logiciel dans la mesure où il est déclenché par un autre flot d'exécution. Les signaux offrent donc un nouveau moyen de communication inter-flots après les fichiers (cf. Chapitre 4) Cependant, ainsi que nous allons le voir, cette communication va rester relativement basique, avec quelques limitations.

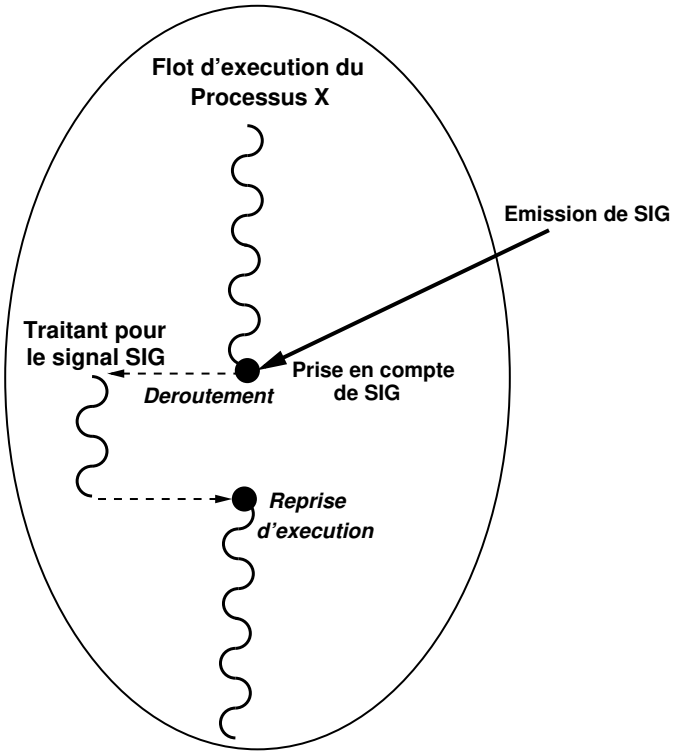


FIGURE 6.1 – Déroutement d'un processus vers un traitant de signal

De la même façon que la palette d'interruptions est limitée, le nombre d'événements pour lesquels il existe des signaux est réduit et dépend du système sur lequel vous travaillez. Pour connaître la liste complète des signaux disponibles, vous pouvez utiliser la commande `kill` (dans un terminal) :

```
kill -l
```

Chaque signal est identifié par son numéro, pour lequel une constante est définie.

En pratique, il est recommandé d'utiliser les **noms des signaux plutôt que leurs numéros**, dans un souci de lisibilité et de portabilité. Certains systèmes implémentent des signaux *temps-réel*, dont le numéro est compris entre `SIGRTMIN` et `SIGRTMAX`, et dont le nom débute par `SIGRT` (cf. Section 6.6).



Un signal ne véhicule aucune information (message, données, etc.) à part son numéro. Il ne sert qu'à indiquer à un processus qu'un certain type d'événement s'est produit.

L'utilisation des signaux dans un programme est assez simple conceptuellement parlant : vous allez en fait écrire et mettre en place des **traitants de signaux** (*signal handlers*) qui vont constituer en quelque sorte la réaction du processus notifié de cet ou des événements qui se sont produits. Par

défaut, un processus possède un traitant pour chaque signal, mais ce traitant peut éventuellement être une fonction vide et donc le processus ne fait rien quand l'événement survient.

Donc, le comportement (i.e la réaction) du processus vis-à-vis du signal dépend de ce dernier. S'il est possible de redéfinir ce comportement, cela n'est pas possible pour tous les signaux, en particulier pour certains qui sont utilisés par l'ordonnanceur du système pour la gestion des processus : `SIGKILL` et `SIGSTOP`. Donc, ces deux signaux ne peuvent pas non plus être bloqués/masqués ou ignorés.



Un processus ne revient pas dans le code (en particulier le `main`) duquel il a été interrompu tant que l'exécution du traitant n'est pas allée jusqu'à son terme.

Exemples de signaux

Par exemple, le signal numéro 11 est `SIGSEGV` : c'est le signal qu'un processus reçoit lorsqu'il déclenche un *segmentation fault*[3]. Un processus qui reçoit le signal `SIGSEGV` se termine immédiatement, mais il serait envisageable de faire autre chose[4].

Autre exemple, lorsqu'un processus se termine, il envoie systématiquement le signal `SIGCHLD` à son parent, qui, par défaut, ignore (ne fait rien quand il reçoit) ce signal (cf. 3.7). Notez que ce signal est utilisé plus généralement pour signaler qu'un événement s'est produit au niveau d'un processus-enfant. C'est donc le cas d'une terminaison, mais aussi le cas d'un arrêt (avec une réception du signal `SIGSTOP` par le processus-enfant) ou d'un redémarrage (avec une réception du signal `SIGCONT` par l'enfant) (cf. 3.7).

Dans le cas d'une écriture dans un tube ou une socket ne possédant pas de lecteur c'est le signal `SIGPIPE` qui est envoyé au processus-écrivain. Le comportement par défaut lors de la réception de ce signal est la terminaison, mais il est possible de le masquer ou l'ignorer. Outre l'émission de ce signal, `errno` prend pour valeur `EPIPE` lors d'une écriture dans une socket ou un tube sans lecteur.

Enfin, il existe deux signaux qui ne sont pas rattachés à des évènements particuliers : ce sont `SIGUSR1` et `SIGUSR2`. Ces deux signaux peuvent donc vous servir dans vos programmes pour la notification d'événements quelconques. Il est quelquefois aussi possible de « détourner » l'utilisation d'un signal particulier pour notifier un événement différent de celui pour lequel il existe à la base.

6.2.1 États d'un signal

Un signal est susceptible d'être dans l'un des états suivants :

— **Émis** : un signal est considéré comme émis quand un processus/thread a notifié la survenue d'un évènement à l'aide de la fonction `kill` (ou d'un mécanisme équivalent[5]) à un autre processus/thread ;

— **Délivré** (ou pris en compte) : un signal est dit délivré quand le traitant associé (rappel : cela peut être une fonction vide), a été exécuté ;

— **Pendant** (*pending*) : un signal est dit pendant quand il a été émis mais qu'il n'a pas encore été pris en compte par le processus destinataire ;

— **Masqué** (ou bloqué) : un signal masqué est un signal pour lequel le processus retarde volontairement et temporairement[6] sa délivrance.

Notez que **ces états ne sont pas mutuellement exclusifs** : un signal peut être, sous certaines conditions, être à la fois masqué **et** pendant (cf. la fonction 6.5), En revanche, dès lors qu'un signal

3. Accès à une adresse non autorisée dans l'espace d'adressage du processus.
4. C'est ce que font les debuggers par exemple, pour vous permettre d'examiner ce qui s'est passé dans le programme.
5. Typiquement, un thread noyau n'appellera pas cette fonction telle quelle mais fera un travail similaire.
6. Temporairement, ça peut aussi vouloir dire jusqu'à la fin du programme parfois ...

est délivré, il n'est ni pendant, ni masqué.

6.2.2 Délivrance d'un signal

D'après les définitions qui précédent, un signal est délivré quand le processus destinataire exécute le traitant associé. Mais il se peut qu'il y ait un délai entre l'émission effective du signal par le processus source et la prise en compte de ce même signal par le processus destinataire. **C'est durant cet intervalle que le signal va être dans l'état pendant**. D'où provient l'existence d'un tel intervalle ? Tout simplement à cause de ce délai entre l'émission d'un signal et sa prise en compte et surtout parce qu'un signal ne peut être délivré qu'à un moment précis durant le cycle de vie du processus.



Un signal ne peut être délivré à un processus que lors d'une **transition de l'état actif kernel vers l'état actif user** [a].

a. Et surtout pas l'inverse !

Pourquoi une telle restriction ? Déjà, c'est une différence notable par rapport aux traitants d'interruptions, car ces derniers font basculer le processus dans l'état actif kernel si jamais l'interruption est reçue pendant qu'il est dans l'état actif user. Cette différence provient de l'origine du traitant : un traitant d'interruption fait partie intégrante du noyau du système d'exploitation tandis qu'**un traitant de signal est un code écrit par l'utilisateur·trice dans la majorité des cas**. Le système n'a donc aucun moyen de contrôler ou d'encadrer ce qui se passe dans ce code. Or, un processus en mode actif kernel possède des privilèges particuliers (le processeur est lui-même en mode privilégié). Il serait donc très dangereux de laisser un code quelconque s'exécuter avec de tels privilèges (cela constituerait une importante brèche de sécurité dans le système). Mais la gestion des signaux impose aussi de devoir manipuler des structures de données qui se trouvent elles-mêmes dans le noyau. Il est donc naturel que la délivrance [7] se produise une fois que ces structures ont été modifiées (en mode noyau) avant de pouvoir appeler le traitant de signal approprié (en mode utilisateur·trice).

Enfin, pour rappel, vous n'exercez aucun contrôle sur la délivrance des signaux car c'est le noyau qui se charge de tout. La seule chose que vous pouvez faire, c'est éventuellement retarder cette délivrance en masquant (si c'est possible) le signal (cf. Section 6.5).



La délivrance des signaux ne se fait pas forcément dans l'ordre où ces derniers ont été émis !

6.2.3 Structures de données mises en œuvre dans le noyau

Examinons maintenant les structures de données utilisées dans le noyau pour la gestion des signaux classiques (le cas des signaux temps-réels est différent). Cet examen va permettre de déduire des limites importantes sur le fonctionnement des signaux. La gestion des signaux est **locale**, c'est-à-dire propre à chaque processus du système. La redéfinition du comportement d'un processus vis-à-vis d'un signal n'a donc aucune espèce d'incidence sur les autres processus du système, sauf dans le cas d'un `fork` où un processus-enfant va hériter du comportement de son parent (cf. Section 6.6).

En supposant que le système dispose de `NSIG` signaux différents (la valeur de `NSIG` importe peu), les structures de données implémentées sont les suivantes :

— un vecteur (tableau) de `NSIG` bits pour gérer les signaux pendants.

7. Connaissez-vous le Texzas ?

— un vecteur (tableau) de `NSIG` bits pour gérer les signaux masqués.
— un vecteur (tableau) de `NSIG` pointeurs de fonctions pour les traitants associés à chaque type de signal [8].

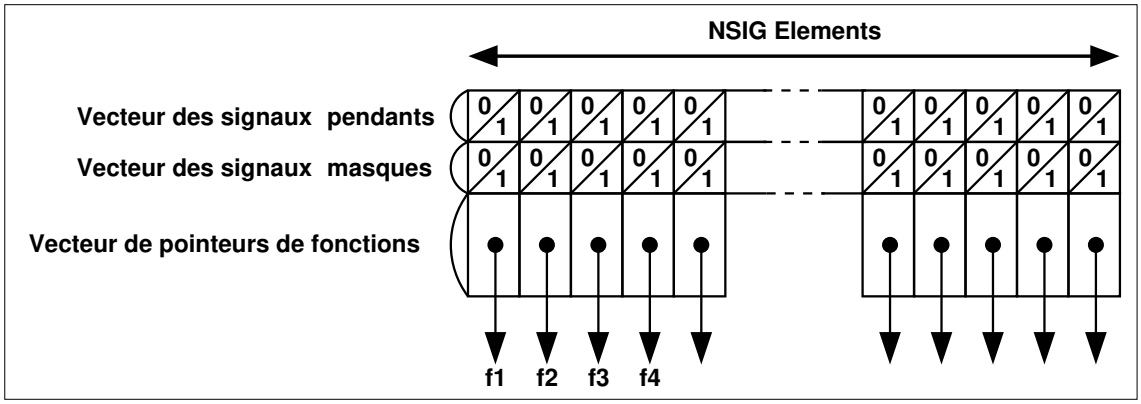


FIGURE 6.2 – Structures de données pour la gestion des signaux classiques.

Comment ces vecteurs de bits sont-ils utilisés ? Prenons le cas d'un signal fictif, `SIGBIDULE` par exemple, et supposons que le processus a mis en place un traitant (une fonction) associé à ce signal. Au début de l'exécution du processus, le bit de signal pendant correspondant à `SIGBIDULE` est à mis à 0. Si jamais le processus reçoit un signal (de type `SIGBIDULE`), mais ne le prend pas encore en compte [9], alors ce bit est mis à 1. **Le signal `SIGBIDULE` passe donc dans l'état pendant et le restera jusqu'à sa prise en compte par le processus**. Lorsque cette prise en compte est effective, alors ce bit repasse à 0. Pour le bit de signal masqué, il est mis à 0 par défaut au début de l'exécution du processus et est modifié si ce dernier demande la mise en place d'un masque pour le signal `SIGBIDULE` (cf. Section 6.5). Le bit de signal masqué est alors positionné à 1 jusqu'à une nouvelle demande de modification par le processus. Dans le cas d'un signal masqué, le bit de signal pendant n'est pas remis à 0 si le signal est reçu mais non pris en compte par le processus destinaire (i.e. le traitant associé n'est pas exécuté). Dès que ce signal est débloqué, il peut de nouveau être pris en compte (si jamais il était pendant).

Exercice 43 (⋆⋆)
À partir de l'examen des structures utilisées pour l'implémentation des signaux, combien d'occurences de signal d'un type donné peuvent être dans l'état *pendant* dans un processus ?



Attention aux rafales de signaux : les signaux implémentent un mécanisme de communication **non-fiable**. Si un signal est reçu deux fois pendant que le processus est actif kernel, alors le traitant ne sera exécuté qu'une seule fois quand le processus repasse dans l'état actif user.

8. Un même traitant peut éventuellement être mis en place pour des signaux de types différents.
9. Où plutôt ne le peut pas parce qu'il est dans l'état actif noyau par exemple.

6.3 Émission et réception d'un signal

Dans cette section, nous allons voir l'interface qui permet d'envoyer et de recevoir (prendre en compte) un signal.

6.3.1 Fonctions d'émission

L'émission d'un signal par un processus en direction d'un autre processus [10] est toujours **explicite**. La fonction/commande employée principalement s'appelle `kill` et peut s'utiliser de deux façons : soit depuis un terminal en tapant une commande shell, soit depuis un processus, s'il exécute une fonction.

1. Émission depuis un terminal :

```
kill -numéro de signal pid
```

2. Émission depuis un processus :

```
int kill(pid_t pid, int signum);
```

Paramètre 1 : Le PID du processus destinataire du signal.
Paramètre 2 : Le numéro du signal à émettre. Si `signum` est égal à 0, alors un signal n'est pas vraiment envoyé : cela constitue un test d'existence du processus de PID `pid`.
Code de Retour : -1 si problème (consulter la valeur d'`errno`) / 0 si succès.



Vous remarquerez que l'ordre des arguments est inversé entre la commande shell et la fonction C !

Il est possible pour un processus de s'envoyer un signal **à lui-même** avec la fonction `raise` :

```
int raise(int signum);
```

Paramètre : Numéro du signal à émettre.
Code de Retour : 0 si succès / Valeur différente de 0 en cas de problème.



Un appel à `raise(sig)` est strictement équivalent à `kill(getpid(),sig)` dans le cas d'un processus avec un unique flot d'exécution, et est équivalent à `pthread_kill(pthread_self(),sig)` dans le cas d'un processus multiflots (cf. Chapitre 7).

Exercice 44 (⋆)
Écrivez un programme qui affiche son pid puis qui effectue une boucle infinie. Depuis un terminal (autre que celui qui a servi à lancer le programme initial), envoyez un signal de type `SIGKILL` ou `SIGTERM` à ce processus.

10. qui peut être lui-même.

Exercice 45 ($\star$)
Reprenez l'exercice précédent, mais l'émission du signal doit maintenant se faire par un processus et non avec une commande shell tapée dans un terminal.

Il existe également une fonction qui permet d'envoyer des données en même temps que le signal :

```
int sigqueue(pid_t pid, int sig, const union sigval value);
```

Paramètre 1 : Le PID du processus auquel le signal est destiné. À la différence de `kill`, seules des valeurs positives sont acceptées.
Paramètre 2 : Le numéro du signal à émettre.
Paramètre 3 : Des données à émettre, sous la forme d'une structure de type `union sigval` :

```
union sigval {
int   sival_int;
void *sival_ptr;
};
```

Il est donc possible d'envoyer un entier ou un pointeur[a].
Code de Retour : -1 si problème (cf. valeur d'`errno`) / 0 en cas de succès

a. Un pointeur n'a pas de sens en dehors d'un processus, mais un processus peut très bien s'envoyer un signal à lui-même (ex. un timer), auquel cas le pointeur peut avoir du sens.

Pour récupérer ces données, il faut utiliser l'option `SA_SIGINFO` pour autoriser un prototype étendu de traitant de signal et récupérer le champ `si_value` du pointeur `info` de type `siginfo_t` (cf. Section 6.4.4).

Pour aller plus loin : émission d'un signal avec l'interface `pidfd` (Non POSIX, spécifique à Linux)

Comme nous l'avons vu en Section 3.2.2, une interface basée non pas sur les PID mais sur des (index de) descripteurs de fichier est également utilisable. Il existe donc une fonction permettant d'envoyer un signal à un processus identifié par ce biais :

```
int pidfd_send_signal(int pidfd, int sig, siginfo_t *info, unsigned int flags);
```

Paramètres 1 : le descripteur de fichier associé au processus-cible (obtenu avec `pidfd_open` par exemple).
Paramètres 2 : le numéro de signal que le processus appelant veut émettre au processus-cible.
Paramètres 3 : Un pointeur (potentiellement `NULL`) sur une structure d'information (cf. Section 6.4.4).
Paramètres 4 : De possibles options (paramètre non utilisé pour le moment).
Code de Retour : -1 si problème (cf. valeur `errno`) / 0 sinon.



Cette fonction n'est pas utilisable directement, il faut utiliser la fonction `syscall` car `pidfd_send_signal` ne possède pas de *wrapper* dans la libc (cf. Section 3.2.2).

6.3.2 Réception

Réception implicite

La réception d'un signal est en général un peu plus déstabilisante pour vous car par défaut, **il n'y a rien besoin de faire pour recevoir un signal**. En effet, la réception d'un signal est **implicite** dans la majorité des cas. Une fois le signal émis, il *sera* pris en compte par le processus destinataire, sauf si ce dernier a décidé de le est masquer bien entendu (cf. Section 6.5). En résumé, **il n'y a pas de fonction particulière à appeler pour recevoir un signal.**

Réception explicite

Cependant, il peut arriver qu'un processus n'ait rien d'autre à faire à un instant donné que de devoir attendre **explicitement** l'arrivée d'un signal. Dans ce cas, il est possible d'utiliser une fonction pour ce faire. Plusieurs fonctions sont même disponibles et la première est la fonction `pause` :

```
int pause(void);
```

Paramètre : Aucun
Code de Retour : -1 et `errno == EINTR` (une erreur est le comportement « normal » de `pause`).

Cette fonction met en sommeil le flot d'exécution appelant (processus ou thread, cf. le chapitre 7) jusqu'à la prise en compte d'un signal qui termine le processus ou bien qui déclenche l'exécution d'un traitant de niveau utilisateur. Dans ce cas, la fonction `pause` retourne **après que** le traitant a été exécuté (correctement). Cependant, la fonction `pause` ne permet pas de choisir le ou les signaux à attendre.

Il existe également une autre fonction d'attente explicite, `sigsuspend`, qui permet à la fois (atomiquement) de mettre en place un masque de signaux, d'attendre un signal et de remettre en place le masque de signaux d'origine suite à la réception du signal (cf. Section 6.5).



Il est totalement inutile de truffer vos programmes d'appels à `pause` ou `sigsuspend` pour recevoir des signaux car ces derniers sont reçus de toute façon par le processus [a] !

a. S'ils ne sont pas bloqués, bien entendu . . .

6.4 Traitants de signaux

Le cœur de la programmation avec des signaux repose sur la (re)définition du comportement du processus vis-à-vis d'un signal donné, c'est-à-dire la mise en place d'un traitant de signal, pour réagir à la survenue d'un événement particulier notifié (via le signal) au processus.

6.4.1 Mise en place

Pour rappel, un traitant de signal est une fonction qui est exécutée quand le signal correspondant est pris en compte par le processus. C'est cette fonction qui interrompt le cours du processus qui reprend normalement son exécution dès qu'il ressort de ce traitant.



N'oubliez pas que les signaux sont non fiables : un traitant peut tout-à-fait être exécuté **une unique fois** alors que de **multiples** occurences d'un signal ont été émises ! Revoyez le point concernant les rafales de signaux (6.2.3) si nécessaire.

La mise en place d'un traitant se fait en trois étapes :

Étape 1 Déclaration du traitant pour le signal concerné : lors de cette étape, vous devez écrire la fonction qui sera exécutée. Il y a des restrictions concernant l'écriture d'une telle fonction, aussi bien au niveau de son prototype que de son corps (cf. Section 6.4.2). *Grosso modo*, il s'agit de déclarer et d'écrire une fonction comme vous le feriez habituellement dans un programme C.

Étape 2 Déclaration et initialisation d'une **variable** de type `struct sigaction`[11]. Il est indispensable de passer par une telle structure pour transmettre au système les informations concernant les modifications de comportement du processus vis-à-vis du signal considéré. Cette structure de type `sigaction` contient (notamment) les champs suivants :

```
struct sigaction {
void     (*sa_handler)(int);
void     (*sa_sigaction)(int, siginfo_t *, void *);
sigset_t   sa_mask;
int        sa_flags;
};
```

— Le champ `sa_handler` est un pointeur de fonction qui sera initialisé avec le nom[12] du traitant à mettre en place.
— Le champ `sa_sigaction` est un pointeur de fonction alternatif, qui peut être utilisé en lieu et place de `sa_handler` afin d'avoir plus d'informations sur le signal lors de sa prise en compte (cf. Section 6.4.4 ci-dessous).
— Le champ `sa_mask` sert à positionner un masque de signaux qui seront automatiquement bloqués lors de l'exécution du traitant, **en plus du signal pour lequel le traitant est mis en place, qui lui est automatiquement bloqué** (cf. Section 6.5 pour la manipulation de ce champ).
— Le champ `sa_flags` sert à positionner des options de mise en place du traitant.

Il est recommandé de **mettre à zéro l'ensemble de cette structure** avant son utilisation[13].



Les champs `sa_handler` et `sa_sigaction` sont mutuellement exclusifs et seul l'un des deux doit être initialisé avec un pointeur de fonction valide.



Sur certains systèmes, la structure `sigaction` n'est pas toujours connue, il faut alors compiler les programmes avec l'option `-D_POSIX_C_SOURCE`.

Étape 3 Appel à la **fonction** `sigaction` pour la mise en place effective du traitant. La déclaration du traitant et de la **structure** `sigaction` initialisée ne sont pas suffisantes pour une mise en place de traitant, l'appel à la **fonction** `sigaction` est **obligatoire**.

```
int sigaction(int signum, const struct sigaction *act,
.                         struct sigaction *oldact);
```

11. Notez bien que je parle d'une variable et pas du type lui-même qui n'a pas besoin d'être redéclaré dans vos programmes, il est déjà dans un .h quelconque.
12. Rappel : en C, **un nom de fonction est équivalent à un pointeur** sur cette fonction ...
13. Avec une fonction de type `memset` ou `bzero` par exemple.

Paramètre 1 : Numéro de signal pour lequel le processus appelant souhaite mettre en place un traitant (ne peut être ni `SIGKILL` ni `SIGSTOP`).
Paramètre 2 : Pointeur sur une structure de type `sigaction`. Si ce pointeur n'est pas `NULL`, alors la structure pointée est utilisée pour redéfinir le comportement du processus vis-à-vis du signal `signum`.
Paramètre 3 : Pointeur sur une structure de type `sigaction` pour sauvegarder (et éventuellement restaurer) l'ancien comportement du processus. Peut être mis à `NULL` si cette sauvegarde n'est pas désirée.
Code de Retour : -1 si problème (consulter la valeur d'`errno`) / 0 si succès

En cas de succès de l'appel à la fonction `sigaction`, le traitant est mis en place et peut être exécuté dès qu'un signal est pris en compte par le processus. Si le pointeur `act` passé en deuxième argument à la fonction n'est pas `NULL`, alors à la réception du signal `signum`, un appel à :

```
act->sa_handler(signum);
```

sera effectué. Un tel traitant est installé jusqu'à une modification du comportement du processus, c'est-à-dire un nouvel appel à `sigaction`. Notez qu'il est possible de mettre en place **un même traitant pour des signaux différents**.



La mise en place d'un traitant ne provoque ni émission ou réception de signal.



La fonction `sigaction` **ne sert pas à recevoir** un signal.



Le comportement du processus vis-à-vis d'un signal est modifié jusqu'à une nouvelle demande de changement.

Exercice 46 ($\star$)
Écrivez un programme qui redéfinit son comportement vis-à-vis du signal `SIGUSR1` : à chaque réception d'un tel signal, le processus devra afficher une chaîne de caractères indiquant qu'il a bien reçu ce signal. Vous émettrez ces signaux de type `SIGUSR1` depuis un terminal.

Exercice 47 ($\star\star$)
Reprenez l'exercice précédent, mais au lieu d'envoyer à ces signaux via une commande shell depuis un terminal, faites-le depuis un second programme C.
— Que constatez-vous ?
— Ce comportement est-il cohérent avec ce que vous connaissez du fonctionnement des signaux ?

Exercice 48 ($\star\star\star$)
Pourquoi un signal est-il **automatiquement bloqué** lors de l'exécution du traitant mis en place par un processus pour réagir à la réception de ce même signal ?

Cas particuliers

Il existe deux traitants particuliers :

— `SIG_DFL` est le traitant correspondant **au comportement par défaut** du processus vis-à-vis d'un signal. Ce comportement peut être d'ignorer le signal (cas du signal `SIGCHLD` par exemple).

— `SIG_IGN` est le traitant qui permet d'**ignorer un signal**. Notez bien que ce comportement est différent d'un blocage de signal : dans le cas d'un signal ignoré, le bit de signal pendant est remis à zéro tandis qu'il reste à 1 dans le cas d'un signal bloqué.



Le cas du signal `SIGCHLD` est particulier car le comportement par défaut (`SIG_DFL`) est d'ignorer le signal. Cependant, ce comportement diffère de `SIG_IGN` (qui est d'ignorer aussi le signal). Si `SIG_IGN` est positionné pour `SIGCHLD`, alors les enfants terminant leur exécution sont **immédiatement** éliminés du système et **ne passent pas dans l'état zombie** (dans le cas où `SIG_DFL` est positionné pour `SIGCHLD`, les enfants sont maintenus dans l'état zombie et un appel à `wait`/`waitpid` permet d'éliminer ces enfants zombies ainsi que de récupérer des informations sur leur état d'éxécution (cf. Section 3.6)). Un comportement similaire peut être mis en place quand l'option `SA_NOCLDWAIT` est utilisée pour la mise en place d'un traitant de signal pour `SIGCHLD` avec `sigaction` (y compris `SIG_DFL`).

6.4.2 Règles d'écriture d'un traitant de signal

Examinons maintenant les règles qui régissent l'écriture d'un traitant de signal. Tout d'abord, un traitant doit posséder un prototype correspondant au format suivant :

```
void nom_du_traitant(int signum);
```

C'est-à-dire que le traitant **ne renvoie pas de résultat** et prend comme **unique argument le numéro du signal** pour lequel il a été mis en place. Cela peut paraître bizarre, mais c'est assez logique : quand le processus est interrompu et que le traitant de signal est exécuté, le processus se retrouve dans une fonction *qu'il n'a pas formellement appelée*. Le numéro de signal passé en argument permet donc au processus de savoir dans quel traitant il se trouve, si jamais plusieurs ont été mis en place. Par ailleurs, si un même traitant a été mis en place pour plusieurs signaux, cela permet au processus de savoir quel signal en particulier a provoqué l'exécution du traitant.

Ensuite, il est préférable que l'écriture d'un traitant respecte les principes suivants :

Principe 1 : Éviter les appels bloquants

Effectuer un appel bloquant dans un traitant de signal est rarement une bonne idée car cela peut provoquer le blocage du processus tout entier. En effet, **tant qu'un processus se trouve dans un traitant, il ne peut pas être dans le `main`.** Donc si un processus est mis en sommeil dans un traitant, il ne peut revenir dans le `main` et peut rester par conséquent indéfiniment dans cet état de sommeil. Il n'est pas interdit de faire des appels bloquants dans un traitant, mais lorsque cela est le cas, il faudra être vigilant·e et en jauger les conséquences.

Principe 2 : Utiliser des fonctions *async-signal-safe*

Plus généralement, il faut être attentif·ve aux fonctions appelées dans un traitant de signal : il existe des fonctions dites ***async-signal-safe***, qui peuvent être appelées sans se poser de questions dans un traitant. Si une fonction n'est pas *async-signal-safe*, cela ne veut pas dire qu'elle ne peut pas être utilisée dans un traitant, mais dans ce cas, il faut bien analyser la situation pour déterminer

si cet appel peut poser un problème. La liste de ces fonctions est disponible dans le manuel (`man -s7 signal` ou `man -s7 signal-safety`).



`printf` ne fait pas partie de la liste des fonctions *async-signal-safe* ! En revanche, `write` en fait partie . . .



`malloc` et `free` ne sont pas *async-signal-safe* !

Principe 3 : Utiliser des variables correctement typées et qualifiées

Les **variables partagées entre le(s) traitant(s) et le `main` du processus doivent être globales, être qualifiées de `volatile` et de type `sig_atomic_t`** [14].

— Premièrement, ces variables sont forcément des **variables globales**, puisque le prototype d'un traitant ne lui permet pas d'avoir des arguments autres que le numéro de signal (dans sa version de base).

— Deuxièmement, l'aspect « volatile » des variables est essentiel, car les optimisations du compilateur concernant ces variables *doivent* être désactivées. En effet, si jamais ces variables sont modifiées uniquement dans le traitant, il est possible que le compilateur les change en constantes, car elles se trouvent dans une fonction (le traitant) qui n'est formellement jamais appelée dans le code du programme et peuvent donc être considérées comme n'étant jamais modifiées dans le programme. Par exemple, si un compilateur détecte qu'une variable n'est jamais modifiée dans un programme, il peut très bien la remplacer par une constante (et ce silencieusement, bien entendu).

Exemple

Examinez l'exemple suivant avec une variable partagée entre le `main` et un traitant quelconque. Le `main` ne fait que consulter (lire) la variable qui est modifiée (écrite) dans le traitant. La variable est bien modifiée dans le programme, certes, mais dans une fonction particulière (le traitant) qui n'est véritablement jamais appelée dans le processus ! En effet, comme il s'agit d'un traitant, cette fonction est exécutée lors de la délivrance d'un signal (qui, pour le coup, est bien un événement extérieur au programme !). Donc, pour le compilateur, tout se passe comme si cette variable partagée n'était jamais modifiée, ce qui n'est bien évidemment pas lecas. Et si votre `main` a l'idée de faire :

```
while(var > 0) {
/* attendre par exemple */
};
```

alors le compilateur pourra remplacer ce code par :

```
while( 1 ) {
/* attendre par exemple */
};
```

14. `sig_atomic_t` porte très mal son nom car il ne s'agit pas d'un type véritablement atomique. Le type `sig_atomic_t` garantit l'« atomicité » du point de vue des traitants d'interruption/signaux mais pas du point de vue des accès effectués par des threads sur la variable. Plutôt que d'utiliser une variable de type `volatile sig_atomic_t`, il est possible d'utiliser une variable `_Atomic`, mais uniquement si cette dernière est également *lock-free* (cf. Section 8.5) afin d'éviter les problèmes d'interblocage décrits dans le paragraphe 8.2.2.

Ce qui va poser — tout le monde en conviendra — un sérieux problème.

— Troisièmement, ces variables doivent être de type `sig_atomic_t` afin de garantir que les accès se fassent correctement du point de vues des traitants de signaux (et même d'interruptions en fait). Ainsi, les accès à ces variables partagées doivent se faire de façon **atomique (c'est-à-dire que l'opération se fait entièrement ou pas du tout)**. Cela permet d'obtenir une garantie quant à la cohérence des données manipulées [15]. Cela constitue une excellente occasion pour introduire la notion d'**exclusion mutuelle**.

Exclusion mutuelle

Dans le cas où vous voulez accéder à une telle variable (e.g. une variable 64 bits sur un système 32 bits), vous devez assurer un accès en `exclusion mutuelle`, c'est-à-dire avec la garantie qu'au moment où un flot d'éxécution accède à cette variable (par exemple le `main`), aucun autre flot ne peut y accéder (par exemple un traitant de signal).
Comment obtenir une telle garantie d'accès ?

— Si la variable est atomique [16], alors il y a *de facto* une forme de garantie d'exclusion mutuelle.

— Si ce n'est pas le cas, il est possible de mettre en place un masque approprié qui bloquera le ou les signaux pendant l'exécution de la **section critique de code**. N'oubliez pas que dans le `main` (ou toute fonction appelée par le `main`, le processus peut être interrompu à tout moment par un traitant de signal (sauf si ce dernier est bloqué bien entendu), mais que si le processus se trouve dans un traitant, alors on est sûr que le `main` ne sera pas exécuté avant la fin du traitant !

— En revanche, un traitant peut toujours être lui-même interrompu par un autre traitant !

6.4.3 Interactions entre appels-système et traitants de signaux

En lisant le manuel, vous remarquerez qu'une proportion non négligeable des appels-systèmes peut être interrompue par l'exécution d'un traitant de signal. Dans ce cas, l'appel-système en question renverra -1 comme code de retour , tandis qu'`errno` sera positionné à `EINTR`.



Ceci n'est pas *stricto sensu* une erreur, et ne doit pas être traité comme tel !

En fait, la valeur -1 ne signifie pas forcément qu'une erreur s'est produite. Il faut plutôt interpréter cela comme un évènement qui fait que l'appel ne va pas jusqu'à son terme. Pour connaître la cause de ce résultat, il **faut** consulter la valeur d'`errno`. Dans le cas d'une interruption de l'appel par un traitant de signal, cette valeur sera donc `EINTR`.



Ceci n'est valable/applicable que pour les signaux pour lesquels un processus a mis en place un traitant défini par l'utilisateur (et donc autre que `SIG_IGN` et de `SIG_DFL`).

Comment garantir qu'un appel-système ne soit pas interrompu par un (traitant de) signal ? Il existe plusieurs solutions à votre disposition :

— bloquer le signal (cf. Section 6.5) avant d'effectuer l'appel-système et le débloquer après son retour. Mais cette solution n'est pas systématiquement applicable.

— positionner l'option `SA_RESTART` lors de la mise en place du traitant, ce qui aura pour effet de réexécuter automatiquement l'appel-système interrompu par le signal. Il existe néanmoins

15. L'atomicité est garantie pour les traitants, mais pas pour les threads, cf. Section 8.5.
16. Attention, la définition de l'atomicité est variable (cf. Section 6.4.2).

un certain nombre d'appels-système pour lesquels cette option `SA_RESTART` ne fonctionne pas et donc l'appel ne sera pas automatiquement réexécuté [17].

— réexécuter manuellement et explicitement l'appel-système, par exemple avec une boucle du type :

```
1 do{
2   ret = appel_systeme(...);
3  } while (( -1 == ret) && ( errno == EINTR));
```



Ceci est **indispensable en programmation réseau**, avec des appels tels que (liste non-exhaustive) : `accept`, `connect`, `select`, `poll`, `read`, `write`, ...dès lors que vous mettez en place une gestion au fil de l'eau de la terminaison de vos processus-enfants (par exemple).



Un flot d'exécution bloqué dans un appel-système peut tout de même recevoir des signaux. En effet, le système d'exploitation peut tout à fait mettre son vecteur des signaux pendants à jour, puis basculer ce flot dans l'état « prêt ». Ce dernier sera éventuellement réordonnancé en mode actif utilisateur·trice et le traitant associé au signal sera donc exécuté.

6.4.4 Le traitant étendu

Si l'option `SA_SIGINFO` est positionnée dans les options d'une structure de type `struct sigaction` via le champ `sa_flags`, alors c'est le champ `sa_sigaction` (au lieu de `sa_handler`) qui est utilisé pour définir le traitant mis en place. Dans ce cas, le prototype d'un tel traitant est :

```
void nom_du_traitant(int signum, siginfo_t *info, void *context);
```

Le traitant prend donc deux paramètres en plus du numéro de signal : un pointeur sur une structure de type `siginfo_t` (cf. le manuel de `sigaction` pour la définition complète) et un pointeur sur un contexte d'exécution. Cette structure de type `siginfo_t` contient notamment les champs suivants :

```
siginfo_t {
int      si_signo;  /* Signal number                      */
int      si_errno;  /* An errno value                     */
int      si_code;   /* Signal code                        */
pid_t    si_pid;    /* Sending process ID                 */
uid_t    si_uid;    /* Real user ID of sending process    */
int      si_status; /* Exit value or signal               */
void    *si_addr;   /* Memory location which caused fault */
sigval_t si_value;  /* Signal value                       */
};
```

et permet d'obtenir plus d'informations quant au signal reçu par le processus et qui a déclenché l'exécution du traitant. Tous les champs ne sont pas nécessairement remplis quand un signal est reçu par le processus. Le paramètre de contexte contient aussi des informations.

17. cf. `man -s7 signal-safety` ou `man -s7 signal` pour en obtenir la liste.



Il n'est pas possible de définir simultanément les pointeurs `sa_handler` et `sa_sigaction` dans une structure de type `struct sigaction`.

6.5 Contrôle des signaux avec les masques

Nous avons vu qu'un signal peut être reçu à n'importe quel moment et interrompre un processus. Mais il est parfois nécessaire d'exécuter une section de code pendant laquelle on veut avoir **la garantie que le processus ne sera pas interrompu par un signal particulier**. Dans ce cas, il faut **bloquer** (on dit aussi masquer) ce signal. Cela se fait en créant et en mettant en place un **masque de signaux**, et la façon de faire ressemble beaucoup à celle utilisée pour la mise en place d'un traitant.

Il est aussi très important de comprendre que la mise en place d'un masque ne sert pas uniquement à bloquer un signal, cela peut aussi servir à le **débloquer**. L'opération effectuée (blocage ou déblocage) va dépendre de la façon dont le masque est construit et de la façon dont il sera installé.

De la même façon que pour les traitants, un masque reste en place jusqu'à ce qu'une demande de modification soit effectuée par le processus.

Structure `sigset_t`

La structure de donnée utilisée pour représenter un masque de signaux est un champ de bits (*bitmap*), de type `sigset_t`. Il n'est pas nécessaire de connaître plus de choses à propos d'une structure de ce type, si ce n'est qu'**elle doit être manipulée avec des fonctions spécifiques** (cf. ci-dessous).

Mise en place d'un masque

La mise en place d'un masque de signaux se fait en deux étapes successives :

1. La construction du masque à l'aide des fonctions suivantes :

```
int sigemptyset(      sigset_t *set);
int sigfillset (      sigset_t *set);
int sigaddset  (      sigset_t *set, int signum);
int sigdelset  (      sigset_t *set, int signum);
int sigismember(const sigset_t *set, int signum);
```

Paramètre 1 : Un pointeur sur un masque (de type `sig_set`) qui sera modifié dans la fonction.
Paramètre 2 : Si présent, le numéro du signal à mettre dans le masque.
Code de Retour : -1 si erreur / 0 en cas de succès / 0 (absence) ou 1 (présence) pour `sigismember`.

   Ces fonctions sont des **opérations ensemblistes** sur des objets de type `sigset_t`.

2. La mise en place effective du masque construit, avec un appel à la fonction `sigprocmask` :

```
int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);
```

Paramètre 1 : L'opération à effectuer, à choisir parmi :
   — `SIG_BLOCK` : le masque des signaux bloqués mis en place est l'**union**[a] du masque actuel du processus et du masque pris en second argument par la

fonction (i.e. `set`).
- — `SIG_UNBLOCK` : les signaux qui sont dans le masque `set` sont enlevés du masque actuel des signaux bloqués du processus.
- — `SIG_SETMASK` : le masque des signaux bloqués mis en place est le masque pris en second argument par la fonction (`set`).

Paramètre 2 : Un pointeur sur la structure de masque de signaux à appliquer.
Paramètre 3 : Un pointeur sur une structure de masque de signaux pour récupérer l'ancien masque du processus (pour éventuellement rétablir le comportement initial du processus). Si vous ne voulez pas sauvegarder cet ancien comportement, vous pouvez mettre simplement `NULL`.
Code de Retour : -1 en cas de problème (cf. la valeur d'`errno`) / 0 si succès .

a. Bitwise OR

Masque et traitants

Au début de l'exécution d'un traitant de signal, un masque est temporairement et automatiquement mis en place par le système. Ce masque contient les signaux suivants :
- — le signal qui a déclenché l'exécution du traitant dans lequel le processus se trouve
- — tous les signaux membres de l'ensemble du champ `sa_mask` de la structure `struct sigaction` passée en paramètre à la fonction `sigaction` lors de la mise en place du traitant.

Au retour du traitant, le système restaure le masque d'origine, c'est à dire le masque tel qu'il était avant l'exécution du traitant.



La restauration du masque d'origine a pour conséquence que toute modification du masque du processus pendant l'exécution du traitant (avec un appel à `sigprocmask`) ne sera pas conservée au retour du traitant !

Mise en place de masque et attente de signaux

Un problème récurrent en programmation système consiste en l'assurance de l'atomicité quand plusieurs opérations sont effectuées. Dans le cas des signaux, il peut arriver que l'on cherche à installer un masque de signaux pour bloquer ou débloquer un signal puis attendre le signal en question à l'aide d'une fonction d'attente explicite (cf. `pause` ou `sigwait`). Si le signal est reçu entre la mise en place du masque et l'appel à la fonction d'attente, alors le flot d'exécution appelant sera bloqué dans la fonction d'attente. Afin d'éviter ce genre de problème, il existe une fonction qui permet d'avoir ce type de comportement :

```
int sigsuspend(const sigset_t *sigmask);
```

Paramètre : Un pointeur sur la structure de masque de signaux à appliquer.
Code de Retour : -1 en cas de problème (cf. la valeur d'`errno`) / pas de valeur de retour sinon.

La fonction `sigsuspend` a le fonctionnement suivant :
1. elle installe le masque de signaux pris en argument et remplace donc le masque précédemment installé ;
2. elle met en sommeil le flot d'exécution appelant jusqu'à ce qu'un signal soit reçu qui provoque la terminaison du processus ou bien l'exécution d'un traitant de niveau utilisateur ;
3. dans le cas où un traitant est exécuté, `sigsuspend` retourne après cette exécution de traitant et le masque existant avant l'appel à `sigsuspend` est réinstallé.



L'installation du masque et la mise en sommeil sont faites atomiquement. De même, l'exécution du traitant de signal et la réinstallation du masque de départ sont effectuées atomiquement.

Signaux pendants et masqués

Un signal reste bloqué tant qu'un processus ne fait pas de demande de modification pour que ce signal soit retiré du masque de signaux bloqués. **Pendant tout le temps où le signal est bloqué, un processus peut tout de même le recevoir : ce signal restera dans l'état pendant.** Il existe une fonction permettant de prendre connaissance des signaux qui sont **à la fois pendants et bloqués** :

```
int sigpending(sigset_t *set);
```

Paramètre : Un pointeur sur un ensemble de signaux.
Code de Retour : -1 si problème (cf. la valeur d'`errno`) si problème / 0 si succès.

Cette fonction modifie le masque pris en argument et afin de savoir si un signal est à la fois bloqué et pendant. Vous devez utiliser la fonction `sigismember` sur ce masque pour connaître les signaux qui en font partie.

Exercice 49 ($\star$)

1. Écrivez un programme qui met en place un traitant de signal pour `SIGUSR1` et envoyez ce signal au processus avec une commande depuis un terminal.
2. Ensuite, modifiez le programme pour bloquer `SIGUSR1` et vérifiez que le masque est bien mis en place en continuant à envoyer le signal avec une commande depuis un terminal.

Exercice 50 ($\star\star$)

Rappel : un processus ne prend pas forcément en compte les signaux dans l'ordre où ils ont été émis. À l'aide des masques, faites en sorte de forcer cet ordre. Effectuez des tests avec `SIGUSR1`, `SIGUSR2` et `SIGSEGV` et vérifiez que votre programme fonctionne correctement : par exemple vous allez envoyer `SIGSEGV` en premier, mais vous voulez qu'il soit pris en compte en dernier, après `SIGUSR1` et `SIGUSR2`. Vous utiliserez une variable partagée entre le `main` et les traitants pour faire cet exercice.

6.6 Points Divers

Signaux et héritage

Rappel : les traitants sont hérités après un `fork` (cf. Section 3.5), **mais pas** après un `exec` (cf. Section 3.9)

Quant aux masques, ils sont herités après un appel à `fork` (cf. Section 3.5) **ou** `exec` (Section 3.9).

Interface obsolète

Il existe une autre fonction servant à installer un traitant de signal : la fonction `signal`. **L'emploi de cette fonction n'est pas recommandé** et il est préférable d'utiliser la fonction `sigaction` à sa place.

Pour aller plus loin : les signaux temps-réel

Les signaux évoqués jusqu'à présent sont les signaux dits *classiques* et ce sont eux que vous allez utiliser le plus souvent. Certains systèmes implémentent également des signaux dits *temps-réel*.

Ces signaux sont numérotés entre `SIGRTMIN` et `SIGRTMAX` (valeurs variables selon les systèmes).

Les différences entre les signaux classiques et les signaux temps-réels sont les suivantes :

- les signaux temps-réels ne sont pas attachés à des évènements particuliers ;
- le comportement par défaut d'un processus à la prise en compte d'un signal temps-réel est de se terminer ;
- de multiples instances d'un même signal peuvent être dans l'état pendant et pas juste une. Les signaux temps-réels sont donc moins sensibles aux problèmes de rafales ;
- il existe une garantie d'ordre de délivrance pour les signaux temps-réels, qui n'existe pas pour les signaux classiques. Cette garantie est valable aussi bien pour des signaux de types différents que pour des instances d'un même type de signal. Dans le cas de signaux de types différents, plus le numéro d'un signal est petit, plus il sera prioritaire. Dans le cas de mélange entre signaux classiques et temps-réels, l'ordre n'est pas spécifié [18].
- certains appels-système doivent être remplacés :
  - `sigaction` ⇒ `rt_sigaction`
  - `sigpending` ⇒ `rt_sigpending`
  - `sigprocmask` ⇒ `rt_sigprocmask`
  - `sigsuspend` ⇒ `rt_sigsuspend`
  - `sigtimedwait` ⇒ `rt_sigtimedwait`

  Ces remplacements sont justifiés par le fait que dans le cas de signaux temps-réels, la taille de la structure `sigset_t` passe de 32 bits (4 octets) à 64 bits (8 octets).

6.7 Pour aller plus loin : gestion des signaux *sans* les traitants.

Originellement, la seule façon d'utiliser les signaux était de (re)définir des traitants de signaux (cf. Section 6.4). Depuis, de nouvelles interfaces sont apparues, qui permettent de s'affranchir de la définition de telles fonctions de gestion et de traitement.

6.7.1 Réception explicite synchrone sans traitant

La première interface se base sur un appel explicite à une fonction de réception, dont il existe deux versions : `sigwait` et `sigwaitinfo`. Leur fonctionnement est similaire : l'appel à une de ces fonctions bloque le flot jusqu'à la réception d'un signal spécifié via un argument passé à la fonction (il est donc possible de sélectionner le ou les signaux que l'on souhaite attendre, à la différence de `pause`).

Le prototype de la fonction `sigwait` est le suivant :

```
int sigwait(const sigset_t *set, int *sig);
```

Paramètre 1 : Un pointeur sur un ensemble de signaux (cf. Section 6.5) que le flot appelant souhaite attendre .

Paramètre 2 : Un pointeur sur un entier qui retourne le numéro de signal effectivement reçu par le flot appelant.

Code de Retour : 0 en cas de succès / la valeur d'`errno` en cas de problème (remarquez que -1 n'est pas renvoyé).

Plus précisément, la fonction `sigwait` bloque le flot appelant jusqu'à ce qu'un des signaux précisés dans l'ensemble `set` passe dans l'état pendant : le bit de signal pendant correspondant est alors remis à 0 dans le vecteur et le numéro de signal est alors retourné dans le paramètre `sig`.

Le prototype de la fonction `sigwaitinfo` est le suivant :

18. Linux, *a priori*, donne la priorité aux signaux classiques.

```
int sigwaitinfo(const sigset_t *set, siginfo_t *info);
```

Paramètre 1 : Un pointeur sur l'ensemble des signaux que l'on souhaite attendre.
Paramètre 2 : Un pointeur sur une structure de type `siginfo_t` (cf. Section 6.4.4).
Code de Retour : -1 si problème (cf. valeur `errno`) / numéro de signal reçu sinon

Tout comme `sigwait`, cette fonction suspend l'exécution du flot appelant jusqu'à ce qu'un des signaux présents dans l'ensemble `set` passe dans l'état pendant. Dans le cas où un signal est déjà dans cet état, la fonction retourne immédiatement et le flot reprend son exécution. Le signal reçu est alors retiré du vecteur des signaux pendants du flot d'exécution et son numéro est renvoyé par la fonction `sigwaitinfo`. Si un pointeur autre que `NULL` est passé en second argument à la fonction, alors la mémoire pointée contient une structure de type `siginfo_t` décrivant le signal reçu.

Les différences entre les fonctions `sigwait` et `sigwaitinfo` sont les suivantes :

— `sigwait` ne renvoie que le numéro du signal à la place d'une structure `siginfo_t` le décrivant [19] ;
— Les valeurs de retour des deux fonctions sont différentes.



Pour utiliser `sigwait` ou `sigwaitinfo`, il est nécessaire de bloquer (via `sigprocmask` ou `pthread_sigmask`) les signaux que l'on cherche à recevoir, afin de garantir qu'ils passent dans l'état *pendant*.

Comparaison entre réception synchrone et asynchrone

Vous avez donc le choix entre gérer les signaux de façon *synchrone* (avec une mise en place de traitant avec `sigaction`) ou de façon *asynchrone* (via `sigwait` ou `sigwaitinfo`). Il est notamment possible de dédier un thread (cf. Chapitre 7) pour cette réception synchrone des signaux. Les avantages de cette méthode sont les suivants :

— Plus d'interruptions des appels-système (i.e. `errno == EINTR`)
— Plus de restrictions concernant les fonctions utilisables (i.e. plus besoin de se poser la question de l'*async-signal-safety*).

L'inconvénient est une mise en place plus complexe : création de thread + blocage *a priori* des signaux à recevoir.

6.7.2 Gestion des signaux via des descripteurs de fichiers (Non-POSIX, spécifique à Linux).

Ainsi que nous l'avons vu en tout début de cours (cf. Section 2.1), « tout est fichier » dans UNIX, c'est-à-dire que tout est accessible via le système de gestion de fichier. Pour rester cohérent avec ce principe, il existe dans GNU/Linux une interface permettant de recevoir des signaux via des descripteurs de fichier. Ainsi, l'activité d'un tel descripteur pourra être surveillée avec les fonctions `poll`, `ppoll` ou `epoll`(par exemple). Dans un premier temps, il faut créer un descripteur de fichier servant à la réception des signaux avec la fonction suivante :

```
int signalfd(int fd, const sigset_t *mask, int flags);
```

Paramètre 1 : L('index d)e descripteur de fichier servant à la réception. Dans le cas ou la valeur de `fd` est -1, alors un nouvel index est créé et retourné par la fonction. Dans le cas contraire, un index valable doit être fourni et qui correspond à un descripteur servant à recevoir des siganux. Si un autre type de descripteur est passé en argument, une erreur

19. C'est exactement la même différence entre le traitant par défaut et le traitant étendu.

sera retournée et `errno` vaudra `EINVAL`.
Paramètre 2 : Un pointeur sur l'ensemble des signaux que l'on souhaite attendre.
Paramètre 3 : Des options, comme `SFD_NONBLOCK` ou `SFD_CLOEXEC`.
Code de Retour : -1 si problème (cf. valeur `errno`) / (index de) descripteur de fichier



Il est indispensable de **bloquer** (avec la fonction `sigprocmask` par exemple) les signaux que l'on souhaite gérer avec un descripteur de fichier afin d'éviter que le comportement par défaut du flot appelant vis-à-vis de ces signaux ne se déclenche au moment de leur prise en compte.



Les remarques concernant les rafales de signaux sont toujours valables dans le cas de `signalfd`, ainsi un seul exemplaire de signal classique (i.e non temps-réel) est dans l'état pendant. Cela signifie qu'une seule structure associée à un type de signal sera lisible sur le descripteur de fichier même si plusieurs occurences de signal ont été émises.



Il n'est pas possible de gérer les signaux `SIGKILL` et `SIGSTOP` avec un descripteur de fichier, ni les signaux générés de façon synchrone comme `SIGSEGV` ou `SIGFPE`.

Pour récupérer les informations relatives au signal, qui sont normalement disponibles quand on définit un traitant (cf. Section 6.4), il faut effectuer une lecture (avec `read` par exemple) sur le descripteur de fichier lié au signal en passant en argument un *buffer* dont la taille doit être au moins celle d'une structure spécifique `struct signalfd_siginfo` dont la définition est la suivante :

```
struct signalfd_siginfo {
uint32_t ssi_signo;    /* Signal number                      */
int32_t  ssi_errno;    /* Error number (unused)              */
int32_t  ssi_code;     /* Signal code                        */
uint32_t ssi_pid;      /* PID of sender                      */
uint32_t ssi_uid;      /* Real UID of sender                 */
int32_t  ssi_fd;       /* File descriptor (SIGIO)            */
uint32_t ssi_tid;      /* Kernel timer ID (POSIX timers)     */
uint32_t ssi_band;     /* Band event (SIGIO)                 */
uint32_t ssi_overrun;  /* POSIX timer overrun count          */
uint32_t ssi_trapno;   /* Trap number that caused signal     */
int32_t  ssi_status;   /* Exit status or signal (SIGCHLD)    */
int32_t  ssi_int;      /* Integer sent by sigqueue(3)        */
uint64_t ssi_ptr;      /* Pointer sent by sigqueue(3)        */
uint64_t ssi_utime;    /* User CPU time consumed (SIGCHLD)   */
uint64_t ssi_stime;    /* System CPU time consumed (SIGCHLD) */
uint64_t ssi_addr;     /* Address that generated signal
.                         (for hardware-generated signals)     */
uint16_t ssi_addr_lsb; /* Least significant bit of address
.                         (SIGBUS; since Linux 2.6.37)         */
uint8_t  pad[X];       /* Pad size to 128 bytes (allow for
```

```
.                           additional fields in the future)     */
};
```



Il est indispensable de lire les données disponibles sur le descripteur de fichier sinon la fonction `poll` continuera de détecter un évènement sur ce descripteur une fois une occurence du signal reçue (risque potentiel de boucle infinie).

6.8 Travail dirigé : gestion des zombies avec les signaux

6.8.1 Récupération des zombies et traitant de signal

Afin de mettre en pratique la gestion des signaux, je vous propose un travail sur la récupération des enfants zombies d'un processus.

Question 1

Quelles sont les différents moyens/techniques pour récupérer un enfant zombie ?

Question 2

Quels sont les avantages/inconvénients de ces méthodes ?

Question 3

On suppose maintenant que l'on va travailler sur un programme qui répond à la question du premier exercice de la Section 3.7. Quel est le problème posé par les enfants zombies dans le cas d'un tel programme ? Quelle méthode proposez-vous pour la prise en compte de ces zombies ?

Question 4

Écrivez une première version d'un traitant avec la fonction `wait`. Testez-là. fonctionne-t-elle ? Cette version est-elle satisfaisante dans **tous** les cas ?

Question 5

Améliorez votre traitant pour une meilleure prise en compte des zombies en évitant le problème de la rafale. Testez à nouveau votre code : est-il fonctionnel ?

Question 6

Que faut-il modifier pour obtenir une version complètement fonctionnelle ? Indice : allez relire les règles d'écriture d'un traitant (cf. Section 6.4.2).

6.8.2 Et maintenant, du côté du `main` ...

Il n'est pas déraisonnable de penser que l'ensemble des processus-enfants zombies sera pris en compte lors de l'exécution du `main`, c'est-à-dire pendant son travail principal. Il peut cependant arriver que ce ne soit pas toujours le cas et alors, le `main` doit attendre que l'ensemble des zombies soit traité avant de terminer son exécution. Cette attente peut prendre plusieurs formes, mais celle-ci est assez raisonnable :

```
1 sig_atomic_t num_enfants = 0;
2
3 /* traitant pour le signal SIGCHLD */
4
5 void traitant(int sig)
6 {
7   /* code du traitant : cf. partie 1*/
```

```
8   /* quand recuperation d'un zombie : */
9
10  num_enfants--;
11 }
12
13 int main(int argc, char **argv)
14 {
15  fd = accept(....);
16
17  if(fd > 0){
18    /* creation d'un enfant */
19    fork();
20    num_enfants++;
21   }
22
23  /* travail du main */
24
25  /* eventuellement, on peut n'avoir
26     plus rien a faire : attente passive */
27
28  while(num_enfants > 0){
29    pause();
30  }
31
32  exit(EXIT_SUCCESS);
33 }
```

Question 1

La gestion de la variable `num_enfants` est-elle correcte ? Que faut-il modifier/ajouter éventuellement ?

Question 2

L'appel à `pause` est-il obligatoire ? Quelle solution alternative proposez-vous ?

Question 3

En supposant que le traitant correspond à celui obtenu à la fin de la partie 1, la solution d'attente dans le `main` est-elle fonctionnelle dans **tous** les cas ?

Question 4

Écrivez une nouvelle solution qui met en place des masques de signaux. Le problème est-il réglé ?



7. Splendeur et tourments du multithreading

7.1 Notions de multithreading

Résumé de l'épisode précédent

Dans le chapitre précédent, vous avez eu l'occasion d'expérimenter une forme particulière de programmation multi-flots avec la mise en place de traitants de signaux. Particulière, parce que l'exécution des flots supplémentaires (i.e. en plus du `main`), que sont les traitants de signaux est dépendante d'événements particuliers et que la synchronisation des flots entre-eux (via la mise en place de masques) est assez limitée et peu pratique [1]. De plus l'écriture d'un traitant est régie par des règles qui, si elles ne sont pas respectées, peuvent entraîner le blocage pur et simple de tout le processus (e.g. un appel non *async-signal-safe* dans un traitant). En revanche, cela préfigure assez bien ce que sera la véritable programmation multiflot : le fait de ne pas contrôler précisément l'exécution des flots, c'est-à-dire au final l'entrelacement des instructions exécutées par chacun, la nécessité de devoir synchroniser ces flots entre eux, notamment pour l'accès à des données partagées, etc.

7.1.1 Vers toujours plus de concurrence : les threads

Vocabulaire 7.1 — Thread. Un *thread*, également appelé **processus léger**, est un flot d'exécution créé au sein d'un processus lourd classique (tels que ceux que vous avez manipulés jusqu'à présent). En fait, c'est un peu plus que juste un flot d'exécution, car au thread est également associé son contexte d'exécution (e.g. l'état des registres du processeur exécutant les instructions du flot, par exemple). Les threads sont également appelés des processus légers, car leur coût de création, destruction et de synchronisation est moindre que dans le cas de processus UNIX traditionnels.

1. Par exemple, le partage de variables entre le `main` et le(s) traitant(s).

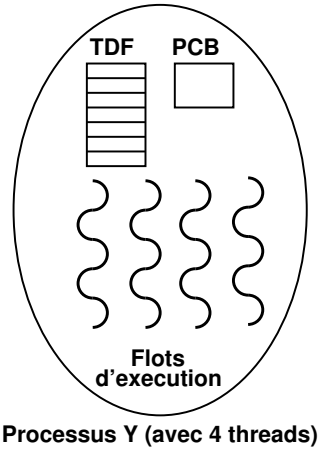


FIGURE 7.1 – Schéma d'un processus multithreadé avec ses ressources associées.



Un processus UNIX possède toujours au moins un flot d'exécution (thread) : il s'agit en règle générale du thread qui exécute le `main` du processus [a].

a. Mais des cas pathologiques sont toujours possibles : par exemple, on peut facilement écrire un programme dont le `main` crée un nouveau thread puis se termine. Dans ce cas, le processus existe toujours avec un unique thread, mais ce n'est pas celui qui exécute le `main`.

Partage et adressage

Si un processus possède donc toujours **au moins** un flot d'exécution, il est possible d'en **créer de nouveaux** en son sein. (c.f Section 7.2.2). Les conséquences sont importantes : **tous les threads d'un même processus se partagent son espace d'adressage**. Donc, **toutes les variables globales** du processus sont potentiellement accessibles par **n'importe quel** thread de ce processus. D'où la nécessité de bien veiller à synchroniser les accès (i.e. lectures/écritures) le cas échéant. Ces problèmes de partage et/ou d'accès à des données communes est une source importante d'erreurs dans les programmes multithreadés [2]. Partager de la mémoire entre thread est donc beaucoup plus simple que dans le cas de processus classiques, puisque il n'y a pas de fonction comme `mmap` à appeler.



Utiliser les threads est une manière de faire de la programmation multi-flots, créer de nouveaux processus en est une autre. Dans ce second cas, une application sera donc multi-processus, ce qui revient à dire qu'il n'y a pas forcément de bijection entre programme (application) et processus.



Qui dit programmation multi-flots dit concurrence. Et qui dit concurrence, dit section critique, synchronisation, etc.

De l'utilité du thread

Rappelez-vous que la création d'un processus mobilise un certain nombre de ressources dans le système et que cela prend du temps également. Cependant, il est parfois inutile (ou trop coûteux) d'utiliser des processus classiques quand ce que l'on cherche à obtenir est la mise en place d'un flot d'exécution dont le seul but est d'exécuter une petite charge de travail. Dans un tel cas, avec un

2. Et désormais, vous ne pourrez vous prévaloir de votre ignorance de la chose !

processus traditionnel, il n'est pas inhabituel que le temps de création et de destruction du processus soit supérieur au temps pris pour exécuter le travail (code) désiré, ce qui est calamiteux en termes de performances. Par exemple, dans le cas d'un processus serveur, il est peut-être plus intéressant de créer un thread pour traiter les requêtes que de créer un nouveau processus (lourd) dédié à chacune.

Une autre situation est la mise en place d'une activité en tâche de fond au sein d'un processus. Par exemple, il peut être utile d'avoir une partie du processus qui prend en charge les communications réseau tandis que le `main` fait autre chose. Avec un processus ne possédant qu'un seul flot, vous devez l'interrompre afin de pouvoir gérer ces communications. Comment le faire de façon **régulière** ? Vous pouvez effectuer ce traitement dans le `main`, mais la régularité n'est pas assurée. C'est possible en armant un *timer*, ce qui revient ni plus ni moins à mettre en place un traitant de signal [3]. Utiliser un thread est ici plus adapté car ce dernier va s'exécuter en même temps que le `main` sans interagir avec lui (en supposant que ces deux flots ne partagent pas de données, bien entendu). En effet, l'ordonnancement des threads est géré par le système directement, sans intervention de votre part [4].

Enfin, dans un contexte où tous les processeurs actuels possèdent de multiples unités d'exécution (CPU multicœur), les threads constituent le moyen le plus direct pour **paralléliser** une application [5] car chaque thread sera exécuté sur un cœur différent (cf. Section 7.1.1).

Pour aller plus loin : la parallélisation avec des threads

Vous souhaitez paralléliser un code avec des threads ? vous avez plusieurs options : le faire à la main , c'est à dire introduire vous-même des threads dans le code de votre application, avec des appels explicites aux fonctions de l'interface pthread (c.f ci-dessous). Mais sachez qu'il existe également un langage appelé **OpenMP**, qui permet de faire une parallélisation de code C avec des threads à l'aide de **directives de compilation**, ce qui suppose un support de la part du compilateur C utilisé [6].

Pour aller plus loin : parallélisation et granularité

Il existe plusieurs moyens pour paralléliser une application : vous pouvez utiliser plusieurs processus ou bien plusieurs threads à l'intérieur d'un processus. Pourquoi choisir une solution plutôt que l'autre ? Cela dépend du travail qui sera exécuté par chacun des flots et c'est ce qu'on appelle la **granularité**. Typiquement, on utilise plutôt des **processus classiques** pour une parallélisation à **gros grain** et des **threads** pour une parallélisation à **grain fin**, c'est à dire quand le travail que chaque thread doit effectuer n'est pas trop conséquent. C'est en vous basant sur votre expérience que vous serez à même de choisir l'outil le plus adapté pour écrire vos programmes parallèles.

3. Le monde est petit ...
4. Ce n'est pas vrai pour tous les threads. En effet, il existe des thread *user* et des threads *kernel*, mais en pratique vous n'allez rencontrer que des threads de niveau *kernel* donc je n'insiste pas plus sur ce point concernant l'ordonnancement des threads par le système.
5. Mais le plus direct ne signifie pas nécessairement le plus facile.
6. cf. l'option `-fopenmp` de `gcc`.



T2 découvrant le multithreading. L'évolution est en marche (ou pas).

7.2 L' interface Pthread

Dans cette section, nous allons présenter l'interface à utiliser pour programmer avec des threads. Cette interface sera celle des *pthreads*, pour POSIX threads. Elle est standard et disponible sur tous les systèmes UNIX.



Quand vous compilez des programmes utilisant des pthreads, il **faut** utiliser l'option de compilation `-lpthread`.

7.2.1 Identification

Un thread est identifié par une structure opaque de type `pthread_t`. Pour connaître son identifiant, un thread doit appeler la fonction suivante :

```
pthread_t pthread_self(void);
```

Paramètres : Aucun.
Code de Retour : Identifiant du thread appelant.



Cet identifiant n'est pas un numéro, cela ne sert à rien de l'afficher comme tel dans votre programme [a].

a. Et comme il s'agit d'une structure opaque, rien ne garantit qu'il s'agisse d'un numéro d'ailleurs ...

Comme le type `pthread_t` est opaque, il n'est pas possible non plus de se servir de l'égalité classique en C (==) pour faire des comparaisons. Il faut utiliser à la place :

```
int pthread_equal(pthread_t t1, pthread_t t2);
```

Paramètre 1 : Identifiant du premier thread.
Paramètre 2 : Identifiant du second thread.
Code de Retour : 0 en cas de différence / 1 en cas d'égalité.



Cet identifiant de thread n'a de sens qu'au sein du processus. Il n'a **pas de valeur absolue dans tout le système** : communiquer un tel identifiant d'un processus à un autre n'a donc strictement aucun intérêt.

Cependant, un thread peut **éventuellement** être identifié par le système avec un numéro : le *thread ID* (ou TID). Dans ce cas, ce TID est similaire au PID d'un processus, mais fonctionne au niveau des threads de ce dernier. Il existe alors un lien entre les divers TIDs et le PID d'un processus : le PID du processus est égal au TID de son thread `main`. La fonction permettant de récupérer cet identifiant ne se conforme pas à la norme POSIX et est spécifique à Linux (à la différence de `getpid`). Elle s'appelle `gettid` et un exemple d'utilisation est donné en Section 7.3.3 :

```
pid_t gettid(void);
```

Paramètres : Aucun.
Code de Retour : Le TID du thread appelant. Cette fonction ne retourne jamais d'erreur.

7.2.2 Création de thread

Nous allons maintenant apprendre à créer de nouveaux flots d'exécution au sein d'un processus déjà existant. Vous avez déjà vu les processus et leur modèle de programmation de type *fork/join* (cf. Section 3.6) où l'on duplique une ressource pré-existante. Vous vous souvenez également que pour exécuter un code différent de celui du processus-parent, il faut ensuite effectuer une opération de recouvrement (cf. Section 3.9).

Créer un nouveau thread ressemble un petit peu à la création d'un nouveau processus, à la différence que le modèle de programmation est de type *fork+exec/join*, c'est-à-dire que l'on indique dès la création du thread le code (en fait, une fonction) que ce dernier doit exécuter. Le prototype de la fonction de création est le suivant :

```
int pthread_create(pthread_t *restrict thread,
.                  const pthread_attr_t *restrict attr,
.                  void *(*start_routine) (void *),
.                  void *restrict arg);
```

Paramètre 1 : Pointeur sur l'identifiant du nouveau thread créé (indéfini en cas d'erreur).
Paramètre 2 : Pointeur sur les attributs du thread : mettre `NULL` pour les attributs par défaut.
Paramètre 3 : Pointeur sur la fonction à faire exécuter par le thread : cette fonction prend un pointeur **générique** (`void *`) en argument et renvoie également un pointeur **générique**.
Paramètre 4 : Pointeur sur l'argument à passer à la fonction exécutée par le thread.
Code de Retour : valeur d'`errno` en cas de problème / 0 en cas de succès.

Cette fonction crée un nouveau thread dans le processus. Ce nouveau thread exécute la fonction `start_routine`, à laquelle on passe un unique argument, le pointeur `arg` (i.e. le dernier argument de la fonction `pthread_create`).



Tout thread peut créer d'autres threads. cette création n'est pas l'apanage uniquement du `main`.

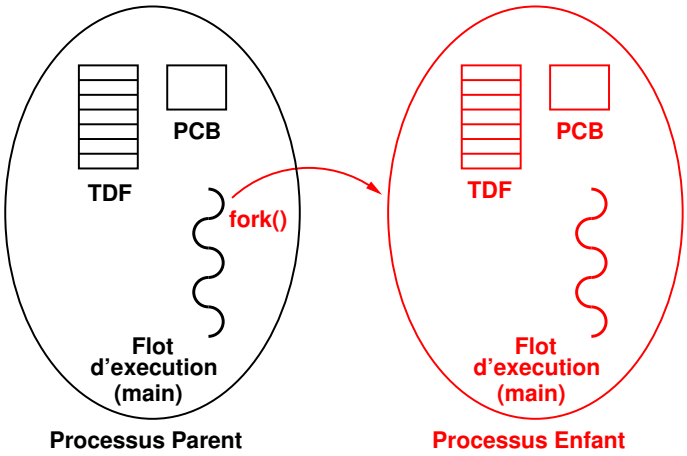


FIGURE 7.2 – Comparaison entre `fork` et `pthread_create`



Le nouveau thread ne démarre pas forcément **dès** que l'appel à `pthread_create` est effectué ! C'est parfois le cas, mais pas toujours, cela dépend donc de l'implémentation de la bibliothèque de threads du système sur lequel vous travaillez. Par conséquent, **vous ne pouvez jamais faire de suppositions sur le démarrage du thread quand vous écrivez vos programmes**, car cela n'est pas portable (cf. l'Interlude non-déterministe).

Exercice 51 (⋆)
Écrivez un programme qui crée deux threads (en plus du `main`). Chaque thread exécutera sa propre fonction. Le `main` effectuera une boucle d'attente active ou un `sleep` de quelques secondes avant de se terminer.

Exercice 52 (⋆)
Modifiez le programme précédent afin que les deux threads exécutent la même fonction, mais avec un paramètre différent.

Exercice 53 (⋆⋆)
Écrivez un programme qui crée 10 threads. Chaque thread devra afficher son « indice » de création. Le `main` effectuera une boucle d'attente active ou un `sleep` de quelques secondes avant de se terminer.
— Est-ce conforme à vos attentes ?
— Comment expliquez-vous ce que vous observez ?

Exercice 54 (⋆)
Comparez le prototype d'une fonction exécutée par un thread avec celui d'un traitant de signal : quels commentaires pouvez-vous formuler ?

Exercice 55 (⋆)
Quel(s) moyen(s) pouvez-vous proposer pour passer plus d'un argument à une fonction exécutée par un thread ?

Pour aller plus loin : duplication de processus multithreadé

Dans le cas de la duplication d'un processus multithreadé, le nouveau processus-enfant ne possédera qu'un unique thread : **celui qui a fait appel à la fonction** `fork`. De plus, il n'est pas obligatoire que ce soit le thread qui exécute la fonction `main`. **Les autres threads ne seront pas dupliqués dans le nouvel espace d'adressage du processus-enfant**.

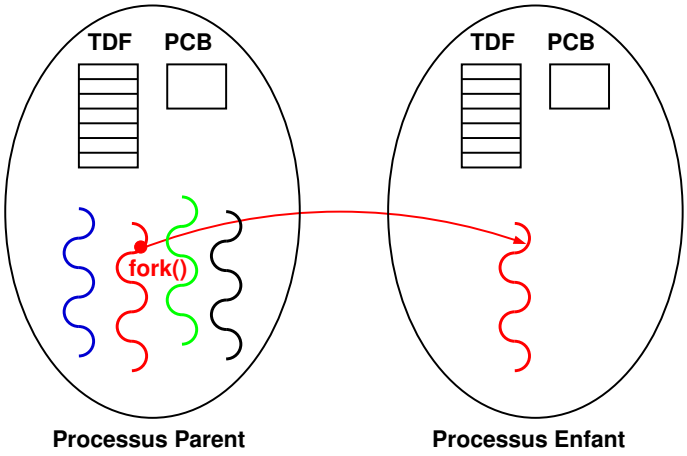


FIGURE 7.3 – Duplication (`fork`) d'un processus multithreadé.

Le fait que seul le thread faisant appel à `fork` subsiste [7] dans le nouveau processus implique toute une série de problèmes potentiels. Par exemple, si des threads sont en train de modifier des données dans une section critique de code, alors il est impossible de garantir que ces mêmes données seront dans un état cohérent quand elles seront dupliquées dans le nouveau processus.

De même, si un thread a fait l'acquisition d'un verrou (cf. Section 8.2) dans le processus-parent avant le fork et que ce même thread n'est pas dupliqué dans le processus-enfant, alors ce dernier pourra être bloqué si son thread essaye de faire l'acquisition du verrou, dont l'état n'est pas garanti.

En fait, ce problème peut se généraliser à **toute fonction de bibliothèque appelée dans le processus-enfant** puisque ces fonctions peuvent utiliser des données partagées ou des verrous [8]. La conséquence est que seules des fonctions *async-signal-safe* peuvent être appelées dans un processus-enfant après un `fork` d'un processus-parent multithreadé.

Certains de ces problèmes peuvent être résolus avec la fonction `pthread_atfork` (cf. Section 8.2).

7.2.3 Terminaison d'un thread

Il existe plusieurs moyens de mettre un terme à l'exécution d'un thread :

1. Soit le thread fait appel à la fonction `pthread_exit` :

```
void pthread_exit(void *retval);
```

Paramètre : Pointeur sur la valeur de retour du thread.
Code de Retour : Aucun, cette fonction est toujours un succès [a].

a. Ceci n'est pas un placement-produit.

7. Subsistence ou Existence ?
8. Et que vous ne connaissez pas *a priori* la façcon dont elles sont implémentées.

2. Soit le thread termine la fonction qu'il exécute (le paramètre `start_routine` de `pthread_create`) avec un appel à l'instruction `return`. C'est équivalent à appeler `pthread_exit` avec la valeur donnée au `return`[9].
3. Soit le thread est annulé par un appel à `pthread_cancel` (cf. Section 7.2.5)
4. Soit le processus entier se termine, et par conséquent **tous les threads** qui en font partie. Un appel à `exit` dans **n'importe quel thread** ou bien un `return` dans le `main` terminent effectivement le processus dans son ensemble.

Cas particulier du `main`

Le thread qui exécute la fonction `main` n'a pas de droits ou privilèges particuliers. Faire un `return` dans la fonction `main` est équivalent à appeler `exit` : cela implique la terminaison de tous les threads du processus, comme indiqué précédemment.

En revanche, il est possible de terminer le `main` avec un appel à `pthread_exit`, auquel cas, seul le thread exécutant le `main` se termine. Le processus prendra fin quand son dernier thread aura terminé son exécution. Dans ce cas, la valeur de retour du processus sera 0, et ce **quelle que soit la valeur** passée en argument au dernier `pthread_exit` ou `return` effectué.

Exercice 56 (⋆)

Écrivez un programme qui crée quelques threads. Ces threads afficheront un message avant de se mettre en sommeil[a]. Le thread exécutant le `main` appellera `pthread_exit` une fois ces créations effectuées en lieu et place de la fonction `exit`.

— Que constatez-vous ?
— Est-ce conforme à vos attentes ?

a. La fonction `sleep` permet de mettre en sommeil le thread appelant, pas tout le processus ...

7.2.4 Synchronisation de threads

Le modèle des threads étant de type *fork+exec/join*, il existe donc une opération permettant d'attendre la fin de l'exécution d'un thread et de récupérer la valeur de retour (le pointeur) que ce dernier a renvoyé. La différence avec les processus est que **n'importe quel** thread peut attendre la terminaison d'un autre thread. Ce n'est pas obligatoirement le flot qui a créé un thread qui doit se synchroniser avec lui. Cette synchronisation est possible grâce à la fonction suivante :

```
int pthread_join(pthread_t thread, void **retval);
```

Paramètre 1 : Identifiant du thread avec lequel le thread appelant veut se synchroniser (i.e. récupérer sa valeur de retour).
Paramètre 2 : Pointeur sur la valeur de retour du thread se terminant. En cas d'annulation du thread par `pthread_cancel`, cette valeur est `PTHREAD_CANCELED` (si annulation réussie).
Code de Retour : Valeur d'`errno` en cas de problème / 0 en cas de succès.

Cette fonction bloque le thread appelant jusqu'à ce que le thread identifié par le premier argument se termine. Si ce thread est déjà terminé au moment de l'appel, `pthread_join` retourne immédiatement.



Si plusieurs threads décident d'attendre le même thread, le résultat est **indéterminé**.

9. Rappelez-vous qu'une fonction exécutée par un thread retourne un pointeur.

Exercice 57 (⋆)
Reprenez les programmes précédents et enlevez les temporisations (i.e. les boucles d'attente active ou les appels à `sleep`) dans la fonction `main`.
— Que constatez-vous ?
— Comment corriger le problème ?

Exercice 58 (⋆⋆)
Écrivez un programme dont le `main` :
1. crée quelques threads ;
2. attend leur terminaison ;
3. récupère les codes de retour et en fait la somme (on supposera que le résultat attendu de chaque thread est un entier).

Exercice 59 (⋆⋆)
1. Écrivez un programme qui calcule le minimum d'un tableau de m entiers (les valeurs pourront être générées aléatoirement).
2. Mesurez le temps pris par ce programme pour plusieurs valeurs de m.
3. Modifiez ensuite ce programme pour que le calcul soit réparti parmi n threads où chaque thread devra donc traiter un sous-tableau d'environ m/n entiers (m n'est pas forcément un multiple de n). Refaites les mesures de temps pour plusieurs valeurs de m et n (1, 2, 4, etc.). Que constatez-vous ?

Le mode « détaché »

Si les threads fonctionnent par défaut en mode *fork+exec/join*, il est néanmoins possible de les « détacher », c'est-à-dire de faire en sorte que l'opération *join* ne soit plus applicable. En clair, il n'est plus nécessaire d'appeler `pthread_join` pour libérer les ressources du thread une fois qu'il est terminé. Ce mode « détaché » peut être spécifié soit lors de la création du thread (par le truchement de l'attribut idoine, avec un appel à la fonction `pthread_attr_setdetachstate`), soit une fois le thread créé avec un appel à la fonction :

```
int pthread_detach(pthread_t thread);
```

Paramètre : Identifiant du thread à « détacher ».
Code de Retour : Valeur d'`errno` en cas de problème / 0 en cas de succès.


L'appel à `pthread_join` n'étant plus possible, il faut trouver un autre moyen pour récupérer la valeur de retour du thread en mode « détaché ».


Le mode « détaché » est souvent utilisé dans le cas de threads dæmons, qui tournent en tâche de fond dans le processus, par exemple pour vérifier l'état des communications, etc.

7.2.5 Annulation de thread

Il est possible (mais pas vraiment recommandé) d'annuler un thread, c'est-à-dire de terminer son exécution de façcon prématurée, avant qu'il n'ait l'occasion de faire appel à `pthread_exit` ou

à l'instruction `return`. Voici la fonction qui permet de réaliser cette action :

```
int pthread_cancel(pthread_t thread);
```

Paramètre : Identifiant du thread à annuler.
Code de Retour : Valeur d'`errno` en cas de problème / 0 en cas de succès.

Veuillez noter qu'un appel réussi à `pthread_cancel` n'annule pas immédiatement le thread, mais indique que la requête en annulation a bien été prise en compte par le système. Le thread-cible sera annulé quand il atteindra un **point d'annulation**, en général quand il fera appel à une fonction pour laquelle le système peut annuler le thread. La liste de ces points d'annulation est disponible dans le manuel (`man -s7 pthreads`). **L'annulation n'est donc pas garantie** : si un thread reste bloqué dans une fonction qui n'est pas un point d'annulation possible, alors un appel à `pthread_cancel` ne servira à rien. Pour savoir si l'annulation a bien été effectuée, vous devez faire un appel à `pthread_join` et la valeur de retour du thread sera alors `PTHREAD_CANCELED` quand l'annulation sera effective.



Même dans le cas où l'annulation est effective, il faut appeler `pthread_join` afin de récupérer les ressources du thread annulé.



Autant que possible, il est préférable de terminer un thread normalement plutôt que de recourir à une annulation : un exemple classique est le thread qui reste bloqué dans un `select` (ou `poll`) alors qu'il n'y a plus de communications réseau qui vont arriver. La tentation est grande alors de vouloir annuler ce thread. Il existe une technique bien connue [a] permettant de ne pas recourir à ce genre d'artifice : l'utilisation d'un tube de communication (cf. Section 4.1) pour lequel le processus sera à la fois lecteur **et** écrivain. L'index de descripteur de ce tube sera mis dans l'ensemble des descripteurs à monitorer avec `select`. Une fois que l'on sait que les communications réseau sont terminées, une simple écriture dans le tube permet de faire sortir le thread du `select`, qui peut alors appeler `pthread_exit`. Simple, efficace, et de bon goût. Et surtout sans annulation.

a. dite du Hokutoshinken ...

7.2.6 Accès concurrents à un fichier

Dans le chapitre consacré à la manipulation du système de fichiers, nous avions vu que lors d'un `fork`, la table des descripteurs de fichiers du processus-parent était dupliquée dans le processus-enfant et que ces deux processus partagaient la même structure de fichier ouvert quand il accédaient au fichier via le (index de) descripteur dupliqué. La conséquence était que **la position courante (*offset*) dans le fichier était aussi partagée entre le parent et le enfant**. Toute modification de cette position courante par l'un des processus étant répercutée dans l'autre (et réciproquement).

Dans le cas d'un processus multithreadé, c'est également le cas puisque **la table des descripteurs est partagée par tous les threads du processus**. Donc, si un thread lit ou écrit dans un fichier via un (index de) descripteur, il modifie la position courante associée à ce(t index de) descripteur qui peut potentiellement être utilisé en même temps par un autre thread. Bref, un bel exemple d'accès concurrent à une ressource partagée, qu'il convient de gérer correctement.

Il existe des mécanismes de verrouillage de fichiers (cf. Section 8.2.3 notamment), mais je ne vais pas les décrire dans le détail. En revanche, je vous rappelle que **les fonctions `pread` et `pwrite` (cf. Section 2.4.1) ne modifient pas la position courante** : leur utilisation en environnement

multithreadé est donc recommandé.

7.3 Concepts avancés

7.3.1 Gestion des *Thread Specific Data*

Vous connaissez les variables globales, les variables locales et les variables locales statiques. Il existe avec les threads une nouvelle catégorie de variables : les variables dites *thread-specific*, qui sont déclarées tout comme des variables **globales**, c'est-à-dire avec un nom visible et accessible par tous les threads et/ou fonctions appelées dans les threads, mais dont la **valeur est spécifique à chaque thread**, comme pour une variable statique locale. Par exemple, c'est le cas de la variable `errno` : chaque thread peut consulter cette variable, mais la valeur d'errno sera privée à chaque thread.

Ces variables ne sont stockées ni dans la pile ni dans le tas du processus, mais dans une zone spécifique appelée *Thread Local Storage* (TLS) [10]. Pour déclarer une variable dans le TLS d'un thread, il suffit de rajouter le qualificatif `_Thread_local` au type de la variable **globale** déclarée. Par exemple :

```
1 #include <threads.h>
2
3 _Thread_local int bidule;
4
5 int main(int argc, char *argv[]){
6  ...
7 }
```

`bidule` est donc une variable entière globale, mais dont la valeur sera propre à chaque thread. Pour en savoir plus sur l'utilisation des TLS :

`http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4324.html`



L'emploi du qualificatif `__thread` remplace (plus ou moins) l'utilisation des fonctions suivantes qui ne seront pas décrites plus avant dans ce document :

- `pthread_key_create`
- `pthread_key_delete`
- `pthread_setspecific`
- `pthread_getspecific`

Exercice 60 (⋆)

Écrivez un programme qui crée quelques threads utilisant une variable stockées dans leur TLS respectifs et affichez la valeur de cette dernière. Est-il possible de se passer de ce mécanisme ?

7.3.2 MT-safe vs Async-signal-safe

Vous connaissez déjà la notion de fonction *async-signal-safe*, c'est-à-dire une fonction pouvant être appelée dans un traitant de signal de façon sûre. Il existe également une autre catégorie de fonctions : les fonctions dites réentrantes ou *multithreading-safe* (MT-safe). **Une fonction MT-safe**

10. Je laisse volontairement de côté l'implémentation de ce TLS, mais vous trouverez des articles consacrés au sujet sur les Internets. Par exemple : `https://it.bmc.uu.se/andlov/dev/manual/UlrichDrepper-TLS-ELFHandlingForThread-LocalStorage(tls.pdf).pdf`

peut être appelée simultanément par plusieurs threads qui s'exécutent concurremment. La réentrance de code est une propriété importante. Par exemple, un programme comportant un nombre important de variables globales ne sera probablement pas réentrant [11].

Pour autant, il ne faut pas confondre *MT-safe* avec *async-signal-safe*, ni même croire que les fonctions *MT-safe* forment un sous-ensemble des fonctions *async-signal-safe*. Pourtant, au premier abord, on pourrait croire qu'une fonction exécutable par plusieurs threads en même temps serait une bonne candidate pour être exécutée par un traitant de signal car un traitant de signal peut être considéré comme une sorte de nouveau flot d'exécution dans le processus, tout comme un thread.

Ce qui peut poser problème, ce n'est pas tant l'**implémentation** de la fonction que son **utilisation** : une fonction non *async-signal-safe* provoquera typiquement un interblocage entre un (ou des) thread(s) et le traitant, quand bien même elle serait *MT-safe*. Toutes les opérations de synchronisation (cf. le chapitre suivant) sont à manier avec précaution car rentrant sans doute dans cette catégorie (*MT-safe* mais non *async-signal-safe*) car la synchronisation implique souvent de devoir mettre en sommeil un thread/processus. C'est également le cas de `malloc` et `free`, à cause de leur utilisation plus que probable d'un verrou [12] en interne. De façcon plus générale, toutes les fonctions de bibliothèques qui utilisent des mécanismes de synchronisation pour protéger des données globales partagées sont de bonnes candidates : on retombe sur les mêmes problèmes que dans le cas du `fork` d'un processus multithreadé (cf. Section 7.2.2).

7.3.3 Threads et signaux

Pendant longtemps, les threads et les signaux étaient comme l'eau et l'huile : ils ne se mélangeaient pas très bien, à tel point que le comportement d'un processus multithreadé en cas de réception d'un signal n'était pas spécifié par la norme POSIX. Mais les choses ont évolué et il existe désormais des fonctions permettant de gérer les signaux dans un processus multithreadé.

Threads, traitants et masques

Un processus multithreadé aura le comportement suivant vis-à-vis des signaux :

- **Chaque** thread du processus possède son **propre masque** de signaux. Dans un tel processus, un appel à `sigprocmask` (cf. Section 6.5) n'est **pas spécifié** par POSIX : un appel à la fonction `pthread_sigmask` (cf. ci-dessous) est recommandé. Cette restriction est aussi valable **même** si l'appel à `sigprocmask` est fait **avant** un appel à `pthread_create`.
- Un thread créé par un appel à `pthread_create` **hérite du masque de signaux** du thread qui a effectué cet appel (et si le masque a été installé avec `pthread_sigmask`, cf. point précédent).
- Un traitant de signal est mis en place pour **l'ensemble des threads** d'un processus, il n'est pas possible de mettre en place un traitant individuel pour chaque thread du processus. En revanche, chaque thread peut masquer le signal individuellement, donc le choix d'un thread candidat pour la délivrance d'un signal peut être effectué ainsi : tous les threads sauf le candidat masqueront le signal. Également, **Les threads qui ne sont pas censés attendre un signal doivent donc le bloquer**, sous peine de blocage. Par exemple un thread appelant `sigwait` pourrait être mis en sommeil et potentiellement n'être jamais réveillé.
- Quand un processus multithreadé reçoit un signal envoyé avec la fonction `kill`, le thread qui exécute le traitant associé est **choisi par le sytème**.
- Dans certains cas particuliers, c'est le thread qui a provoqué l'envoi du signal qui va se le voir délivré. Par exemple, si un thread provoque une erreur de segmentation, c'est ce thread précisément qui exécutera le traitant associé au signal `SIGSEGV` et pas un autre.

Enfin, les contraintes d'ordonnancement sont un peu différentes que dans le cas d'un processus

11. C'est pour cela que les variables globales, c'est mal.
12. Un mécanisme de synchronisation possible.

monothreadé : en effet dans un tel cas, si le processus se trouve dans le traitant alors vous avez la garantie qu'il n'est pas dans le `main`. Dans le cas d'un processus multithreadé, **seul le thread qui reçoit le signal est bloqué tant que le traitant n'est pas fini. Les autres threads peuvent toujours s'exécuter**, ce qui veut dire que les sections critiques et les données partagées entre traitants et threads doivent faire l'objet d'une attention toute particulière.

Interface de manipulation des signaux

Certaines limitations évoquées ci-dessus peuvent néanmoins être levées en employant les fonctions idoines. En ce qui concerne l'envoi de signaux, si vous ne voulez pas envoyer un signal à un thread en particulier, vous pouvez continuer à utiliser la fonction `kill`. Dans le cas contraire, vous devez utiliser la fonction suivante :

```
int tgkill(int tgid, int tid, int sig);
```

Paramètre 1 : Numéro du groupe auquel appartient le thread-cible. En règle générale, ce numéro de groupe est égal au **PID** du processus auquel appartient le thread.
Paramètre 2 : Numéro d'identification du thread-cible (TID). Ce numéro est obtenu avec un appel à `gettid` (cf. Section 7.2.1).
Paramètre 3 : Numéro du signal à envoyer.
Code de Retour : -1 en cas de problème (cf. valeur d'`errno`) / 0 en cas de succès.



Cette fonction ne peut pas être appelée directement, vous devez utiliser la fonction `syscall` pour l'exécuter. Exemple, pour un thread qui veut s'envoyer le signal `SIGUSR1`, on pourra utiliser l'appel suivant (Non-POSIX, spécifique à Linux) :

```
syscall(SYS_tgkill, getpid(), gettid(), SIGUSR1);
```

Cette fonction permet donc à un processus d'envoyer un signal à un thread appartenant à **n'importe quel autre processus**. Il existe une autre fonction, plus simple, si jamais vous aviez envie qu'un thread d'un processus envoie un signal à un autre thread du **même** processus :

```
int pthread_kill(pthread_t thread, int sig);
```

Paramètre 1 : Identifiant du thread-cible (cet identifiant n'a de sens qu'à l'intérieur d'un processus, à la différence du TID, qui est valable dans tout le système [a]).
Paramètre 2 : Numéro du signal à envoyer.
Code de Retour : Valeur d'`errno` (>0) en cas de problème / 0 en cas de succès.

a. *System-wide identifier*

La fonction `pthread_sigqueue` permet d'envoyer des données en même temps que le signal à un thread dans le processus (potentiellement le thread appelant) :

```
int pthread_sigqueue(pthread_t thread, int sig, const union sigval value);
```

Paramètre 1 : Identifiant du thread-cible (cf. `pthread_kill`).
Paramètre 2 : Numéro du signal à envoyer.
Paramètre 3 : Données à émettre, sous la forme d'une structure de type `union sigval` :
`union sigval {`

```
    int   sival_int;
    void *sival_ptr;
    };
```

Code de Retour : Valeur d'`errno` (>0) en cas de problème / 0 en cas de succès

L'usage de la fonction `sigprocmask` n'est pas possible dans le cas d'un processus multithreadé (le comportement de `sigprocmask` dans un tel cas est indéfini), il faut employer à la place la fonction `pthread_sigmask`, dont le fonctionnement est similaire à celui de `sigprocmask` (cf. Section 6.5), mais applicable pour des threads de façon individuelle :

```
int pthread_sigmask(int how, const sigset_t *set, sigset_t *oldset);
```

Paramètre 1 : Opération à effectuer, à choisir parmi :
- `SIG_BLOCK` : le masque des signaux bloqués mis en place est l'**union** [a] du masque actuel du processus et de du masque pris en second argument par la fonction (`set`).
- `SIG_UNBLOCK` : les signaux qui sont dans le masque `set` sont enlevés du masque actuel des signaux bloqués du processus.
- `SIG_SETMASK` : le masque des signaux bloqués mis en place est le masque pris en second argument par la fonction (`set`).

Paramètre 2 : Pointeur sur le masque de signaux à appliquer.

Paramètre 3 : Pointeur sur un masque de signaux pour récupérer l'ancien masque du processus (pour éventuellement rétablir le comportement initial du processus). Si vous ne voulez pas sauvegarder l'ancien comportement, vous pouvez mettre `NULL`.

Code de Retour : Valeur d'`errno` (>0) en cas de problème / 0 si succès.

a. Bitwise OR

Exercice 61 (⋆⋆)

Écrivez un programme dont le main crée trois threads qui effectuent chacun une boucle infinie et affichent une chaîne de caractères. À l'aide de masques, faites en sorte que le signal `SIGUSR1` soit reçu par le deuxième thread et le signal `SIGUSR2` par le troisième thread. L'émission des signaux se fera avec la commande `kill` depuis un autre terminal.



Maintenant que vous maîtrisez le multithreading, vous pouvez lancer un cri de victoire, tout comme Jean-Claude. Youpi.



8. Synchronisation inter-flots

Vous savez maintenant créer de nouveaux processus, qui peuvent éventuellement être eux-mêmes multithreadés : la programmation multi-flots est donc désormais à votre portée. Des écueils, cependant, demeurent : tout d'abord votre absence de contrôle de l'ordonnancement des multiples flots d'exécution qui coexistent au sein de votre programme (ou application), et ensuite le fait très probable que ces flots partagent des données ou souhaitent accéder à des ressources partagées simultanément.

Ces accès concurrents et le fait que l'exécution des flots dépend de l'ordonnanceur du système impliquent que pour obtenir un programme qui fonctionne, c'est-à-dire un programme où les données manipulées resteront dans un état cohérent, il est indispensable que l'exécution desdits flots soit organisée afin que cette intégrité et cette cohérence soient préservées. Il faut donc que ces multiples flots s'accordent sur un **ordre d'accès aux sections critiques et aux ressources partagées du programme**. En d'autres termes, **il est indispensable que ces flots se synchronisent les uns avec les autres.**

Dans les chapitres précédents, certains mécanismes ont déjà été présentés mais servant uniquement à contrôler (plus ou moins) l'exécution de flots entre eux. Ces mécanismes ne sont donc pas utilisés pour protéger des sections critiques, comme ceux décrits dans le reste de ce chapitre. Ces mécanismes sont :

— l'opération `wait` (ou `waitpid`) pour synchroniser des processus-enfants avec leur parent, afin que ce dernier puisse prendre connaissance du code de retour de ses enfants (cf. Section 3.6).
— l'opération `pthread_join` pour synchroniser des threads entre eux (attente et récupération du résultat d'un thread) (cf. Section 7.2.4).
— l'ouverture bloquante d'un tube de communication nommé pour synchroniser des processus quelconques (cf. Section 4.1.2).

Dans ce chapitre, vous allez découvrir des mécanismes supplémentaires de synchronisation, notamment pour protéger les sections critiques de code dans vos programmes ou mieux organiser leurs exécutions respectives. L'idée principale est que cette synchronisation repose quelque part sur un **consensus** : tous les flots impliqués **doivent** utiliser les mécanismes de synchronisation, sinon le

but recherché (e.g. la cohérence des données) ne sera pas atteint. C'est là une source d'erreur assez classique (on a vite fait d'oublier un thread). De plus, il est assez facile de se tromper en utilisant de tels mécanismes, ce qui peut conduire à des interblocages, autre source de problèmes lors de la conception de vos programmes.

Ce chapitre pourra être complémenté par diverses lectures. Je vous recommande le *Little Book of Semaphores* [2] qui vous permettra de mieux comprendre le fonctionnement de certains mécanismes présentés ici, et vous donnera l'occasion d'essayer de les implémenter par vous-mêmes. Vous trouverez sur les internets des cours divers et variés sur les synchronisations, par exemple [4].

Alors, *ready* ?



I was born ready !

8.1 Les sémaphores

Le premier mécanisme de synchronisation que nous allons étudier est dénommé **sémaphore**.

8.1.1 Fonctionnement théorique

Le fonctionnement d'un sémaphore peut être assimilé à celui d'une **boîte contenant des jetons**. Au départ, la boîte contient un certain nombre de jetons : ce nombre est positif ou nul, mais jamais négatif (il n'y a pas d'anti-jetons). Ce nombre conditionne le nombre de flots autorisés à entrer dans la section critique protégée par le sémaphore ou bien à continuer leur exécution. Trois opérations sont applicables à un sémaphore :

1. l'opération **INIT** : il s'agit de l'initialisation de l'objet. C'est lors de cette opération que le nombre initial de jetons est défini. Ce nombre pourra varier dans le temps et même dépasser cette valeur de départ : le nombre initial de jetons n'est donc **pas le nombre maximal** de jetons disponibles dans le sémaphore à un instant donné.
2. l'opération **P()** (*Probeer te verlagen*) : il s'agit de la prise d'un jeton dans le sémaphore. Deux cas peuvent se produire :
   — S'il reste **au moins un jeton** de disponible, il est retiré du sémaphore et le flot qui a pris ce jeton peut **continuer son exécution**. Le nombre de jeton disponible est alors décrémenté.
   — S'il ne reste **plus de jeton** disponible dans le sémpahore, alors le flot est **bloqué** jusqu'à ce qu'un autre flot remette un jeton dans le sémaphore.

   Cette prise de jeton est donc une opération potentiellement **bloquante**. De plus, elle est théoriquement **atomique**.
3. l'opération **V()** (*vrijgave*) : il s'agit de la remise d'un jeton dans le sémaphore (incrémentation du nombre de jetons). Cette opération est également **atomique**. Veuillez noter qu'il est possible pour un flot de **remettre un jeton même s'il n'en n'avait pas pris un auparavant**.

Suite à cette remise de jeton, il est alors possible que le nombre de jetons disponibles dans le sémaphore soit supérieur au nombre de départ (celui fixé lors de l'initialisation du sémaphore).

Un sémaphore n'offre pas de garantie d'exclusion mutuelle systématique pour deux raisons : d'une part le nombre de jetons peut être supérieur à 1, et même quand c'est le cas, le fait que n'importe quel flot soit capable de remettre un jeton annule cette garantie. Les sémaphores servent donc plus à **coordonner des exécutions de flots** qu'à protéger des sections critiques [1]. Il est cependant possible de garantir l'exclusion mutuelle avec un sémaphore — sous certaines conditions — mais cela demande une vigilance et une discipline particulières.

8.1.2 Interface pratique

Il existe deux interfaces pour utiliser les sémaphores : l'interface IPC [2] System V et l'interface POSIX. La première est un peu moins pratique à utiliser, elle ne sera pas décrite dans ce document, mais sachez qu'elle reste disponible. En ce qui concerne l'interface POSIX, un sémaphore sera un objet opaque de type `sem_t`, peu importe donc ce que cet objet contient ou son implémentation.



Vous devez compiler vos programmes avec l'option `-lpthread` ou `-lrt` quand ils utilisent les sémaphores POSIX.

Types de sémaphores

De plus, il existe au sein de l'interface POSIX deux types de sémaphores, comme les tubes de communication (cf. Section 4.1.1), c'est à dire :

— les sémaphores anonymes, utilisables entre processus affiliés ;
— les sémaphores nommés, utilisables entre processus quelconques.

D'un point de vue de l'interface, la seule différence se situe au niveau des opérations de création/initialisation. En dehors de cette opération de création/initialisation, les deux types de sémaphores s'utilisent de façon similaire.



Si les sémaphores sont avant tout utilisés pour synchroniser des processus, ils sont bien entendu utilisables pour synchroniser des threads au sein d'un même processus.

Création et initialisation d'un sémaphore nommé

La création d'un sémaphore nommé se fait avec la fonction `sem_open`, qui, comme `open`, possède deux prototypes selon les cas (cf. Section 2.4.1) :

```
sem_t *sem_open(const char *name, int oflag);
sem_t *sem_open(const char *name, int oflag,
.               mode_t mode, unsigned int value);
```

Paramètre 1 : Nom (chaîne de caractères) du sémaphore à créer. Ce nom est obligatoirement de la forme `/nom` et sa taille est au maximum de `NAME_MAX-4` (i.e. 251) caractères. Ce nom ne peut pas contenir le caractère '/' (en dehors de celui du début, qui est obligatoire).
Paramètre 2 : Opération(s) à effectuer pour le sémaphore.

1. Mais il ne s'agit pas là d'une règle absolue, bien entendu.
2. Inter Process Communication

— `O_RDONLY` : utilisation en lecture seule [a].
— `O_WRONLY` : utilisation en écriture seule [b].
— `O_RDWR` : utilisation en lecture et écriture.
— `O_CREAT` : création du sémaphore s'il n'existe pas. Le prototype de la fonction prend alors deux arguments supplémentaires :
  — un mode de création : cet argument s'utilise de la même façon qu'avec `open` (cf. Section 2.4.1).
  — une valeur correspondant au nombre initial de jetons dans le sémaphore.
— `O_EXCL` : mode exclusif, une erreur est renvoyée si le sémaphore existe déjà et qu'on demande sa création. `O_EXCL` est souvent utilisé conjointement avec `O_CREAT`.

Code de Retour : `SEM_FAILED` si problème (cf. valeur d'`errno`) / un pointeur sur un sémaphore en cas de succès.

a. Je pense que l'utilité d'un sémaphore en mode lecture seule est proche du zéro absolu.
b. Idem.

Désallocation de sémaphore nommé

Un sémaphore nommé peut être désalloué à l'aide de la fonction suivante :

```
int sem_unlink(const char *name);
```

Paramètre : Le nom du sémaphore que le flot souhaite désallouer.
Code de Retour : -1 en cas de problème (cf. la valeur d'`errno`) / 0 si succès.

Détruire un sémaphore pour lequel des flots sont en attente produit un résultat indéterminé, tout comme l'utilisation d'un sémaphore détruit.

Initialisation d'un sémaphore anonyme

Dans le cas anonyme, il n'est pas nécessaire de créer le sémaphore : il suffit de déclarer une variable de type `sem_t` quelque part dans le programme [3]. Il est cependant indispensable de l'initialiser avec la fonction suivante :

```
int sem_init(sem_t *sem, int pshared, unsigned int value);
```

Paramètre 1 : Un pointeur sur l'« objet » sémaphore à initialiser : ce pointeur doit être alloué **avant** l'initialisation.
Paramètre 2 : Le mode de partage : si `pshared` vaut 0, alors le sémaphore n'est partagé qu'entre les threads d'un même processus. Si `pshared` vaut autre chose, alors le sémaphore peut être partagé entre des processus différents. Dans ce cas, **le sémaphore doit être placé dans une zone de mémoire partagée** (cf. Section 4.2) entre ces processus.
Paramètre 3 : Le nombre de jetons initialement placés dans le sémaphore (ce nombre peut être être nul).
Code de Retour : -1 en cas de problème (cf. la valeur d'`errno`) / 0 si succès.



Attention à n'initialiser qu'une **seule fois** un sémaphore partagé entre processus [a]
De façon générale, réinitialiser un sémaphore produit un résultat non spécifié.

a. Typiquement, il suffit de faire la séquence d'actions suivantes :
— création de la zone de mémoire partagée.

3. En tant que variable globale, très vraisemblablement.

— initialisation du sémaphore dans cette zone de mémoire.
— création des processus-enfants qui partagent la zone de mémoire (et donc le sémpahore initialisé).



Un sémaphore situé dans une zone de mémoire partagée ne peut pas être placé n'importe où : son adresse doit être *alignée* (sur `alignof(sem_t)`).

Désallocation de sémaphore anonyme

Un sémaphore anonyme peut être désalloué à l'aide de la fonction suivante :

```
int sem_destroy(sem_t *sem);
```

Paramètre : Un pointeur sur le sémaphore que le flot souhaite désallouer.
Code de Retour : -1 en cas de problème (cf. la valeur d'`errno`) / 0 si succès.

Détruire un sémaphore pour lequel des flots sont en attente produit un résultat indéterminé, tout comme l'utilisation d'un sémaphore détruit.

Prise d'un jeton

Pour l'acquisition d'un jeton, il existe deux fonctions :

```
int sem_wait   (sem_t *sem);
int sem_trywait(sem_t *sem);
```

Paramètre : Un pointeur sur le sémaphore dont le flot souhaite acquérir un jeton.
Code de Retour : -1 en cas de problème (cf. la valeur d'`errno`) / 0 si succès.

La première fonction effectue une prise bloquante de jeton, tandis que la seconde effectue une prise non-bloquante de jeton : le flot continue son exécution si un jeton est disponible (et le nombre de jetons est décrémenté), mais si aucun jeton n'est disponible, alors la fonction retourne -1, `errno` vaut `EAGAIN` et **le flot n'est pas bloqué**.

Remise d'un jeton

La fonction de remise d'un jeton est triviale :

```
int sem_post(sem_t *sem);
```

Paramètre : Un pointeur sur le sémaphore pour lequel le flot remet un jeton.
Code de Retour : -1 en cas de problème (cf. la valeur d'`errno`) / 0 si succès.



La fonction `sem_post` est *async-signal-safe*, mais `sem_wait` et `sem_trywait` ne le sont pas.

Exercice 62 (⋆⋆)
Démarrage synchronisé : écrivez un programme qui lance dix processus-enfants. Chaque enfant se contente d'afficher une chaîne de caractères qui lui est propre, mais cet affichage doit se produire uniquement quand **tous** les enfants ont effectivement été créés par le processus-parent.

Exercice 63 (⋆⋆)

Écrivez un programme qui crée un nombre pair de threads :

— la moitié de ces threads sont des **producteurs**, qui écrivent chacun un caractère à la fois dans un buffer partagé. Ce buffer possède `NSLOTS` places de libre (au maximum). Les thread producteurs ne peuvent écrire dans le buffer que s'il y a de la place de libre.

— l'autre moitié de ces threads sont des **consommateurs**. Chaque consommateur lit un caractère à la fois depuis le buffer partagée. Un consommateur ne peut lire dans le buffer que s'il y a effectivement quelque chose à lire, c'est-à-dire quand un producteur a écrit dans le buffer. Une fois la lecture effectuée, la place est indiquée comme étant libérée (pour être utilisée par un producteur).

Vous ferez attention à bien synchroniser les exécutions des différents producteurs et consommateurs. Également, vous veillerez à ce que les données dans le buffer restent cohérentes et mettrez en place une gestion adéquate des places libres dans le buffer (ex : gestion circulaire, gestion du premier emplacement libre/à lire, etc.)



On peut vraiment tout faire avec des sémaphores...

8.2 Les verrous

Le deuxième mécanisme de synchronisation s'appelle le **verrou** : il permet de protéger une section critique de code en y **garantissant un accès en exclusion mutuelle**. De ce fait, on appelle souvent les verrous des *mutex* (pour MUTual EXclusion).

8.2.1 Fonctionnement théorique

Le fonctionnement d'un verrou repose sur deux opérations : la prise du verrou et sa libération.

1. opération **LOCK** : cette opération consiste à essayer de prendre possession du verrou. Deux cas sont possibles :
   — Soit le verrou est libre (i.e. aucun autre flot n'en n'a encore fait l'acquisition), auquel cas le flot prend le verrou et en devient le **propriétaire effectif** : il peut continuer son exécution, ce qui signifie que si le verrou protège une section critique de code, le flot peut accéder à cette section et en exécuter le code.
   — Soit le verrou est déjà pris (un autre flot en est déjà le propriétaire effectif). Dans ce cas, le flot qui essaye de faire l'acquisition du verrou sera mis en sommeil (**bloqué**) jusqu'à ce que le verrou soit libéré par son propriétaire.
2. opération **UNLOCK** : quand un flot effectue cette opération sur un verrou dont il est le propriétaire, ce dernier est libéré. Il peut donc être acquis à nouveau (par le même flot ou bien par un autre flot).

Pour protéger un accès à une variable non atomique, on utilisera typiquement un verrou. En effet, si l'accès à une variable est atomique, alors l'exclusion mutuelle est intrinsèque, ce qui n'est pas vrai dans le cas contraire. En revanche, et tout comme dans le cas des sémaphores, tous les flots qui souhaitent accéder à une section critique de code **doivent** prendre le verrou pour y accéder. **Un consensus est donc indispensable** : si l'un des flots ne le respecte pas, alors les données ne seront sans doute pas cohérentes.



Si un flot est déjà propriétaire d'un verrou et qu'il effectue à nouveau une opération d'acquisition (**LOCK**) sur ce même verrou, alors ce flot sera mis en sommeil, car le verrou a déjà été acquis (par lui-même, certes, mais il y a déjà acquisition) [a].

a. Il existe cependant des verrous dit récursifs, qui peuvent être acquis plusieurs fois par leur propriétaire.



Pour un flot, c'est (normalement) une erreur de libérer un verrou dont il n'est pas le propriétaire.

Différences entre verrou et sémaphore

Un verrou diffère d'un sémaphore par plusieurs aspects :

1. Un verrou possède un unique propriétaire, tandis qu'un sémaphore n'a pas de propriétaire
2. Seul le propriétaire du verrou peut le libérer. Dans le cas d'un sémaphore, n'importe quel flot est autorisé à remettre un jeton, même s'il n'en n'avait pas pris un.
3. Un verrou (utilisé correctement) garantit l'exclusion mutuelle *systématiquement*, ce que ne fait pas un sémaphore.



Vous pourrez trouver certaines sources indiquant que les sémaphores assurent l'exclusion mutuelle, ce qui est contradictoire avec ce qui est indiqué juste auparavant. Ce qui est important de comprendre, c'est la présence du mot « systématiquement » dans le texte précédent. En effet, un verrou — utilisé correctement **et** dans un programme exempt d'erreur de programmation [a] — garantit l'exclusion mutuelle dans **tous les cas**.
Par ailleurs, il est *possible* d'utiliser des sémaphores pour garantir l'exclusion mutuelle, mais cela implique d'appliquer des **contraintes supplémentaires**, qui ne ne sont pas nécessaires dans le cas d'une utilisation correcte (et dans un programme exempt d'erreur) de ce mécanisme de synchronisation. C'est pour cela que cette garantie n'est pas *systématique*.

a. Rappelons qu'un consensus entre tous les flots accédant à une section critique et concernant l'utilisation d'un verrou est nécessaire. Si un flot ne respecte pas ce consensus et n'utilise pas de verrou pour accéder à la section critique, alors il est évident que l'exclusion mutuelle ne sera pas garantie.

Exercice 64 (⋆)
Un sémaphore initialisé avec un unique jeton est-il équivalent à un verrou ?

Exercice 65 (⋆⋆⋆)
Quelles conditions supplémentaires sont-elles nécessaires pour garantir l'exclusion mutuelle avec un sémaphore ?

8.2.2 Interface pratique

Les verrous sont implémentés dans la bibliothèque *pthread* : ce sont des objets opaques de type `pthread_mutex_t`. Si le fonctionnement théorique fait état de deux fonctions pour les verrous, l'interface pratique en comporte plus : en effet, il y a (forcément) besoin d'initialiser et de désallouer l'objet après son utilisation.

Initialisation

La première fonction est donc l'initialisation :

```
int pthread_mutex_init(pthread_mutex_t *mutex,
.                      const pthread_mutexattr_t *mutexattr);
```

Paramètre 1 : Un pointeur sur le verrou à initialiser.
Paramètre 2 : Un pointeur sur les attributs du verrou : vous pouvez mettre `NULL` pour mettre les attributs par défaut (ce que vous voulez dans 99,99 % des cas). C'est notamment par le biais des attributs qu'il est possible de créer un verrou récursif.
Code de Retour : Cette fonction renvoie toujours 0.

Il est également possible d'initialiser statiquement un verrou avec la constante `<PTHREAD_MUTEX_INITIALIZER`[4], mais uniquement au moment de la déclaration de la variable de type `mutex`. Par exemple :

```
pthread_mutex_t mon_verrou = PTHREAD_MUTEX_INITIALIZER;
```

Dans tous les autres cas, il faut utiliser la fonction `pthread_mutex_init`.



Le fait que les mutex soient implémentés dans la bibliothèque *pthread* signifie que ces objets sont plutôt utilisés entre les threads d'un même processus. Néanmoins, il est toujours possible de les utiliser entre des processus, mais dans ce cas, il faut déclarer ces verrous dans une zone de **mémoire partagée** (cf. Section 4.2). Il faut également positionner un attribut de partage de verrou à l'aide de la fonction `pthread_mutexattr_setpshared`. Enfin, un verrou situé dans une zone de mémoire partagée ne peut pas être placé n'importe où : sa structure doit être *alignée* (sur `alignof(pthread_mutex_t)`).

Désallocation de la structure

La fonction pour désallouer la structure associée au verrou est la suivante :

```
int pthread_mutex_destroy(pthread_mutex_t *mutex);
```

Paramètre : Un pointeur sur la structure de verrou à libérer.
Code de Retour : Valeur d'`errno` en cas de problème / 0 si succès.

Il faut que le verrou soit libre pour pouvoir le désallouer. N'importe quel flot peut procéder à cette désallocation, pas forcément celui qui l'a initialisé.

Acquisition d'un verrou

Pour faire l'acquisition d'un verrou, vous avez à votre disposition deux fonctions :

4. `PTHREAD_RECURSIVE_MUTEX_INITIALIZER_NP` pour un verrou récursif et `PTHREAD_ERRORCHECK_MUTEX_INITIALIZER_NP` pour un verrou à vérification d'erreur.

```
int pthread_mutex_lock   (pthread_mutex_t *mutex);
int pthread_mutex_trylock(pthread_mutex_t *mutex);
```

Paramètre : Un pointeur sur le verrou à prendre.
Code de Retour : Valeur d'`errno` en cas de problème [a] / 0 si succès.

a. `EBUSY` si le verrou est déjà pris dans la cas de `pthread_mutex_trylock`

`pthread_mutex_lock` permet une acquisition **bloquante** de verrou tandis que `pthread_mutex_trylock` est une version **non-bloquante** de cette fonction.

Libération de verrou

La fonction de libération d'un verrou est également triviale :

```
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

Paramètre : Un pointeur sur le verrou à libérer.
Code de Retour : Valeur d'`errno` en cas de problème / 0 si succès.

Je vous rappelle que **seul le propriétaire d'un verrou peut procéder à sa libération**.



Ne confondez pas `pthread_mutex_unlock` avec `pthread_mutex_destroy` !

Exercice 66 (⋆)

— Une variable partagée entre des threads et lue/écrite par chacun des threads peut-elle être considérée comme de la section critique de code ?
— Un mécanisme de protection est-il indispensable ?

Exercice 67 (⋆)

Une variable partagée entre des threads où un seul thread lit la variable tandis que les autres la modifient peut-elle être considérée comme de la section critique de code (au niveau **des** écritures et de **la** lecture) ?

Exercice 68 (⋆)

Écrivez un programme où le processus crée n threads qui se partagent un tableau de m entiers (ce tableau pourra être une variable globale du programme pour simplifier les choses) et qui calculent le minimum du tableau (ce minimum pourra également être stocké dans une variable globale du programme).

Prise de verrou et *async-signal-safety*

La fonction `pthread_mutex_lock` fait partie de la catégorie des fonctions *MT-safe*, mais pas *async-signal-safe*. La propriété *MT-safe* est triviale, puisque il est évident que cette fonction doit pouvoir être appelée par plusieurs threads simultanément : si cela n'était pas le cas, ce mécanisme de synchronisation n'existerait même pas.

En revanche, pourquoi cette fonction n'est-elle pas *async-signal-safe* ? Imaginons un instant que nous mettions en place un traitant de signal comportant une section critique de code partagée avec le `main` :

```
1 pthread_mutex_t verrou = PTHREAD_MUTEX_INITIALIZER ;
```

```
2
3  void  traitant(int sig)
4  {
5    pthread_mutex_lock(&verrou);
6
7    /* Section Critique */
8
9    pthread_mutex_unlock(&verrou);
10 }
11
12 int main(int argc, char *argv[])
13 {
14   pthread_mutex_lock(&verrou);
15
16   /* Section Critique */
17
18   pthread_mutex_unlock(&verrou);
19
20   exit(EXIT_SUCCESS);
21 }
```

Que se passerait-il si jamais le processus recevait le signal **après avoir fait l'acquisition du verrou, en pleine section critique, mais avant de le libérer** ? À ce moment, le thread exécutant le `main`, qui est **propriétaire** du verrou, serait interrompu et le traitant exécuté. Je vous rappelle qu'on ne revient dans le `main` qu'une fois que l'exécution du traitant est achevée. Or, on essaye de prendre le verrou dans ce traitant, et ce verrou n'est pas libre. Résultat : le traitant est bloqué dans le `pthread_mutex_lock` et par voie de conséquence, le processus tout entier. `pthread_mutex_lock` n'est qu'un cas particulier d'appel (potentiellement) bloquant, dont je vous rappelle que l'utilisation dans les traitants de signaux doit faire l'objet d'une attention toute particulière (c.f Section 6.4.2).

8.2.3 Cas particulier des fichiers

Il existe des verrous particuliers pour les accès à des fichiers, et c'est la fonction `flock` qui sera utilisée dans ce cas :

```
int flock(int fd, int operation);
```

Paramètre 1 : (Index du) Descripteur de fichier pour lequel un flot d'exécution souhaite faire l'acquisition d'un verrou.

Paramètre 2 : Opération(s) à effectuer pour le verrou de fichier.

- `LOCK_SH` : Acquisition d'un verrou en mode partagé (*shared*) ;
- `LOCK_EX` : Acquisition d'un verrou en mode exclusif ;
- `LOCK_UN` : Libération d'un verrou déjà acquis par le flot appelant ;
- `LOCK_NB` : Passage des opérations en mode non-bloquant (faire un `bitwise` OR).

Code de Retour : O en cas de succès/ -1 en cas d'échec avec `errno` positionné (peut être `EINTR`)

Un flot d'exécution peut posséder un verrou soit en mode partagé, soit en mode exclusif, mais pas les deux simultanément.



Les verrous créés avec `flock` sont en réalité positionnés sur les descripteurs de fichiers (accessibles via leur index) et pas sur le fichier lui-même.



Les verrous sur les (descripteurs de) fichiers sont conservés après un recouvrement (appel à `exec`).



Il est possible de placer des verrous sur des *portions* de fichiers avec `fcntl` (voir le manuel à ce sujet, notamment pour voir les différences avec les verrous créés par `flock`).

8.2.4 Pour aller plus loin : retour sur la duplication de processus multithreadés

Maintenant que vous connaissez le fonctionnement d'un verrou, revenons un instant sur les problèmes potentiels liés aux verrous en cas de duplication de processus multithreadé (cf. Section 7.2.2).

Prise de verrou et duplication de processus

Soit un processus multithreadé comportant plusieurs threads qui accèdent à une section critique de code protégée par un verrou partagé entre tous ces threads. Supposons qu'un de ces threads accède à ladite section critique : pour ce faire, il devient propriétaire du verrou qui n'est désormais plus libre. Si un autre thread effectue un appel à `fork` **avant** d'essayer d'accéder à cette même section critique de code, mais **après** que l'autre thread y a accédé, alors le nouveau processus-enfant créé sera bloqué dès sa tentative d'accès à cette même section critique. En effet, lors de la duplication, un nouveau verrou est dupliqué dans le processus-enfant, dans l'état où se trouvait le verrou d'origine au moment du `fork` : c'est-à-dire dans l'état pris. Or, seul le thread ayant fait l'appel à `fork` est dupliqué dans le processus-enfant (qui est donc mono-threadé) : il essaye alors de prendre le verrou qui est déjà pris et se retrouve bloqué. Comme il s'agit de l'unique thread du processus, alors c'est tout le processus qui est bloqué.



Comme l'indique le manuel : ce comportement est une limite de l'implémentation actuelle et pourra se voir modifié dans une future version de la bibliothèque pthreads.

Appels de fonctions de bibliothèque consécutifs à un `fork`

Le cas précédent explique également pourquoi il est recommandé d'appeler des fonctions *async-signal-safe* après un `fork` dans un processus-enfant dupliqué à partir d'un processus-parent multithreadé : en effet, il est possible qu'en interne, de telles fonctions utilisent une (des) variable(s) globale(s) protégée(s) par un verrou. Comme dans le cas précédent, si un thread effectue l'appel à la fonction et devient propriétaire du verrou et qu'un autre thread effectue un appel à `fork` **avant** d'appeler cette même fonction, alors le nouveau processus-enfant sera bloqué au moment de l'appel à cette fonction car le verrou interne sera déjà pris.

Une solution au problème ...

D'où vient le problème ? Tout simplement de la duplication d'un verrou déjà pris dans un processus monothreadé qui veut en faire l'acquisition. Or, puisqu'il n'y a qu'un unique thread dans ce processus, le notion de section critique est caduque et le verrou devrait être libre. Idéalement, il faudrait réinitialiser ce verrou dans le processus-enfant juste avant le retour de `fork`, de façon à ce que le thread du nouveau processus-enfant puisse en faire l'acquisition sans problème. Pour ce faire, il existe la fonction suivante :

```
int pthread_atfork(void (*prepare)(void),
.                  void (*parent)(void), void (*child)(void));
```

Paramètre 1 : Un pointeur sur une fonction exécutée par le processus-parent **avant** la création du processus-enfant (`NULL` est une valeur licite).
Paramètre 2 : Un pointeur sur une fonction exécutée par le **processus-parent** avant le retour de `fork` (`NULL` est une valeur licite).
Paramètre 3 : Un pointeur sur une fonction exécutée par le **processus-enfant** avant le retour de `fork` (`NULL` est une valeur licite).
Code de Retour : Valeur d'`errno` en cas de problème / 0 si succès.

Il est possible d'appeler plusieurs fois `pthread_atfork` pour installer de multiples traitants dans les processus. Les traitants `prepare` seront exécutés dans un ordre LIFO, tandis que les traitants `parent` et `child` seront exécutés en mode FIFO. Typiquement, pour résoudre le problème posé par la duplication d'un verrou déjà pris, il faudrait écrire un traitant `child` qui réinitialise le verrou avec `pthread_mutex_init` [5].

8.3 Les variables de condition

Le troisième mécanisme de synchronisation utilisé est la **variable de condition** (*condition variable*). Une variable de condition permet à des flots de se synchroniser par rapport à la réalisation d'une **condition logique**.

8.3.1 Fonctionnement théorique

Des flots d'exécution travaillent sur une condition logique (ex : `variable > 0`) : certains flots vont attendre (i.e. mis en sommeil) que la condition logique soit vérifiée, tandis que d'autres vont modifier la valeur de la variable, ce qui aura peut-être pour conséquence que cette condition se vérifie. Deux opérations sont applicables sur les variables de condition :

1. l'opération **signal** est effectuée par un flot quand **la condition est réalisée**. Un des flots en attente de la réalisation de cette condition est réveillé et peut continuer son exécution.
2. l'opération **wait** permet à un flot d'être **mis en sommeil** jusqu'à ce que la condition logique qu'il attend soit vérifée.



Une variable de condition ne **mémorise pas les appels à** `signal` émis par les flots. Par conséquent, si un flot appelle `signal` et qu'autre un flot (le « destinaire ») fait un appel à `wait` **après** (temporellement parlant), alors il sera **mis en sommeil** car il ne pourra pas prendre en compte le précédent appel à `signal`.

5. Et non pas un traitant qui libère le mutex, car le nouveau thread n'en n'est pas le propriétaire ...



Les variables de conditions sont souvent appelées des conditions (tout court). Attention à ne pas confondre une variable de condition (le moyen de la synchronisation) avec la condition logique (l'objectif de la synchronisation).

Exercice 69 (⋆)
Comment peut-on qualifier la condition logique utilisée ?

8.3.2 Interface pratique

Les variables de conditions sont implémentées dans la bibliothèque *pthread*, tout comme les verrous. Cela signifie que ce mécanisme sera plutôt utilisé entre les threads d'un processus, mais il est toujours possible de les utiliser entre threads de processus différents, pourvu que la variable de condition soit déclarée dans une zone de mémoire partagée. Une variable de condition est de type `pthread_cond_t` et est un objet opaque. Si d'un point de vue fonctionnel seules deux opérations sont pertinentes (`signal` et `wait`), plus de fonctions sont en réalité implémentées dans la bibliothèque.

Initialisation de la (variable de) condition

La première opération pratique sur les (variables de) condition est son initialisation. Évidemment il ne faut pas réinitialiser une (variable de) condition à chaque fois que vous l'utilisez. Voici le prototype de la fonction pthread_cond_init :

```
int pthread_cond_init(pthread_cond_t *cond,
.                     pthread_condattr_t *cond_attr);
```

Paramètre 1 : Un pointeur sur l'objet condition à initialiser.
Paramètre 2 : Un pointeur sur les attributs de la condition. Vous pouvez mettre `NULL` pour mettre les attributs par défaut [a].
Code de Retour : Valeur non nulle si problème (mais pas de code d'`errno`)/ 0 en cas de succès.

a. Ce que vous voulez très probablement. De plus, l'implémentation LinuxThreads ne supporte pas les attributs ...

Il est également possible d'initialiser statiquement une variable de condition avec la constante `PTHREAD_COND_INITIALIZER`. Par exemple :

```
pthread_cond_t condition = PTHREAD_COND_INITIALIZER;
```



De même que les verrous, il est possible de partager des variables de condition entre des threads appartenant à des processus distincts. Là encore, il faut déclarer ces variables de condition dans une zone de **mémoire partagée** (cf. Section 4.2). et positionner un attribut de partage de à l'aide de la fonction dédiée `pthread_condattr_setpshared`. Enfin, une variable de condition située dans une zone de mémoire partagée ne peut pas être placée n'importe où : son adresse doit être *alignée* (sur `alignof(pthread_cond_t)`).

Désallocation de la condition

l'opération duale de la précédente est `pthread_cond_destroy`, qui permet de désallouer les ressources associées à l'objet (variable de) condition alloué dans le programme. Son prototype est trivial :

```
int pthread_cond_destroy(pthread_cond_t *cond);
```

Paramètre : Un pointeur sur la (variable de) condition dont on libère les ressources.
Code de Retour : `errno == EBUSY` en cas de problème / 0 si succès.

Signalisation de la condition

Dans la bibliothèque *pthread*, il existe en fait deux fonctions qui permettent de signaler qu'une condition logique est vérifiée : `pthread_cond_signal` et `pthread_cond_broadcast`. Ces deux fonctions possèdent un prototype identique :

```
int pthread_cond_signal(pthread_cond_t *cond);
int pthread_cond_broadcast(pthread_cond_t *cond);
```

Paramètre : Un pointeur sur la structure de (variable de) condition à signaler.
Code de Retour : Valeur non nulle en cas de problème (mais pas de code d'`errno`)/ 0 si succès.

La différence entre ces deux fonctions est assez limitée : un appel à `pthread_cond_signal` ne va réveiller qu'**un seul** flot en attente de la réalisation de la condition logique, tandis qu'un appel à `pthread_cond_broadcast` va réveiller **tous** les flots en attente de cette réalisation.



Dans le cas de `pthread_cond_signal`, il n'est pas possible de choisir le flot qui sera réveillé.

Attente de la réalisation de la condition (logique)

```
int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex);
```

Paramètre 1 : Un pointeur sur la structure de (variable de) condition utilisée.
Paramètre 2 : Un pointeur sur la structure de verrou à libérer.
Code de Retour : Valeur non nulle en cas de problème (mais pas de code d'`errno`)/ 0 si succès.

La fonction `pthread_cond_wait` fonctionne en deux étapes :

1. Libération du verrou pris en argument (i.e. `pthread_mutex_unlock`) **puis** mise en sommeil du flot appelant. **Ces deux actions sont effectuées atomiquement**. Le flot mis en sommeil peut uniquement être réveillé par un **autre flot** qui appelle `pthread_cond_signal` ou `pthread_cond_broadcast`.
2. Réveil du flot et acquisition du verrou pris en argument (i.e. comme si un appel à `pthread_mutex_lock` était effectué). Deux cas se présentent :

Cas 1 : Si le verrou n'est pas libre actuellement (il est déjà pris par un autre flot qui travaille sur la condition logique), le flot reste en attente (sommeil) jusqu'à sa libération (par cet autre flot)

Cas 2 : Si le verrou est libre, le flot continue son exécution dans la section critique, avec une **garantie d'exclusion mutuelle**. Le flot peut donc modifier la condition logique de façon cohérente.

Le fonctionnement et le prototype de `pthread_cond_wait` sont un peu délicats à appréhender. Le second argument (le pointeur sur le verrou) est-il vraiment nécessaire ? Quel est son rôle exact ? Cette contraintre d'atomicité est-elle réellement indispensable ? Nous allons répondre à ces — légitimes — questions.

De la nécessité du verrou

Admettons qu'un flot souhaite attendre qu'une condition logique soit réalisée : ce flot va donc appeler la fonction `pthread_cond_wait` pour réaliser cette attente. Ce premier flot sera réveillé par un autre flot qui, après avoir réalisé la condition logique (ex. modification de la variable testée dans la condition logique), fera appel à la fonction `pthread_cond_signal` (ou `pthread_cond_broadcast`). Il faut se souvenir que cette condition logique est une **section critique**, qui doit par conséquent être **protégée**, notamment en garantissant son accès en **exclusion mutuelle**. Il est donc **indispensable d'utiliser un verrou conjointement** à une (variable de) condition pour assurer cette protection.

Du rôle du verrou

Puisque cette condition logique n'est pas réalisée (mais il est nécessaire de le tester pour s'en assurer) le flot appelle `pthread_cond_wait` et par conséquent libère le verrou (second argument de la fonction) puis est mis en sommeil. Cette libération est **indispensable**, car si le verrou n'était pas libéré, et que le flot actuel (propriétaire dudit verrou) était mis en sommeil, alors **aucun autre flot ne pourrait prendre le verrou**, et donc avoir la possibilité de modifier la condition logique. Par conséquent, aucun autre flot ne pourrait réveiller le flot en sommeil : il en résulterait une **situation d'interblocage**

De la contrainte d'atomicité

Les deux opérations de libération du verrou et de mise en sommeil du flot sont réalisées **atomiquement**. Mais pourquoi imposer une telle contrainte ? Raisonnons par l'absurde et supposons que ce ne soit pas le cas et examinons les conséquences. S'il n'y a pas d'atomicité, alors le flot appelant la fonction `pthread_cond_wait` peut tout-à-fait être interrompu (par un autre flot) entre la libération du verrou et sa mise en sommeil. Il est alors possible que cet autre flot rentre dans la section critique, modifie la condition logique (faisant en sorte qu'elle soit vérifiée) et signale cet état (avec un appel à `pthread_cond_signal`) **juste avant** que l'autre flot ne soit mis en sommeil. Ce dernier va donc reçevoir cette signalisation, mais comme cette opération n'est pas mémorisée (c.f la remarque en 8.3.1), il va s'endormir alors qu'il ne le devrait pas, **pour n'être jamais réveillé** ! Une nouvelle fois, une situation d'interblocage peut survenir.

8.3.3 Implémentation d'un moniteur par l'exemple

L'utilisation conjointe de ces deux mécanismes (verrou + variable de condition) permet d'implémenter un **moniteur**. Pratiquement, ce sont ces moniteurs que vous allez utiliser plutôt que les variables de condition. Pour mettre en place un moniteur, vous avez besoin des éléments suivants :

- une condition logique, c'est-à-dire des données partagées pour lesquelles cette condition sera vérifiée ou non.
- une (variable de) condition pour permettre d'attendre ou de signaler que la condition logique est vérifiée.
- un verrou pour garantir que les accès aux données partagées (objet de la condition logique) soient mutuellement exclusifs.

Maintenant, nous allons implémenter ce moniteur : soit une variable `toto` partagée entre plusieurs threads d'un même processus. Certains de ces threads vont modifier la valeur cette variable tandis que d'autres vont attendre que la valeur de cette variable soit au moins égale à 13 (par exemple) pour continuer leur exécution.

Voici les variables déclarées pour réaliser cette implémentation :

```
1 pthread_mutex_t mon_mutex = PTHREAD_MUTEX_INITIALIZER; //Pour
    le moniteur
```

```
2 pthread_cond_t  ma_cond    = PTHREAD_COND_INITIALIZER;  //Pour
     le moniteur
3 int             toto       = 2; // Variable pour la condition
     logique
```

Bien entendu, toutes ces variables sont globales pour le processus, donc partagées entre tous les threads.

Signalisation avec un moniteur

Comment effectuer la signalisation de la condition logique ? Le code aura la structure suivante :

```
1  void *fonction1(void *arg){
2   pthread_mutex_lock(&mon_mutex);
3
4   /* modification de toto ici
5     (toto++, toto--, etc.)   */
6
7
8   if(toto >= 13){
9    pthread_cond_broadcast(&ma_cond);
10  }
11
12  pthread_mutex_unlock(&mon_mutex);}
13
14  pthread_exit(NULL);
15 }
```

Déjà, n'oubliez pas que la variable `toto`, c'est-à-dire ce qui forme la fondement de la condition logique, est de la **section critique**, et doit donc être protégée par un verrou, d'où la prise du mutex `mon_mutex` tout au début. Vient ensuite la modification de la variable (éventuellement) puis son test pour savoir si la condition est vérifiée ou non (`if(toto >= 13)`). Rappelez-vous qu'un tel test est également de la section critique (c.f exo 8.2.2), donc **même sans modification de la variable, la prise de verrou est indispensable**. Si la condition logique est remplie, alors on peut le signaler aux flots en attente de réalisation de cette condition, avec un appel à `pthread_cond_signal` ou `pthread_cond_broadcast`. Enfin, la section critique étant finie, le flot peut libérer le verrou.

Attente avec un moniteur

Pour les flots en attente de réalisation de la condition logique, le code est le suivant :

```
1  void *fonction2(void *arg){
2   pthread_mutex_lock(&mon_mutex);
3
4   if(toto < 13){
5    pthread_cond_wait(&ma_cond, &mon_mutex);
6   }
7
8   /* Travail sur toto              */
9   /* modification, consultation    */
10  /* Toujours en section critique */
11
12  pthread_mutex_unlock(&mon_mutex);
13
```

```
14   pthread_exit(NULL);
15 }
```

Dans le cas de l'attente, le flot va forcément tester la valeur de la variable : ce test constitue donc une section critique et il faut prendre le verrou qui la protège. Ensuite, deux cas se présentent :

1. Si la condition logique est réalisée, alors le flot continue son exécution et peut travailler sur cette variable (encore de la section critique). Une fois ce travail terminé, le flot peut libérer le verrou pour laisser l'occasion aux autres flots d'accéder à cette section critique (aussi bien en attente qu'en modification).
2. Si la condition logique n'est pas réalisée (i.e. `toto < 13`), alors le flot doit attendre qu'elle se réalise : il va donc appeler la fonction `pthread_cond_wait` et être mis en sommeil. Lorsque ce flot sera réveillé (parce que la condition est réalisée) alors il pourra continuer son exécution, travailler en section critique et libérer le verrou à son tour, verrou qu'il aura pris dans la fonction `pthread_cond_wait` après son réveil et juste avant de sortir de la fonction.

Exercice 70 (⋆⋆⋆)
Cette implémentation de moniteur est malheureusement fausse.
— Où se trouve l'erreur ?
— Comment faut-il la corriger ?

Exercice 71 (⋆⋆⋆)
Implémentez des sémaphores avec des moniteurs : commencez par définir la structure de donnée adéquate puis implémentez les opérations `INIT`, `P()` et `V()`.

8.4 Les barrières de synchronisation

Le dernier mécanisme de synchronisation s'appelle la **barrière de synchronisation** : elle permet de bloquer l'exécution de flots jusqu'à ce qu'un nombre déterminé de flots atteigne un certain point dans leur exécution respective (i.e. un point de synchronisation).

8.4.1 Fonctionnement théorique

Une barrière de synchronisation possède un fonctionnement relativement simple : un flot rentrant dans ladite barrière y restera bloqué jusqu'à ce qu'un certain nombre d'autres flots (nombre défini au préalable) y rentrent à leur tour. Une fois que ce nombre est atteint, tous les flots qui étaient bloqués peuvent reprendre leur exécution. Il y a donc deux opérations :

1. opération **INIT** : cette opération consiste à initialiser la barrière, c'est à dire principalement lui indiquer le nombre de flots que l'on cherche à synchroniser ;
2. opération **WAIT** : quand un flot effectue cette opération, soit il reste bloqué car le nombre de flots entrés n'est pas encore égal au nombre fixé au moment de l'initialisation, soit il est le dernier à y entrer et dans ce cas, il va libérer tous les autres flots en attente et reprend son exécution normalement.

Exercice 72 (⋆⋆)
Implémentez une *barrière de synchronisation entre threads* à l'aide d'un moniteur. Pour se faire, il vous faudra :
— définir une structure de données `my_barrier_t`
— écrire la fonction d'initialisation :
```
int my_barrier_init(my_barrier_t *my_barrier, const int num_threads);
```
— écrire la fonction d'attente dans la barrière :
```
int my_barrier_wait(my_barrier_t *my_barrier);
```

Exercice 73 (⋆⋆⋆)
Dans l'exercice précédent, vous avez implémenté une barrière de synchronisation inter-threads avec un moniteur.
— Trouvez le problème posé par la solution basée sur un moniteur.
— Proposez un correctif. Est-il fonctionnel dans tous les cas ?

8.4.2 Interface pratique

Les barrières de synchronisation sont implémentées dans la bibliothèque *pthread* : ce sont des objets opaques de type `pthread_barrier_t`.

Initialisation

La première fonction est donc l'initialisation :

```
int pthread_barrier_init(pthread_barrier_t *restrict barrier,
.                        const pthread_barrierattr_t *restrict attr,
.                        unsigned count);
```

Paramètre 1 : Un pointeur sur la (structure de) barrière à initialiser.
Paramètre 2 : Un pointeur sur les attributs de la barrière : vous pouvez mettre `NULL` pour mettre les attributs par défaut (i.e. `PTHREAD_PROCESS_PRIVATE`, cf. ci-dessous).
Paramètre 3 : Le nombre de threads qui sont censés entrer dans cette barrière de synchronisation.
Code de Retour : Valeur d'`errno` en cas de problème / 0 si succès.

Le comportement de cette fonction est indéfini si :
— Un thread est toujours bloqué dans la barrière prise en argument ;
— La barrière prise en argument a déjà été initialisée.



Le pointeur sur l'objet barrière étant (sur)qualifié par le mot-clef `restrict`, il est *interdit* d'utiliser une copie de ce pointeur dans la fonction de synchronisation `pthread_barrier_wait` (résultat indéfini).



Les barrières sont implémentés dans la bibliothèque *pthread*, elles sont donc utilisables entre des threads d'un même processus (évidemment) mais également entre threads appartenant à des processus distincts. Dans ce cas, l'objet barrière doit être situé dans une zone de mémoire partagée (cf. Section 4.2). Il faut également positionner un attribut de partage de barrière à l'aide de la fonction `pthread_barrierattr_setpshared`. L'attribut prendra alors la valeur `PTHREAD_PROCESS_SHARED` (la valeur par défaut étant `PTHREAD_PROCESS_PRIVATE`). Enfin, une barrière de synchronisation située dans une zone de mémoire partagée ne peut pas être placée n'importe où : son adresse doit être *alignée* (sur `alignof(pthread_barrier_t)`).

Désallocation de la barrière

La fonction pour désallouer la structure associée à la barrière est la suivante :

```
int pthread_barrier_destroy(pthread_barrier_t *barrier);
```

Paramètre : Un pointeur sur la (structure de) barrière à libérer.
Code de Retour : Valeur d'`errno` en cas de problème / 0 si succès.

Attente/synchronisation des flots

La fonction permettant de synchroniser des flots est la suivante :

```
int pthread_barrier_wait(pthread_barrier_t *barrier);
```

Paramètre : Un pointeur sur la (structure de) barrière servant à synchroniser les flots appelants.
Code de Retour : En cas de succès, **un** flot (parmi tous ceux se synchronisant) retourne la valeur `PTHREAD_BARRIER_SERIAL_THREAD` tandis que les autres renvoient 0. Une valeur d'`errno` est retournée en cas de problème (excepté `EINTR`).



La fonction `pthread_barrier_wait` *doit* être appelée par autant de flots que le paramètre `count` de la fonction d'initialisation . Cependant, une barrière peut être utilisée plusieurs fois et des flots différents peuvent appeler la fonction d'attente à chaque séquence de synchronisation. La seule chose importante est que `count` appels soit effectués lors de chaque séquence.

Exercice 74 ($\star$)
Écrivez un programme dans lequel le thread principal crée 10 threads (qui affichent une chaîne de caractère par exemple) mais tous ces nouveaux threads doivent attendre que le dernier soit créé pour effectuer leur affichage.

8.5 Pour aller (vraiment) plus loin : synchronisation et variables atomiques

8.5.1 Rappels sur l'atomicité dans les systèmes d'exploitation

En programmation système (et dans le domaine des systèmes d'exploitation plus largement), la notion d'atomicité est importante. **Pour rappel, une opération peut être qualifiée d'*atomique* si l'exécution (ou le déroulement) de cette opération se fait intégralement ou pas du tout.** L'existence d'un état intermédiaire (pour cette opération) est donc impossible, ce qui permet d'avoir des garanties sur son résultat. Concrètement, au niveau du système d'exploitation, cette garantie s'exprime par le fait que si un flot commence à exécuter une opération qualifiée d'atomique, alors ce flot ne pourra pas être interrompu par le système afin qu'un *autre* flot puisse exécuter cette même opération.



Allégorie : élève de T2 témoin de l'exécution de son programme mal (voire non) synchronisé.

Exemples

1. nous avons vu que — sous certaines conditions — une écriture dans un tube de communication est atomique (cf. Section 4.1.4), ce qui a pour conséquence que les données provenant de multiples écrivains ne se mélangeront pas dans un tube partagé par tous ces écrivains. En effet, si un (flot) écrivain commence à écrire des données dans le tube, aucun autre (flot) écrivain ne pourra le faire tant que le premier flot n'aura pas terminé (ou bien s'il n'a rien écrit du tout).
2. une section critique de code protégée par un mécanisme d'exclusion mutuelle (par exemple un verrou) peut être qualifiée d'atomique car si thread commence à exécuter cette section critique, l'exclusion mutuelle garantit qu'aucun autre thread ne pourra accéder à cette section tant que le premier thread n'en sera pas sorti.

8.5.2 Fonctionnement théorique (cas du langage C11 et +)

Cette notion d'atomicité peut être appliquée aux données (i.e. aux variables) qui sont accédées dans un programme. Les opérations considérées sont alors les lectures (*load*) et les écritures (*store*) effectuées sur les variables. Donc, *a priori*, une définition de variable atomique serait la suivante : une variable ou une donnée *pourrait* être qualifiée d'atomique si ses accès (lecture et/ou écriture) sont des opérations atomiques.

Or, sur un système n bits, un *accès* (lecture ou écriture) à une variable de taille inférieure ou égale à n bits est atomique (à condition que la variable en question soit correctement **alignée**[6]). Par (contre-)exemple, un accès à une variable 64 bits sur un système 32 bits n'est pas atomique. En C, une variable entière (type `int`), est codée sur au moins 16 bits et selon le modèle choisi (LLP64, LP64, ILP64 ou SILP64), la longueur est soit de 32 bits, soit de 64 bits. Les processeurs actuels étant 64 bits, les accès sur une variable entière sont donc atomiques.

Maintenant, examinons l'exemple suivant :

```
1  #include <stdio.h>
2  #include <pthread.h>
3  #define NTHREADS 10
4
5  int x = 0;
6
7  void *thread(void *arg)
8  {
9    for(int i = 0 ; i < 1000 ; i++){
10     ++x;
11   }
12   return NULL;
13 }
14
15 int main(void)
16 {
17   pthread_t tid[NTHREADS];
18
19   for(int i = 0 ; i < NTHREADS ; i++)
20     pthread_create(&tid[i], NULL, thread, NULL);
21
22   for(int i = 0 ; i < NTHREADS ; i++)
```

6. C11 introduit de nouveaux opérateurs pour gérer de façon portable cet aspect, comme `alignof` par exemple.

```
23      pthread_join(tid[i], NULL);
24
25    printf("Compteur : x == %d\n", x);
26  }
```

L'exécution de ce programme ne donne pas systématiquement le résultat escompté :

```
mercierg@Palamede: ./exemple_atomicite_1
Compteur : x == 8000
mercierg@Palamede: ./exemple_atomicite_1
Compteur : x == 8241
mercierg@Palamede: ./exemple_atomicite_1
Compteur : x == 10000
mercierg@Palamede: ./exemple_atomicite_1
Compteur : x == 9000
```

Le problème vient du ++ qui n'est en réalité pas atomique car il implique deux opérations : une lecture puis une écriture (rappelez-vous que ++x est équivalent à `x = x + 1`). Si chacune de ces opérations est elle-même atomique (car x est une variable entière), la succession des ces opérations ne l'est — en revanche — pas. Un thread peut tout à fait lire et écrire dans la variable x alors qu'un autre thread a juste lu cette variable et pas encore écrit dedans. La donnée est donc dans un état incohérent à la fin de l'exécution du programme, d'où la valeur inattendue. Par ailleurs, qualifier la variable en `volatile` ne règle pas le problème.

Que se passe-t'il si l'on se contente d'effectuer des opérations atomiques sur une variable entière, comme dans l'exemple ci-dessous ? :

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <pthread.h>
5
6  int x = 0;
7
8  void *thread1(void *arg)
9  {
10     printf("Thread 1: Sleeping for 2 seconds\n");
11     sleep(2);
12
13     printf("Thread 1: Setting x to 666\n");
14     x = 666;
15
16     printf("Thread 1: Exiting\n");
17     return NULL;
18 }
19
20 void * thread2(void *arg)
21 {
22   printf("Thread 2: Waiting for 666\n");
23   while (x != 666);   // attente active
24
25   printf("Thread 2: Got 666 -- exiting!\n");
26   return NULL;
27 }
```

```
28
29 int main(void)
30 {
31   pthread_t t1, t2;
32
33   pthread_create(&t1, NULL, thread1, NULL);
34   pthread_create(&t2, NULL, thread2, NULL);
35
36   pthread_join(t1, NULL);
37   pthread_join(t2, NULL);
38
39   printf("Main: x == %d\n", x);
40 }
```

Le premier thread effectue une écriture atomique tandis que le second effectue des lectures atomiques elles aussi. L'exécution semble correcte :

```
mercierg@Palamede: ./exemple_atomicite_2
Thread 1: Sleeping for 2 seconds
Thread 2: Waiting for 666
Thread 1: Setting x to 666
Thread 1: Exiting
Thread 2: Got 666 - exiting!
Main: x == 666
```

Cependant, dès que l'on cherche à optimiser un peu le code (en compilant avec `-O1`, `-O2` ou `-O3` par exemple), alors l'exécution ne se passe plus aussi bien :

```
mercierg@Palamede: ./exemple_atomicite_2
Thread 1: Sleeping for 2 seconds
Thread 2: Waiting for 666
Thread 1: Setting x to 666
Thread 1: Exiting

(programme bloqué)
```

Le problème vient du fait que le compilateur optimise la boucle d'attente active à l'aide d'un registre ce qui fait que l'écriture de la variable dans le premier thread n'est pas vue dans le second. Dans ce cas, qualifier la variable `x` avec le mot-clef `volatile` permet de régler le problème.

Concrètement, avoir des accès atomiques n'est pas suffisant pour obtenir des programmes s'exécutant correctement. Il est nécessaire d'avoir des garanties concernant la **synchronisation inter-threads** et l'**ordre mémoire** (*memory ordering*). C'est pour cela que le mot-clef `_Atomic` (un qualificateur de type) a été introduit depuis C11. Ce mot-clef permet de contrôler plus finement ces aspects dans les programmes mais n'est pas sans conséquences sur les performances. Il est donc à utiliser à bon escient et avec parcimonie, après une analyse de votre code.



Le mot-clef `volatile` n'apporte aucune garantie en termes d'atomicité, de synchronisation inter-threads et d'ordre mémoire. Il ne peut pas être utilisé pour obtenir de tels effets.

8.5.2.1 Atomicité et synchronisation inter-threads

Nous allons maintenant préciser cette notion de synchronisation inter-threads dans le cas des variables atomiques. Deux threads (ou plus) se synchronisent quand il se mettent d'accord sur l'état de la mémoire qu'ils partagent. Utiliser des variables atomiques permet de mettre en place un tel consensus entre tous ces threads. Ce consensus peut s'exprimer ainsi :

> Dès qu'un thread écrit dans une variable atomique, tous les autres threads qui liront cette variable dans le futur verront les modifications effectuées par ce premier thread sur cette variable atomique **ainsi que toutes les modifications effectuées sur la mémoire avant et jusqu'à cette variable atomique (que ces modifications concernent des variables atomiques ou non importe peu).**

Voyons un exemple concret de ce fonctionnement. Considérons le pseudo-code suivant :

```
1  int x = 0;
2  int y = 0;
3
4  thread1(){
5    x = 2;
6    y = 3;
7  }
8
9  thread2(){
10   while(y !=3); //attente active sur y
11   printf("valeur de x == %i\n", x);
12 }
```

y étant une variable « classique », l'exécution de ce code ne permet pas de savoir avec certitude ce qui sera affiché. En effet, si le compilateur ou le CPU ne modifient pas l'ordre d'éxécution — en particulier l'affectation de x et y dans le premier thread — alors 2 sera affiché comme valeur de x. Maintenant, si cet ordre est modifié alors 0 sera affiché comme valeur de x.

Il est possible de forcer la synchronisation entre les deux threads en déclarant y comme *atomique* :

```
1          int x = 0;
2  _Atomic int y = 0; // y est maintenant atomique
3
4  thread1(){
5    x = 2;
6    y = 3;
7  }
8
9  thread2(){
10   while(y !=3); //attente active sur y
11   printf("valeur de x == %i\n", x);
12 }
```

Comme les threads se synchronisent sur y, toutes les écritures dans le premier thread qui se produisent **avant** l'écriture dans y sont visibles dans le second thread **après** la lecture de y. Dans ce cas, le seul résultat possible au niveau de l'affichage est 2.

En fait, la synchronisation se produit quand un thread lit une variable atomique écrite par un autre thread.



Cette synchronisation n'implique **aucun** blocage au niveau des threads. Ainsi, ce type de synchronisation fonctionne différemment des fonctions vues auparavant comme `wait`, `waitpid` `pthread_join` ou des mécanismes comme les sémaphores, les mutex et les moniteurs. C'est pour cette raison qu'une boucle d'attente active est présente dans l'exemple ci-dessus.

8.5.2.2 L'ordre mémoire *aquire/release*

En fait, ce type de synchronisation est la conséquence d'un **ordre mémoire** appelé *acquire/release* qu'il faut comprendre ainsi :

acquire = load = lecture
release = store = écriture

Dans ce contexte, quand un thread B lit (*acquire*) une valeur écrite (*release*) par un thread A, alors cette écriture du thread A synchronise la lecture dans le thread B. Toutes les écritures en mémoire dans le thread A (y compris les écritures non-atomiques) qui se produisent **avant** cette écriture atomique deviennent des effets de bord visibles dans le thread B. Plus précisément, une fois que l'écriture atomique a eu lieu dans le thread A, le thread B a la garantie de voir tout ce que le thread B a écrit en mémoire avant (et jusqu'à) cette écriture atomique. La synchronisation n'est établie qu'entre les threads qui libèrent (*release*) et acquièrent (*acquire*) **la même variable atomique**. Les autres threads (qui n'accèdent pas à cette variable) peuvent très bien constater un ordre mémoire différent de l'un ou des deux threads synchronisés.

Également, cet ordre mémoire impose des contraintes sur les accès mémoire autres que ceux concernant la variable atomique :

- ***release*** : aucune lecture ou écriture dans le thread courant ne peut être déplacée **après** cette écriture atomique (en revanche une lecture ou écriture placée après l'écriture atomique peut être déplacée avant).
- ***acquire*** : aucune lecture ou écriture dans le thread courant ne peut être déplacée **avant** cette lecture atomique (en revanche une lecture ou écriture placée avant la lecture atomique peut être déplacée après).

8.5.2.3 L'ordre mémoire de type cohérence séquentielle

En réalité, l'ordre mémoire mis en place par défaut dans le langage C est encore plus strict que l'ordre *acquire/release*. Cet ordre mémoire est appelé **cohérence séquentielle** (*sequential consistency*). Il impose un ordre total où tous les threads voient l'ensemble des modifications sur les variables atomiques dans le même ordre (là où l'ordre *acquire/release* ne concerne qu'une variable prise individuellement). Concrètement :

- les écritures sont des opérations *release* ;
- les lectures sont des opérations *acquire* ;
- les opérations de type lecture-modification-écriture effectuent un *acquire* puis un *release* ;
- Aucune opération atomique ne peut être réorganisée vis-à-vis des autres opérations atomiques dans le même thread (ce qui restait possible avec l'ordre *acquire/release*).

Ce dernier point peut être illustré par l'exemple suivant :

```
1         int a = 0;
2         int b = 0;
3         int c = 0;
4 _Atomic int x = 0;
5 _Atomic int y = 0;
6
7 thread1(){
```

```
8     a = 1;
9
10    b = 2;
11
12    x = 666; //release
13
14    while(y != 111); //acquire
15
16    c = 3;
17  }
18
19  thread2(){
20    y = 111; //release
21  }
```

Dans cet exemple, remonter le code en ligne 14 n'importe où avant dans le premier thread est compatible avec l'ordre *acquire/release* (vis-à-vis des autres variables `a`, `b` et `c`), mais pas avec la cohérence séquentielle. Cette dernière impose que les opérations sur `x` et `y` dans le premier thread ne soient pas inversées.

Il est possible de modifier l'ordre mémoire utilisé par le langage, y compris pour des opérations individuelles. Utiliser un ordre différent de la cohérence séquentielle est souvent motivé par des considérations de performance mais je vous déconseille de modifier cet ordre, à moins que vous ne sachiez *parfaitement* ce que vous êtes en train de faire.

8.5.3 Les variables atomiques en pratique

Plusieurs types de variables peuvent être considérés comme (plus ou moins) atomique :

- les variables déclarées avec le qualificateur de type `_Atomic` (disponible depuis C11). Les accès aux variables ainsi qualifiées sont atomiques. Les opérations combinées (*op*=, `++` et `--`) sont également atomiques sur ces variables. Ces variables permettent de bénéficier des garanties apportées par la cohérence séquentielle ;
- les variables de type `sig_atomic_t` sont atomiques mais uniquement pour les accès entre des traitants de signal et le flot principal du programme (`main`). **Ce niveau d'atomicité est donc trop « faible » pour pouvoir être utilisé dans le cas d'accès effectués par des threads.** car le type `sig_atomic_t` ne fournit aucune synchronisation inter-threads ni ordre mémoire, seuls des accès (lecture ou écriture) esont garantis atomiques [7] ;
- sur un système *n* bits, un *accès* à une variable de taille *n* bits ou moins est en général atomique (si la variable en question respecte son alignement). Cependant, les opérations combinées sur ces variables ne sont **pas** atomiques car une telle opération implique une lecture **et** une écriture et un thread pourra être interrompu entre ces deux sous-opérations. Enfin, si un thread écrit dans une telle variable, le résultat ne sera pas forcément visible par d'autres threads y accédant (à cause des manipulations de compilateur et de CPU, même si l'utilisation du qualificateur `volatile` permet éventuellement de régler certains problèmes).

L'utilisation du mot-clef `_Atomic` requiert d'inclure l'en-tête `<stdatomic.h>` et éventuellement de rajouter l'option `-latomic` à la compilation. Les règles d'emploi de ce qualificateur `_Atomic` sont les suivantes :

- il est utilisable avec n'importe quel type de base du langage C ;
- il est utilisable en conjonction avec les autres qualificateurs de type existants en C : `const`, `restrict` et `volatile` ;

7. Dans les systèmes Linux actuels, le type `sig_atomic_t` est d'ailleurs défini comme un entier (type `int`)

— il n'est pas utilisable avec une variable de type tableau ;
— il n'est pas utilisable avec une fonction, c'est-à-dire qu'une fonction ne peut pas être qualifiée d'`_Atomic`, mais son type de retour ainsi que les types de ses paramètres peuvent tout-à-fait être qualifiés d'`_Atomic` ;
— il est utilisable avec des structures (i.e. de type `struct`, cf. remarque ci-dessous) ;
— il est utilisable avec des unions (i.e. de type `union`, cf. remarque ci-dessous) ;
— il est utilisable avec (ou sur) des pointeurs. Attention à bien faire la distinction entre le pointeur et ce sur quoi il pointe. Le pointeur lui-même peut être qualifié d'`_Atomic` (ou non) et pointer sur une variable elle-même qualifiée d'`_Atomic` (ou non) [8].



Dans le cas des structures et des unions, il n'est pas possible d'accéder aux champs d'une structure ou d'une union déclarée `_Atomic`. Cependant il est possible de copier **atomiquement** le contenu d'une structure non-atomique dans une structure atomique. L'opération inverse (copier atomiquement le contenu d'une structure atomique dans une structure non-atomique) est aussi possible. Il en est de même pour les unions de type. Il faut donc obligatoirement passer par une structure ou une union *non-atomique* pour effectuer des modifications sur une structure ou une union atomique. Accèder à un champ d'une structure ou union atomique aboutit à un comportement non spécifié.

8.5.3.1 Implémentation des variables atomiques

Utiliser une variable atomique permet *a priori* de se passer d'un mécanisme effectif de synchronisation pour la consulter ou la modifier, comme un verrou par exemple. Cependant, le support des variables atomiques est limité à ce que le matériel (i.e le processeur) est capable de faire. Ainsi, il est possible que dans l'éventualité où le matériel ne possèderait pas intrinsèquement de support d'atomicité pour un type de variable donné, alors un verrou sera tout de même utilisé (mais de façon transparente pour l'utilisateur·trice) afin de garantir cette atomicité. La fonction `atomic_is_lock_free` permet de tester une variable d'un type particulier afin de savoir si ce type est nativement supporté. Il existe *a minima* un type qui est atomique et implémenté sans verrou (*lock-free*) : il s'agit du type opaque `atomic_flag`, qui doit être manipulé avec des macros. Les opérations supportées sont limitées, de type *test-and-set* et *clear*.



S'il est possible d'utiliser une variable atomique pour échanger des informations entre un traitant de signal et le `main` plutôt qu'une variable globale de type `volatile sig_atomic_t`, il est indispensable que cette variable soit *lock-free*. Dans le cas contraire, la variable ne serait pas *async-signal-safe*.

8.5.3.2 Barrières mémoire

Nous avons déjà rencontré la notion de barrière mémoire, notamment dans l'exercice d'échange de données entre processus via une zone de mémoire partagée avec l'utilisation d'une variable « sentinelle » pour indiquer la fin de l'écriture. Afin de garantir que l'ordre des opérations serait respecté (i.e. que l'écriture de la sentinelle serait bien effectuée *après* la copie de la donnée en mémoire partagée), la fonction `__sync_synchronize()` avait été utilisée. Cette barrière permet à la fois d'empêcher les modifications d'ordre de la part du compilateur et du processeur. Cependant, cette fonction n'est pas portable (il s'agit d'un built-in de gcc).

L'ordre *aquire/release* implique une forme de barrière mémoire entre les threads. Il est possible d'utiliser ce type de barrière sans avoir à effectuer de lecture ou d'écriture, avec la fonction portable

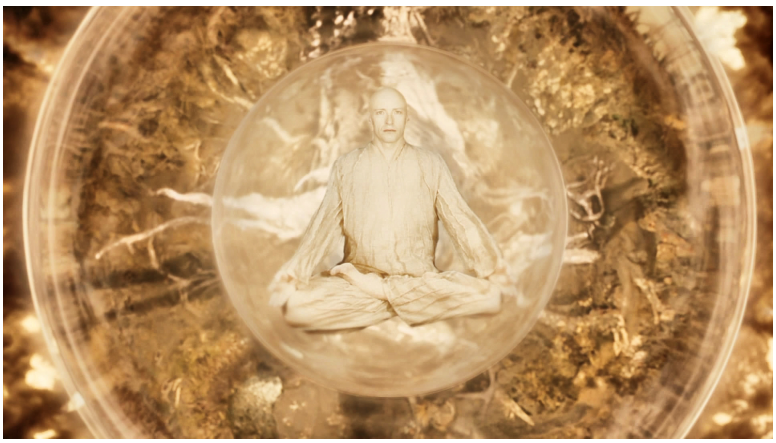
8. C'est exactement la même problématique que quand vous utilisez un autre qualificateur comme `volatile` ou `const`...

`atomic_thread_fence`.

À noter que dans l'exercice mentionné ci-dessus, une troisième solution (portable elle aussi) serait de qualifier la variable sentinelle en `_Atomic` dans les processus écrivain et lecteur.

8.6 Compléments et exercices

Retrouvez d'autres exercices sur la concurrence à cet url [3]. Je rappelle que [2] contient d'intéressants compléments pour mieux comprendre l'utilisation des mécanismes de synchronisation (rapidement) évoqués dans ce chapitre.



L'élève de T2, parangon de zénitude une fois le cours de Programmation Système achevé.



Références bibliographiques

[1] Achille (ou Jean-Pierre) BRAQUELAIRE. *Méthodologie de la programmation en C. Norme C99 - API POSIX*. 4e édition. Dunod, 2005 (cf. page 10).

[2] Allen B. DOWNEY. *The Little Book of Semaphores*. 2e édition. 2016. URL : `http://greenteapress.com/wp/semaphores/` (cf. pages 106, 131).

[3] Petr HUDEČEK et Michal POKORNÝ. *The Deadlock Empire*. 2016. URL : `http://deadlockempire.github.io/` (cf. page 131).

[4] Raymond NAMYST. *Cours de systèmes d'exploitation : synchronisations*. Pour la petite histoire, il s'agit de *mon* professeur de programmation système... URL : `https://raymond-namyst.emi.u-bordeaux.fr/ens/se/lectures/OS-Course-3.pdf` (cf. page 106).

[5] Jean-Marie RIFFLET et Jean-Baptiste YUNES. *UNIX : programmation et communication*. 1re édition. Dunod, 2003 (cf. page 10).



Notes de lecture



Index

A

appel-système 19
atomicité 57, 80, 81, 118, 123
attente active 47
attente passive 47

B

barrière de synchronisation 121

C

close 22

D

daemon 41
dirfd 38
dprintf 26
dup 27
dup2 28
dup3 29

E

exclusion mutuelle 81
execv 48

F

fclose 30
fcntl 23
fdopen 32
feof 30
fileno 31
flock 114
fopen 29
fork 38
fread 30
freopen 31
fseek 31
fstat 33
ftruncate 62
fwrite 30

G

getpid 37
getppid 41
gettid 95
granularité 93

I

I-nœud 16

K

kill 74

L

link 20

lseek 26
lstat 32

M

mkfifo 55
mmap 60
moniteur 119
msync 61
munmap 61
mutex 110
mémoire partagée 59

N

nombre de lecteurs 53
nombre d'écrivains 54

O

open 20
opendir 38

P

page-mémoire 59
parallélisation 93
pause 76
pclose 58
pidfd_getfd 38
pidfd_open 37
pidfd_send_signal 75
pipe 54
pipe2 55
popen 58
pread 26
processus 35
processus légers 91
pseudo-type 23, 37
pthread_atfork 116
pthread_attr_setdetachstate 99
pthread_barrier_destroy 123
pthread_barrier_init 122, 123
pthread_barrier_wait 123
pthread_cancel 100
pthread_cond_broadcast 118
pthread_cond_destroy 118
pthread_cond_init 117
pthread_cond_signal 118
pthread_cond_wait 118
pthread_create 95
pthread_detach 99
pthread_equal 94
pthread_exit 97
pthread_join 98
pthread_kill 103
pthread_mutex_destroy 112
pthread_mutex_init 112
pthread_mutex_lock 113
pthread_mutex_trylock 113
pthread_mutex_unlock 113
pthread_self 94
pthread_sigmask 104
pthread_sigqueue 104
pwrite 26

R

raise 74
read 23
redirections 27, 51, 53
réentrance 101

S

sched_yield 48
section critique 81
sem_destroy 109
sem_init 108
sem_open 108
sem_post 109
sem_trywait 109
sem_unlink 108
sem_wait 109
setsid 41
shm_open 64
shm_unlink 64
sigaction 78
sigaddset 83
sigdelset 83
sigemptyset 83
sigfillset 83
sigismember 83
signalfd 88
sigpending 85
sigprocmask 84
sigqueue 75
sigsuspend 84
sigwait 86
sigwaitinfo 87
sleep 47

stat 32
synchronisation . . 43, 66, 91, 92, 98, 102, 105
system 52
sémaphore 106

T

tgkill 103
thread 91
tube 51
tubes 53

U

unlink 23

V

variable de condition 116
verrou 110

W

wait 44
waitpid 45
write 23

Z

zombie 46